Workgroup:	BiDirectional or Server-Initiated HTTP
Internet-Draft:	WAMP
Published:	3 November 2026
Intended Status:	Experimental
Expires:	7 May 2027
Author:	T. Oberstein *typedef int GmbH*

The Web Application Messaging Protocol

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of BCP 78 and BCP 79.

Internet-Drafts are working documents of the Internet Engineering Task Force (IETF). Note that other groups may also distribute working documents as Internet-Drafts. The list of current Internet-Drafts is at https://datatracker.ietf.org/drafts/current/.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as "work in progress."

This Internet-Draft will expire on 7 May 2027.

Copyright Notice

Copyright (c) 2026 IETF Trust and the persons identified as the document authors. All rights reserved.

This document is subject to BCP 78 and the IETF Trust's Legal Provisions Relating to IETF Documents (https://trustee.ietf.org/license-info) in effect on the date of publication of this document. Please review these documents carefully, as they describe your rights and restrictions with respect to this document. Code Components extracted from this document must include Revised BSD License text as described in Section 4.e of the Trust Legal Provisions and are provided without warranty as described in the Revised BSD License.

Table of Contents

1. WAMP Basic Profile 8
1.1. Basic vs Advanced Profile 8
1.2. Introduction 8
1.3. Protocol Overview 9
1.3.1. Realms, Sessions and Transports 9
1.3.2. Peers and Roles 9
1.3.3. Publish & Subscribe 10
1.3.4. Remote Procedure Calls 11
1.4. Design Aspects 11
1.4.1. Application Code 11
1.4.2. Language Agnostic 12
1.4.3. Symmetric Messaging 12
1.4.4. Peers with multiple Roles 12
1.4.5. Relationship to WebSocket 12
2. Building Blocks 12
2.1. Identifiers 13
2.1.1. URIs 13
2.1.2. IDs 14
2.2. Serializers 15
2.3. Transports 16
2.3.1. WebSocket Transport 16
2.3.2. Transport and Session Lifetime 17
2.3.3. Protocol Errors 18
3. Messages 19
3.1. Extensibility 20
3.2. No Polymorphism 20
3.3. Structure 20

3.4. Message Definitions 20
3.4.1. Session Lifecycle 21
3.4.2. Publish & Subscribe 21
3.4.3. Routed Remote Procedure Calls 23
3.5. Message Codes and Direction 24
3.6. Extension Messages 26
3.7. Empty Arguments and Keyword Arguments 26
4. Sessions 26
4.1. Session Establishment 28
4.1.1. HELLO 28
4.1.2. WELCOME 30
4.1.3. ABORT 31
4.2. Session Closing 33
4.2.1. GOODBYE 33
5. Publish and Subscribe 34
5.1. Subscribing and Unsubscribing 34
5.1.1. SUBSCRIBE 35
5.1.2. SUBSCRIBED 36
5.1.3. Subscribe ERROR 36
5.1.4. UNSUBSCRIBE 37
5.1.5. UNSUBSCRIBED 37
5.1.6. Unsubscribe ERROR 38
5.2. Publishing and Events 38
5.2.1. PUBLISH 38
5.2.2. PUBLISHED 39
5.2.3. Publish ERROR 40
5.2.4. EVENT 40
6. Remote Procedure Calls 41
6.1. Registering and Unregistering 42
6.1.1. REGISTER 42

6.1.2. REGISTERED 43
6.1.3. Register ERROR 43
6.1.4. UNREGISTER 43
6.1.5. UNREGISTERED 44
6.1.6. Unregister ERROR 44
6.2. Calling and Invocations 44
6.2.1. CALL 45
6.2.2. INVOCATION 46
6.2.3. YIELD 47
6.2.4. RESULT 48
6.2.5. Invocation ERROR 49
6.2.6. Call ERROR 50
6.3. Caller Leaving During an RPC Invocation 51
6.4. Callee Leaving During an RPC Invocation 52
7. Security Model 52
7.1. Ordering Guarantees 52
7.2. Transport Encryption and Integrity 53
7.3. Router Authentication 54
7.4. Client Authentication 54
7.5. Routers are trusted 54
8. Basic Profile URIs 54
9. WAMP Advanced Profile 57
9.1. Feature Announcement 57
9.2. Additional Messages 59
9.2.1. CHALLENGE 60
9.2.2. AUTHENTICATE 60
9.2.3. CANCEL 60
9.2.4. INTERRUPT 60

10. Meta API 60
10.1. Session Meta API 60
10.1.1. Events 62
10.1.2. Procedures 63
10.2. Registration Meta API 66
10.2.1. Events 68
10.2.2. Procedures 69
10.3. Subscriptions Meta API 72
10.3.1. Events 73
10.3.2. Procedures 74
11. Advanced RPC 77
11.1. Progressive Call Results 77
11.2. Progressive Call Invocations 85
11.3. Call Timeouts 94
11.4. Call Canceling 96
11.5. Call Re-Routing 98
11.6. Caller Identification 100
11.7. Call Trust Levels 101
11.8. Pattern-based Registrations 102
11.8.1. Prefix Matching 102
11.8.2. Wildcard Matching 103
11.8.3. Design Aspects 103
11.9. Shared Registration 106
11.9.1. Load Balancing 106
11.9.2. Hot Stand-By 107
11.10. Sharded Registration 107
11.10.1. "All" Calls 107
11.10.2. "Partitioned" Calls 107
11.11. Registration Revocation 107

12. Advanced PubSub 108
12.1. Subscriber Black- and Whitelisting 108
12.2. Publisher Exclusion 112
12.3. Publisher Identification 112
12.4. Publication Trust Levels 113
12.5. Pattern-based Subscription 114
12.5.1. Prefix Matching 115
12.5.2. Wildcard Matching 116
12.5.3. Design Aspects 117
12.6. Sharded Subscription 117
12.7. Event History 118
12.8. Event Retention 122
12.9. Subscription Revocation 123
12.10. Session Testament 123
13. Authentication Methods 125
13.1. Ticket-based Authentication 127
13.2. Challenge Response Authentication 129
13.3. Salted Challenge Response Authentication 132
13.4. Cryptosign-based Authentication 142
13.4.1. Client Authentication 144
13.4.2. TLS Channel Binding 150
13.4.3. Router Authentication 151
13.4.4. Trustroots and Certificates 152
13.4.5. Remote Attestation 161
13.4.6. Example Message Exchanges 162
13.5. Dynamic Authentication API 173
13.6. Authorization 173
14. Advanced Security Features 174
14.1. Payload Passthru Mode 174
14.2. Payload End-to-End Encryption 184

15. Advanced Transports and Serializers 184
15.1. RawSocket Transport 184
15.2. Message Batching 190
15.3. HTTP Longpoll Transport 191
15.4. Binary support in JSON 193
15.5. Multiplexed Transport 195
16. WAMP Interfaces 196
16.1. WAMP IDL 197
16.1.1. Application Payload Typing 197
16.1.2. WAMP IDL Attributes 199
16.1.3. WAMP Service Declaration 203
16.2. Interface Catalogs 204
16.2.1. Catalog Archive File 205
16.2.2. Catalog Metadata 205
16.2.3. Catalog Sharing and Publication 207
16.3. Interface Reflection 208
17. Router-to-Router Links 209
18. Advanced Profile URIs 210
18.1. Session Close 210
18.2. Authentication 210
18.3. Authorization 211
18.4. Remote Procedure Calls 211
18.5. Terminology 212
18.5.1. Fundamental 213
18.5.2. Authentication and Authorization (AA) 214
18.5.3. Remote Procedure Calls 215
18.5.4. Publish and Subscribe 215
19. IANA Considerations 215
20. Conformance Requirements 216
20.1. Terminology and Other Conventions 216

21. Contributors 216

22. Normative References 217

23. Informative References 217

Author's Address 217

1. WAMP Basic Profile

This document defines the Web Application Messaging Protocol (WAMP). WAMP is a routed protocol that provides two messaging patterns: Publish & Subscribe and routed Remote Procedure Calls. It is intended to connect application components in distributed applications. WAMP uses WebSocket as its default transport, but can be transmitted via any other protocol that allows for ordered, reliable, bi-directional, and message-oriented communications.

1.1. Basic vs Advanced Profile

This document first describes a Basic Profile for WAMP in its entirety, before describing an Advanced Profile which extends the basic functionality of WAMP.

The separation into Basic and Advanced Profiles is intended to extend the reach of the protocol. It allows implementations to start out with a minimal, yet operable and useful set of features, and to expand that set from there. It also allows implementations that are tailored for resource-constrained environments, where larger feature sets would not be possible. Here implementers can weigh between resource constraints and functionality requirements, then implement an optimal feature set for the circumstances.

Advanced Profile features are announced during session establishment, so that different implementations can adjust their interactions to fit the commonly supported feature set.

1.2. Introduction

This section is non-normative.

The WebSocket protocol brings bi-directional real-time connections to the browser. It defines an API at the message level, requiring users who want to use WebSocket connections in their applications to define their own semantics on top of it.

The Web Application Messaging Protocol (WAMP) is intended to provide application developers with the semantics they need to handle messaging between components in distributed applications.

WAMP was initially defined as a WebSocket sub-protocol, which provided Publish & Subscribe (PubSub) functionality as well as Remote Procedure Calls (RPC) for procedures implemented in a WAMP router. Feedback from implementers and users of this was included in a second version of the protocol which this document defines. Among the changes was that WAMP can now run over any transport which is message-oriented, ordered, reliable, and bi-directional.

WAMP is a routed protocol, with all components connecting to a *WAMP Router*, where the WAMP Router performs message routing between the components.

WAMP provides two messaging patterns: *Publish & Subscribe* and *routed Remote Procedure Calls*.

Publish & Subscribe (PubSub) is an established messaging pattern where a component, the *Subscriber*, informs the router that it wants to receive information on a topic (i.e., it subscribes to a topic). Another component, a *Publisher*, can then publish to this topic, and the router distributes events to all Subscribers.

Routed Remote Procedure Calls (RPCs) rely on the same sort of decoupling that is used by the Publish & Subscribe pattern. A component, the *Callee*, announces to the router that it provides a certain procedure, identified by a procedure name. Other components, *Callers*, can then call the procedure, with the router invoking the procedure on the Callee, receiving the procedure's result, and then forwarding this result back to the Caller. Routed RPCs differ from traditional client-server RPCs in that the router serves as an intermediary between the Caller and the Callee.

The decoupling in routed RPCs arises from the fact that the Caller is no longer required to have knowledge of the Callee; it merely needs to know the identifier of the procedure it wants to call. There is also no longer a need for a direct connection between the caller and the callee, since all traffic is routed. This enables the calling of procedures in components which are not reachable externally (e.g. on a NATted connection) but which can establish an outgoing connection to the WAMP router.

Combining these two patterns into a single protocol allows it to be used for the entire messaging requirements of an application, thus reducing technology stack complexity, as well as networking overheads.

1.3. Protocol Overview

This section is non-normative.

1.3.1. Realms, Sessions and Transports

A Realm is a WAMP routing and administrative domain, optionally protected by authentication and authorization. WAMP messages are only routed within a Realm.

A Session is a transient conversation between two Peers attached to a Realm and running over a Transport.

A Transport connects two WAMP Peers and provides a channel over which WAMP messages for a WAMP Session can flow in both directions.

WAMP can run over any Transport which is message-based, bidirectional, reliable and ordered.

The default transport for WAMP is WebSocket [RFC6455], where WAMP is an officially registered subprotocol.

1.3.2. Peers and Roles

A WAMP Session connects two Peers, a Client and a Router. Each WAMP Peer MUST implement one role, and MAY implement more roles.

A Client MAY implement any combination of the Roles:

- Callee
- Caller
- Publisher
- Subscriber

and a Router MAY implement either or both of the Roles:

- Dealer
- Broker

> This document describes WAMP as in client-to-router communication. Direct client-to-client communication is not supported by WAMP. Router-to-router communication MAY be defined by a specific router implementation.

A *Router* is a component which implements one or both of the Broker and Dealer roles. A *Client* is a component which implements any or all of the Subscriber, Publisher, Caller, or Callee roles.

WAMP *Connections* are established by Clients to a Router. Connections can use any *transport* that is message-based, ordered, reliable and bi-directional, with WebSocket as the default transport.

WAMP *Sessions* are established over a WAMP Connection. A WAMP Session is joined to a *Realm* on a Router. Routing occurs only between WAMP Sessions that have joined the same Realm.

The *WAMP Basic Profile* defines the parts of the protocol that are required to establish a WAMP connection, as well as for basic interactions between the four client and two router roles. WAMP implementations are required to implement the Basic Profile, at minimum.

The *WAMP Advanced Profile* defines additions to the Basic Profile which greatly extend the utility of WAMP in real-world applications. WAMP implementations may support any subset of the Advanced Profile features. They are required to announce those supported features during session establishment.

1.3.3. Publish & Subscribe

The Publish & Subscribe ("PubSub") messaging pattern involves peers of three different roles:

- Subscriber (Client)
- Publisher (Client)
- Broker (Router)

A Publisher publishes events to topics by providing the topic URI and any payload for the event. Subscribers of the topic will receive the event together with the event payload.

Subscribers subscribe to topics they are interested in with Brokers. Publishers initiate publication first at Brokers. Brokers route events incoming from Publishers to Subscribers that are subscribed to respective topics.

The Publisher and Subscriber will usually run application code, while the Broker works as a generic router for events decoupling Publishers from Subscribers.

1.3.4. Remote Procedure Calls

The (routed) Remote Procedure Call ("RPC") messaging pattern involves peers of three different roles:

- Callee (Client)
- Caller (Client)
- Dealer (Router)

A Caller issues calls to remote procedures by providing the procedure URI and any arguments for the call. The Callee will execute the procedure using the supplied arguments to the call and return the result of the call to the Caller.

Callees register procedures they provide with Dealers. Callers initiate procedure calls first to Dealers. Dealers route calls incoming from Callers to Callees implementing the procedure called, and route call results back from Callees to Callers.

The Caller and Callee will usually run application code, while the Dealer works as a generic router for remote procedure calls decoupling Callers and Callees.

1.4. Design Aspects

This section is non-normative.

WAMP was designed to be performant, safe and easy to implement. Its entire design was driven by a implement, get feedback, adjust cycle.

An initial version of the protocol was publicly released in March 2012. The intent was to gain insight through implementation and use, and integrate these into a second version of the protocol, where there would be no regard for compatibility between the two versions. Several interoperable, independent implementations were released, and feedback from the implementers and users was collected.

The second version of the protocol, which this RFC covers, integrates this feedback. Routed Remote Procedure Calls are one outcome of this, where the initial version of the protocol only allowed the calling of procedures provided by the router. Another, related outcome was the strict separation of routing and application logic.

While WAMP was originally developed to use WebSocket as a transport, with JSON for serialization, experience in the field revealed that other transports and serialization formats were better suited to some use cases. For instance, with the use of WAMP in the Internet of Things sphere, resource constraints play a much larger role than in the browser, so any reduction of resource usage in WAMP implementations counts. This lead to the decoupling of WAMP from any particular transport or serialization, with the establishment of minimum requirements for both.

1.4.1. Application Code

WAMP is designed for application code to run within Clients, i.e. *Peers* having the roles Callee, Caller, Publisher, and Subscriber.

Routers, i.e. Peers of the roles Brokers and Dealers are responsible for **generic call and event routing** and do not run application code.

This allows the transparent exchange of Broker and Dealer implementations without affecting the application and to distribute and deploy application components flexibly.

> Note that a **program** that implements, for instance, the Dealer role might at the same time implement, say, a built-in Callee. It is the Dealer and Broker that are generic, not the program.

1.4.2. Language Agnostic

WAMP is language agnostic, i.e. can be implemented in any programming language. At the level of arguments that may be part of a WAMP message, WAMP takes a 'superset of all' approach. WAMP implementations may support features of the implementing language for use in arguments, e.g. keyword arguments.

1.4.3. Symmetric Messaging

It is important to note that though the establishment of a Transport might have a inherent asymmetry (like a TCP client establishing a WebSocket connection to a server), and Clients establish WAMP sessions by attaching to Realms on Routers, WAMP itself is designed to be fully symmetric for application components.

After the transport and a session have been established, any application component may act as Caller, Callee, Publisher and Subscriber at the same time. And Routers provide the fabric on top of which WAMP runs a symmetric application messaging service.

1.4.4. Peers with multiple Roles

Note that Peers might implement more than one role: e.g. a Peer might act as Caller, Publisher and Subscriber at the same time. Another Peer might act as both a Broker and a Dealer.

1.4.5. Relationship to WebSocket

WAMP uses WebSocket as its default transport binding, and is a registered WebSocket subprotocol.

2. Building Blocks

WAMP is defined with respect to the following building blocks

1. Identifiers
2. Serializers
3. Transports

For each building block, WAMP only assumes a defined set of requirements, which allows to run WAMP variants with different concrete bindings.

2.1. Identifiers

2.1.1. URIs

WAMP needs to identify the following persistent resources:

1. Topics
2. Procedures
3. Errors

These are identified in WAMP using Uniform Resource Identifiers (URIs) [RFC3986] that MUST be Unicode strings.

When using JSON as WAMP serialization format, URIs (as other strings) are transmitted in UTF-8 [RFC3629] encoding.

Examples

- `com.myapp.mytopic1`
- `com.myapp.myprocedure1`
- `com.myapp.myerror1`

The URIs are understood to form a single, global, hierarchical namespace for WAMP. The namespace is unified for topics, procedures and errors, that is these different resource types do NOT have separate namespaces.

To avoid resource naming conflicts, the package naming convention from Java is used, where URIs SHOULD begin with (reversed) domain names owned by the organization defining the URI.

Relaxed/Loose URIs

URI components (the parts between two `.`s, the head part up to the first `.`, the tail part after the last `.`) MUST NOT contain a `.`, `#` or whitespace characters and MUST NOT be empty (zero-length strings).

The restriction not to allow `.` in component strings is due to the fact that `.` is used to separate components, and WAMP associates semantics with resource hierarchies, such as in pattern-based subscriptions that are part of the Advanced Profile. The restriction not to allow empty (zero-length) strings as components is due to the fact that this may be used to denote wildcard components with pattern-based subscriptions and registrations in the Advanced Profile. The character `#` is not allowed since this is reserved for internal use by Dealers and Brokers.

As an example, the following regular expression could be used in Python to check URIs according to the above rules, when **NO empty URI components are allowed**:

```
pattern = re.compile(r"^([^\s\.#]+\.)*([^\s\.#]+)$")
```

When **empty URI components are allowed** (which is the case for specific messages that are part of the Advanced Profile), this following regular expression can be used (shown used in Python):

```
pattern = re.compile(r"^(([^\s\.#]+\.)|\.)*([^\s\.#]+)?$")
```

Strict URIs

While the above rules MUST be followed, following a stricter URI rule is recommended: URI components SHOULD only contain lower-case letters, digits and `_`.

As an example, the following regular expression could be used in Python to check URIs according to the above rules, when **NO empty URI components are allowed**:

```
pattern = re.compile(r"^([0-9a-z_]+\.)*([0-9a-z_]+)$")
```

When **empty URI components are allowed**, which is the case for specific messages that are part of the Advanced Profile, the following regular expression can be used (shown in Python):

```
pattern = re.compile(r"^(([0-9a-z_]+\.)|\.)*([0-9a-z_]+)?$")
```

Following the suggested regular expression for **strict URIs** will make URI components valid identifiers in most languages (modulo URIs starting with a digit and language keywords) and the use of lower-case only will make those identifiers unique in languages that have case-insensitive identifiers. Following this suggestion can allow implementations to map topics, procedures and errors to the language environment in a completely transparent way.

Reserved URIs

Further, application URIs MUST NOT use `wamp` as a first URI component, since this is reserved for URIs predefined with the WAMP protocol itself.

Examples

- `wamp.error.not_authorized`
- `wamp.error.procedure_already_exists`

2.1.2. IDs

WAMP needs to identify the following ephemeral entities each in the scope noted:

1. Sessions (*global scope*)
2. Publications (*global scope*)
3. Subscriptions (*router scope*)
4. Registrations (*router scope*)
5. Requests (*session scope*)

These are identified in WAMP using IDs that are integers between (inclusive) **1** and $\mathbf{2^{53}}$ (9007199254740992):

- IDs in the *global scope* MUST be drawn *randomly* from a *uniform distribution* over the complete range [1, 2^53]
- IDs in the *router scope* CAN be chosen freely by the specific router implementation
- IDs in the *session scope* MUST be incremented by 1 beginning with 1 and wrapping to 1 after it reached 2^53 (for each direction - *Client-to-Router* and *Router-to-Client*) {#session_scope_id}

> The reason to choose the specific lower bound as 1 rather than 0 is that 0 is the null-like (falsy) value for many programming languages. The reason to choose the specific upper bound is that 2^53 is the largest integer such that this integer and *all* (positive) smaller integers can be represented exactly in IEEE-754 doubles. Some languages (e.g. JavaScript) use doubles as their sole number type. Most languages do have signed and unsigned 64-bit integer types that both can hold any value from the specified range.

The following is a complete list of usage of IDs in the three categories for all WAMP messages. For a full definition of these see messages section.

Global Scope IDs

- `WELCOME.Session`
- `PUBLISHED.Publication`
- `EVENT.Publication`

Router Scope IDs

- `EVENT.Subscription`
- `SUBSCRIBED.Subscription`
- `REGISTERED.Registration`
- `UNSUBSCRIBE.Subscription`
- `UNREGISTER.Registration`
- `INVOCATION.Registration`

Session Scope IDs {#session_scope_ids}

- `SUBSCRIBE.Request`
- `SUBSCRIBED.Request` (mirrored `SUBSCRIBE.Request`)
- `UNSUBSCRIBE.Request`
- `UNSUBSCRIBED.Request` (mirrored `UNSUBSCRIBE.Request`)
- `PUBLISH.Request`
- `PUBLISHED.Request` (mirrored `PUBLISH.Request`)
- `REGISTER.Request`
- `REGISTERED.Request` (mirrored `REGISTER.Request`)
- `UNREGISTER.Request`
- `UNREGISTERED.Request` (mirrored `UNREGISTER.Request`)
- `CALL.Request`
- `RESULT.Request` (mirrored `CALL.Request`)
- `CANCEL.Request` (mirrored `CALL.Request`)
- `INVOCATION.Request`
- `YIELD.Request` (mirrored `INVOCATION.Request`)
- `INTERRUPT.Request` (mirrored `INVOCATION.Request`)
- `ERROR.Request` (mirrored original request ID)

2.2. Serializers

WAMP is a message based protocol that requires serialization of messages to octet sequences to be sent out on the wire.

A message serialization format is assumed that (at least) provides the following types:

- `integer` (non-negative)
- `string` (UTF-8 encoded Unicode)
- `bool`
- `list`
- `dict` (with string keys)

> WAMP *itself* only uses the above types, e.g. it does not use the JSON data types `number` (non-integer) and `null`. The *application payloads* transmitted by WAMP (e.g. in call arguments or event payloads) may use other types a concrete serialization format supports.

There is no required serialization or set of serializations for WAMP implementations (but each implementation MUST, of course, implement at least one serialization format). Routers SHOULD implement more than one serialization format, enabling components using different kinds of serializations to connect to each other.

The WAMP Basic Profile defines the following bindings for message serialization:

1. JSON
2. MessagePack
3. CBOR

Other bindings for serialization may be defined in the WAMP Advanced Profile.

With JSON serialization, each WAMP message is serialized according to the JSON specification as described in [RFC7159].

Further, binary data follows a convention for conversion to JSON strings. For details see the Appendix.

With MessagePack serialization, each WAMP message is serialized according to the MessagePack specification.

Version 5 or later of MessagePack MUST BE used, since this version is able to differentiate between strings and binary values.

With CBOR serialization, each WAMP message is serialized according to the CBOR specification as described in [RFC8949].

2.3. Transports

WAMP assumes a transport with the following characteristics:

1. message-based
2. reliable
3. ordered
4. bidirectional (full-duplex)

There is no required transport or set of transports for WAMP implementations (but each implementation MUST, of course, implement at least one transport). Routers SHOULD implement more than one transport, enabling components using different kinds of transports to connect in an application.

2.3.1. WebSocket Transport

The default transport binding for WAMP is WebSocket ([RFC6455]).

In the Basic Profile, WAMP messages are transmitted as WebSocket messages: each WAMP message is transmitted as a separate WebSocket message (not WebSocket frame). The Advanced Profile may define other modes, e.g. a **batched mode** where multiple WAMP messages are transmitted via single WebSocket message.

The WAMP protocol MUST BE negotiated during the WebSocket opening handshake between Peers using the WebSocket subprotocol negotiation mechanism ([RFC6455] section 4).

WAMP uses the following WebSocket subprotocol identifiers (for unbatched modes):

- `wamp.2.json`
- `wamp.2.msgpack`
- `wamp.2.cbor`

With `wamp.2.json`, *all* WebSocket messages MUST BE of type **text** (UTF8 encoded payload) and use the JSON message serialization.

With `wamp.2.msgpack`, *all* WebSocket messages MUST BE of type **binary** and use the MessagePack message serialization.

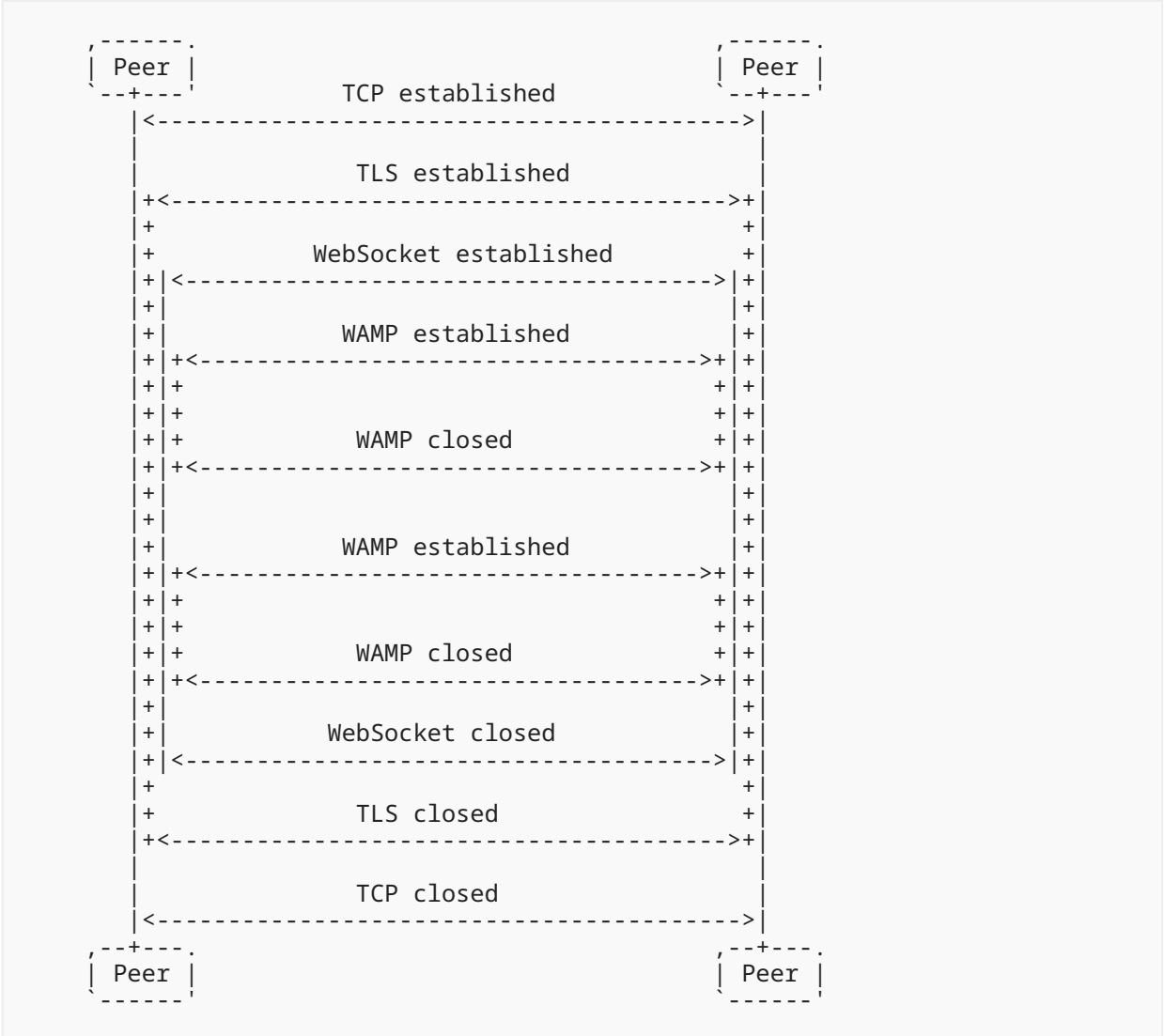
With `wamp.2.cbor`, *all* WebSocket messages MUST BE of type **binary** and use the CBOR message serialization.

> To avoid incompatibilities merely due to naming conflicts with WebSocket subprotocol identifiers, implementers SHOULD register identifiers for additional serialization formats with the official WebSocket subprotocol registry.

2.3.2. Transport and Session Lifetime

WAMP implementations MAY choose to tie the lifetime of the underlying transport connection for a WAMP connection to that of a WAMP session, i.e. establish a new transport-layer connection as part of each new session establishment. They MAY equally choose to allow re-use of a transport connection, allowing subsequent WAMP sessions to be established using the same transport connection.

The diagram below illustrates the full transport connection and session lifecycle for an implementation which uses WebSocket over TCP as the transport and allows the re-use of a transport connection.



2.3.3. Protocol Errors

WAMP implementations MUST abort sessions (disposing all of their resources such as subscriptions and registrations) on protocol errors caused by offending peers.

Following scenarios have to be considered protocol errors:

- Receiving `WELCOME` message, after session was established.
- Receiving `HELLO` message, after session was established.
- Receiving `CHALLENGE` message, after session was established.
- Receiving `GOODBYE` message, before session was established.
- Receiving `ERROR` message, before session was established.
- Receiving `ERROR` message with invalid REQUEST.Type.
- Receiving `SUBSCRIBED` message, before session was established.
- Receiving `UNSUBSCRIBED` message, before session was established.

- Receiving `PUBLISHED` message, before session was established.
- Receiving `RESULT` message, before session was established.
- Receiving `REGISTERED` message, before session was established.
- Receiving `UNREGISTERED` message, before session was established.
- Receiving `INVOCATION` message, before session was established.
- Receiving `YIELD` message with invalid INVOCATION.Request.
- Receiving message with non-sequential session scope request ID, such as `SUBSCRIBE`, `UNSUBSCRIBE`, `PUBLISH`, `REGISTER`, `UNREGISTER`, and `CALL`. Note that there are exeptions for `CALL` when the *Progressive Call Invocations* advanced feature is enabled. See the *Progressive Call Invocations* section in the advanced profile for details.
- Receiving protocol incompatible message, such as empty array, invalid WAMP message type id, etc.
- Catching error during message encoding/decoding.
- Any other exceptional scenario explicitly defined in any relevant section of this specification below (such as receiving a second `HELLO` within the lifetime of a session).

In all such cases WAMP implementations:

1. MUST send an `ABORT` message to the offending peer, having reason `wamp.error.protocol_violation` and optional attributes in ABORT.Details such as a human readable error message.
2. MUST abort the WAMP session by disposing any allocated subscriptions/registrations for that particular client and without waiting for or processing any messages subsequently received from the peer,
3. SHOULD also drop the WAMP connection at transport level (recommended to prevent denial of service attacks)

3. Messages

All WAMP messages are a `list` with a first element `MessageType` followed by one or more message type specific elements:

```
    [MessageType|integer, ... one or more message type specific
        elements ...]
```

The notation `Element|type` denotes a message element named `Element` of type `type`, where `type` is one of

- `uri`: a string URI as defined in URIs
- `id`: an integer ID as defined in IDs
- `integer`: a non-negative integer
- `string`: a Unicode string, including the empty string
- `bool`: a boolean value (`true` or `false`) - integers MUST NOT be used instead of boolean value
- `dict`: a dictionary (map) where keys MUST be strings, keys MUST be unique and serialization order is undefined (left to the serializer being used)

- `list`: a list (array) where items can be again any of this enumeration

Example

A `SUBSCRIBE` message has the following format

```
[SUBSCRIBE, Request|id, Options|dict, Topic|uri]
```

Here is an example message conforming to the above format

```
[32, 713845233, {}, "com.myapp.mytopic1"]
```

3.1. Extensibility

Some WAMP messages contain `Options|dict` or `Details|dict` elements. This allows for future extensibility and implementations that only provide subsets of functionality by ignoring unimplemented attributes. Keys in `Options` and `Details` MUST be of type `string` and MUST match the regular expression `[a-z][a-z0-9_]{2,}` for WAMP predefined keys. Implementations MAY use implementation-specific keys that MUST match the regular expression `_[a-z0-9_]{3,}`. Attributes unknown to an implementation MUST be ignored.

3.2. No Polymorphism

For a given `MessageType` and number of message elements the expected types are uniquely defined. Hence there are no polymorphic messages in WAMP. This leads to a message parsing and validation control flow that is efficient, simple to implement and simple to code for rigorous message format checking.

3.3. Structure

The application payload (that is call arguments, call results, event payload etc) is always at the end of the message element list. The rationale is: Brokers and Dealers have no need to inspect (parse) the application payload. Their business is call/event routing. Having the application payload at the end of the list allows Brokers and Dealers to skip parsing it altogether. This can improve efficiency and performance.

3.4. Message Definitions

WAMP defines the following messages that are explained in detail in the following sections.

The messages concerning the WAMP session itself are mandatory for all Peers, i.e. a Client MUST implement `HELLO`, `ABORT` and `GOODBYE`, while a Router MUST implement `WELCOME`, `ABORT` and `GOODBYE`.

All other messages are mandatory per role, i.e. in an implementation that only provides a Client with the role of Publisher MUST additionally implement sending `PUBLISH` and receiving `PUBLISHED` and `ERROR` messages.

3.4.1. Session Lifecycle

3.4.1.1. HELLO

Sent by a Client to initiate opening of a WAMP session to a Router attaching to a Realm.

```
    [HELLO, Realm|uri, Details|dict]
```

3.4.1.2. WELCOME

Sent by a Router to accept a Client. The WAMP session is now open.

```
    [WELCOME, Session|id, Details|dict]
```

3.4.1.3. ABORT

Sent by a Peer to abort the opening of a WAMP session. No response is expected.

```
    [ABORT, Details|dict, Reason|uri]

    [ABORT, Details|dict, Reason|uri, Arguments|list]

    [ABORT, Details|dict, Reason|uri, Arguments|list, ArgumentsKw|dict]
```

3.4.1.4. GOODBYE

Sent by a Peer to close a previously opened WAMP session. Must be echo'ed by the receiving Peer.

```
    [GOODBYE, Details|dict, Reason|uri]
```

3.4.1.5. ERROR

Error reply sent by a Peer as an error response to different kinds of requests.

```
    [ERROR, REQUEST.Type|int, REQUEST.Request|id, Details|dict, Error|uri]

    [ERROR, REQUEST.Type|int, REQUEST.Request|id, Details|dict, Error|uri,
        Arguments|list]

    [ERROR, REQUEST.Type|int, REQUEST.Request|id, Details|dict, Error|uri,
        Arguments|list, ArgumentsKw|dict]
```

3.4.2. Publish & Subscribe

3.4.2.1. PUBLISH

Sent by a Publisher to a Broker to publish an event.

```
    [PUBLISH, Request|id, Options|dict, Topic|uri]

    [PUBLISH, Request|id, Options|dict, Topic|uri, Arguments|list]

    [PUBLISH, Request|id, Options|dict, Topic|uri, Arguments|list,
        ArgumentsKw|dict]
```

3.4.2.2. PUBLISHED

Acknowledge sent by a Broker to a Publisher for acknowledged publications.

```
    [PUBLISHED, PUBLISH.Request|id, Publication|id]
```

3.4.2.3. SUBSCRIBE

Subscribe request sent by a Subscriber to a Broker to subscribe to a topic.

```
    [SUBSCRIBE, Request|id, Options|dict, Topic|uri]
```

3.4.2.4. SUBSCRIBED

Acknowledge sent by a Broker to a Subscriber to acknowledge a subscription.

```
    [SUBSCRIBED, SUBSCRIBE.Request|id, Subscription|id]
```

3.4.2.5. UNSUBSCRIBE

Unsubscribe request sent by a Subscriber to a Broker to unsubscribe a subscription.

```
    [UNSUBSCRIBE, Request|id, SUBSCRIBED.Subscription|id]
```

3.4.2.6. UNSUBSCRIBED

Acknowledge sent by a Broker to a Subscriber to acknowledge unsubscription.

```
    [UNSUBSCRIBED, UNSUBSCRIBE.Request|id]
```

3.4.2.7. EVENT

Event dispatched by Broker to Subscribers for subscriptions the event was matching.

```
    [EVENT, SUBSCRIBED.Subscription|id, PUBLISHED.Publication|id, Details|
dict]

    [EVENT, SUBSCRIBED.Subscription|id, PUBLISHED.Publication|id, Details|
dict,
        PUBLISH.Arguments|list]

    [EVENT, SUBSCRIBED.Subscription|id, PUBLISHED.Publication|id, Details|
dict,
        PUBLISH.Arguments|list, PUBLISH.ArgumentsKw|dict]
```

> An event is dispatched to a Subscriber for a given `Subscription|id` only once. On the other hand, a Subscriber that holds subscriptions with different `Subscription|ids` that all match a given event will receive the event on each matching subscription.

3.4.3. Routed Remote Procedure Calls

3.4.3.1. CALL

Call as originally issued by the Caller to the Dealer.

```
   [CALL, Request|id, Options|dict, Procedure|uri]

   [CALL, Request|id, Options|dict, Procedure|uri, Arguments|list]

   [CALL, Request|id, Options|dict, Procedure|uri, Arguments|list,
       ArgumentsKw|dict]
```

3.4.3.2. RESULT

Result of a call as returned by Dealer to Caller.

```
    [RESULT, CALL.Request|id, Details|dict]

    [RESULT, CALL.Request|id, Details|dict, YIELD.Arguments|list]

    [RESULT, CALL.Request|id, Details|dict, YIELD.Arguments|list,
        YIELD.ArgumentsKw|dict]
```

3.4.3.3. REGISTER

A Callees request to register an endpoint at a Dealer.

```
    [REGISTER, Request|id, Options|dict, Procedure|uri]
```

3.4.3.4. REGISTERED

Acknowledge sent by a Dealer to a Callee for successful registration.

```
    [REGISTERED, REGISTER.Request|id, Registration|id]
```

3.4.3.5. UNREGISTER

A Callees request to unregister a previously established registration.

```
    [UNREGISTER, Request|id, REGISTERED.Registration|id]
```

3.4.3.6. UNREGISTERED

Acknowledge sent by a Dealer to a Callee for successful unregistration.

```
    [UNREGISTERED, UNREGISTER.Request|id]
```

3.4.3.7. INVOCATION

Actual invocation of an endpoint sent by Dealer to a Callee.

```
    [INVOCATION, Request|id, REGISTERED.Registration|id, Details|dict]

    [INVOCATION, Request|id, REGISTERED.Registration|id, Details|dict,
        CALL.Arguments|list]

    [INVOCATION, Request|id, REGISTERED.Registration|id, Details|dict,
        CALL.Arguments|list, CALL.ArgumentsKw|dict]
```

3.4.3.8. YIELD

Actual yield from an endpoint sent by a Callee to Dealer.

```
    [YIELD, INVOCATION.Request|id, Options|dict]

    [YIELD, INVOCATION.Request|id, Options|dict, Arguments|list]

    [YIELD, INVOCATION.Request|id, Options|dict, Arguments|list, ArgumentsKw|
dict]
```

3.5. Message Codes and Direction

The following table lists the message type code for all messages defined in the WAMP basic profile and their direction between peer roles.

Reserved codes may be used to identify additional message types in future standards documents.

"Tx" indicates the message is sent by the respective role, and "Rx" indicates the message is received by the respective role.

Code	Message	Publisher	Broker	Subscriber	Caller	Dealer	Callee
1	HELLO	Tx	Rx	Tx	Tx	Rx	Tx
2	WELCOME	Rx	Tx	Rx	Rx	Tx	Rx
3	ABORT	TxRx	TxRx	TxRx	TxRx	TxRx	TxRx
6	GOODBYE	TxRx	TxRx	TxRx	TxRx	TxRx	TxRx
8	ERROR	Rx	Tx	Rx	Rx	TxRx	TxRx
16	PUBLISH	Tx	Rx				
17	PUBLISHED	Rx	Tx				
32	SUBSCRIBE		Rx	Tx			
33	SUBSCRIBED		Tx	Rx			
34	UNSUBSCRIBE		Rx	Tx			
35	UNSUBSCRIBED		Tx	Rx			
36	EVENT		Tx	Rx			
48	CALL				Tx	Rx	
50	RESULT				Rx	Tx	
64	REGISTER					Rx	Tx
65	REGISTERED					Tx	Rx
66	UNREGISTER					Rx	Tx
67	UNREGISTERED					Tx	Rx
68	INVOCATION					Tx	Rx
70	YIELD					Rx	Tx

Table 1

3.6. Extension Messages

WAMP uses type codes from the core range [0, 255]. Implementations MAY define and use implementation specific messages with message type codes from the extension message range [256, 1023]. For example, a router MAY implement router-to-router communication by using extension messages.

3.7. Empty Arguments and Keyword Arguments

Implementations SHOULD avoid sending empty `Arguments` lists.

E.g. a `CALL` message

```
    [CALL, Request|id, Options|dict, Procedure|uri, Arguments|list]
```

where `Arguments == []` SHOULD be avoided, and instead

```
    [CALL, Request|id, Options|dict, Procedure|uri]
```

SHOULD be sent.

Implementations SHOULD avoid sending empty `ArgumentsKw` dictionaries.

E.g. a `CALL` message

```
    [CALL, Request|id, Options|dict, Procedure|uri, Arguments|list,
ArgumentsKw|dict]
```

where `ArgumentsKw == {}` SHOULD be avoided, and instead

```
    [CALL, Request|id, Options|dict, Procedure|uri, Arguments|list]
```

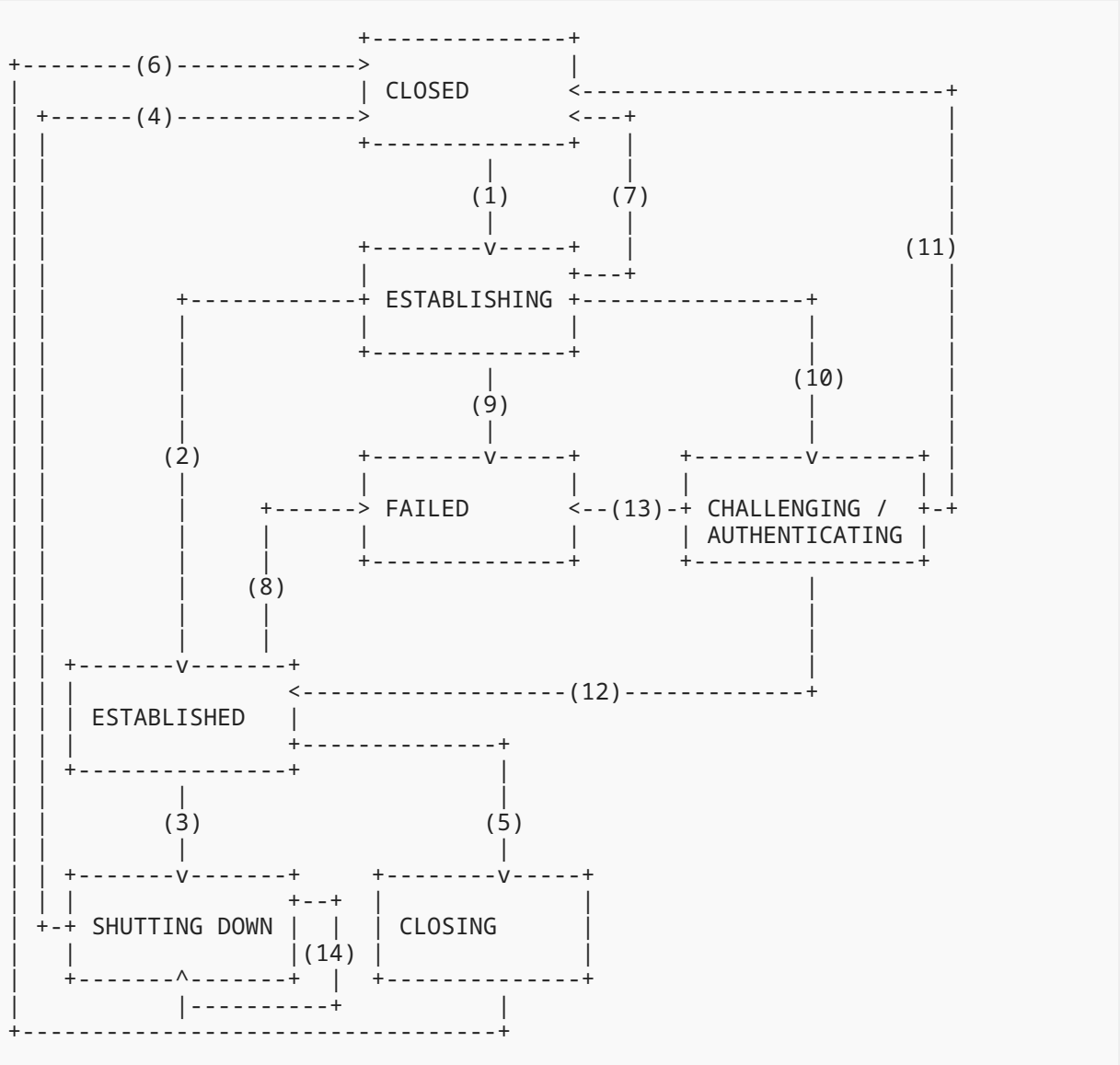
SHOULD be sent when `Arguments` is non-empty.

4. Sessions

The message flow between Clients and Routers for opening and closing WAMP sessions involves the following messages:

1. `HELLO`
2. `WELCOME`
3. `ABORT`
4. `GOODBYE`

The following state chart gives the states that a WAMP peer can be in during the session lifetime cycle.



The state transitions are listed in this table:

#	State
1	Sent HELLO
2	Received WELCOME
3	Sent GOODBYE
4	Received GOODBYE
5	Received GOODBYE
6	Sent GOODBYE
7	Received invalid HELLO / Send ABORT
8	Received HELLO or AUTHENTICATE
9	Received other
10	Received valid HELLO [needs authentication] / Send CHALLENGE
11	Received invalid AUTHENTICATE / Send ABORT
12	Received valid AUTHENTICATE / Send WELCOME
13	Received other
14	Received other / ignore

Table 2

4.1. Session Establishment

4.1.1. HELLO

After the underlying transport has been established, the opening of a WAMP session is initiated by the Client sending a `HELLO` message to the Router

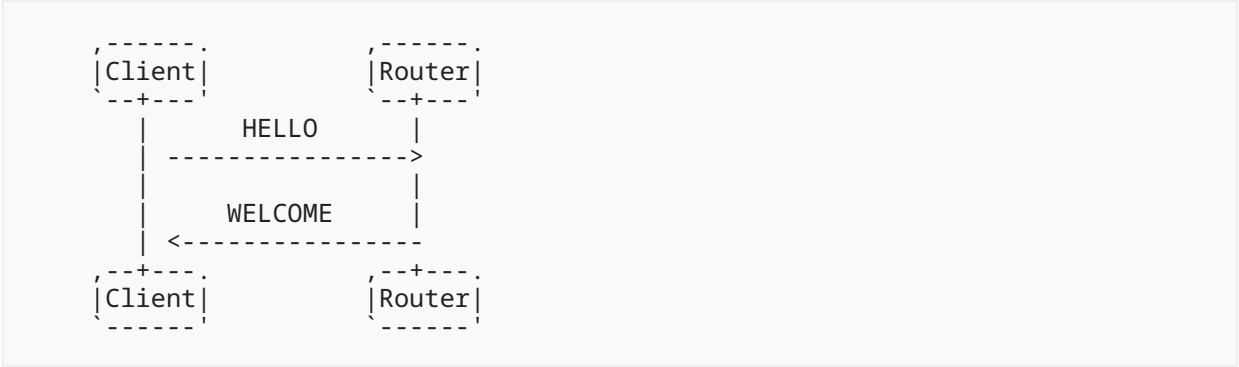
```
    [HELLO, Realm|uri, Details|dict]
```

where

- `Realm` is a string identifying the realm this session should attach to
- `Details` is a dictionary that allows to provide additional opening information (see below).

The `HELLO` message MUST be the very first message sent by the Client after the transport has been established.

In the WAMP Basic Profile without session authentication the Router will reply with a `WELCOME` or `ABORT` message.



A WAMP session starts its lifetime when the Router has sent a `WELCOME` message to the Client, and ends when the underlying transport closes or when the session is closed explicitly by either peer sending the `GOODBYE` message (see below).

It is a protocol error to receive a second `HELLO` message during the lifetime of the session and the Peer MUST close the session if that happens.

Client: Role and Feature Announcement

WAMP uses *Role & Feature announcement* instead of *protocol versioning* to allow

- implementations only supporting subsets of functionality
- future extensibility

A Client must announce the roles it supports via `Hello.Details.roles|dict`, with a key mapping to a `Hello.Details.roles.<role>|dict` where `<role>` can be:

- `publisher`
- `subscriber`
- `caller`
- `callee`

A Client can support any combination of the above roles but must support at least one role.

The `<role>|dict` is a dictionary describing features supported by the peer for that role.

This MUST be empty for WAMP Basic Profile implementations, and MUST be used by implementations implementing parts of the Advanced Profile to list the specific set of features they support.

Example: A Client that implements the Publisher and Subscriber roles of the WAMP Basic Profile.

```
[1, "somerealm", {
  "roles": {
      "publisher": {},
      "subscriber": {}
  }
}]
```

Client: Agent Identification

When a software agent operates in a network protocol, it often identifies itself, its application type, operating system, software vendor, or software revision, by submitting a characteristic identification string to its operating peer.

Similar to what browsers do with the `User-Agent` HTTP header, both the `HELLO` and the `WELCOME` message MAY disclose the WAMP implementation in use to its peer:

```
HELLO.Details.agent|string
```

and

```
WELCOME.Details.agent|string
```

Example: A Client "HELLO" message.

```
[1, "somerealm", {
     "agent": "AutobahnJS-0.9.14",
     "roles": {
        "subscriber": {},
        "publisher": {}
     }
}]
```

Example: A Router "WELCOME" message.

```
[2, 9129137332, {
    "agent": "Crossbar.io-0.10.11",
    "roles": {
      "broker": {}
    }
}]
```

4.1.2. WELCOME

A Router completes the opening of a WAMP session by sending a `WELCOME` reply message to the Client.

```
[WELCOME, Session|id, Details|dict]
```

where

- `Session` MUST be a randomly generated ID specific to the WAMP session. This applies for the lifetime of the session.
- `Details` is a dictionary that allows to provide additional information regarding the open session (see below).

In the WAMP Basic Profile without session authentication, a `WELCOME` message MUST be the first message sent by the Router, directly in response to a `HELLO` message received from the Client. Extensions in the Advanced Profile MAY include intermediate steps and messages for authentication.

> Note. The behavior if a requested `Realm` does not presently exist is router-specific. A router may e.g. automatically create the realm, or deny the establishment of the session with a `ABORT` reply message.

Router: Role and Feature Announcement

Similar to a Client announcing Roles and Features supported in the `HELLO` message, a Router announces its supported Roles and Features in the `WELCOME` message.

A Router MUST announce the roles it supports via `Welcome.Details.roles|dict`, with a key mapping to a `Welcome.Details.roles.<role>|dict` where `<role>` can be:

- `broker`
- `dealer`

A Router must support at least one role, and MAY support both roles.

The `<role>|dict` is a dictionary describing features supported by the peer for that role. With WAMP Basic Profile implementations, this MUST be empty, but MUST be used by implementations implementing parts of the Advanced Profile to list the specific set of features they support

Example: A Router implementing the Broker role of the WAMP Basic Profile.

```
[2, 9129137332, {
   "roles": {
      "broker": {}
   }
}]
```

4.1.3. ABORT

Both the Router and the Client may abort a WAMP session by sending an `ABORT` message.

```
[ABORT, Details|dict, Reason|uri]

[ABORT, Details|dict, Reason|uri, Arguments|list]

[ABORT, Details|dict, Reason|uri, Arguments|list, ArgumentsKw|dict]
```

where

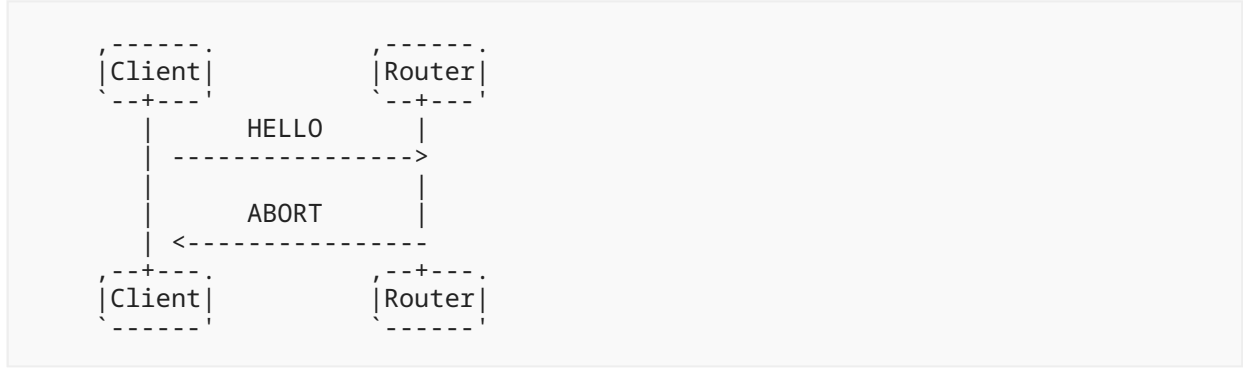
- `Reason` MUST be a URI.

- `Details` MUST be a dictionary that allows to provide additional, optional closing information (see below).

No response to an `ABORT` message is expected.

There are few scenarios, when `ABORT` is used:

- During session opening, if peer decided to abort connect.



Example

```
[3, {"message": "The realm does not exist."},
    "wamp.error.no_such_realm"]
```

- After session is opened, when protocol violation happens (see "Protocol errors" section).

Examples

- Router received second HELLO message.

```
[3, {"message":
        "Received HELLO message after session was established."},
    "wamp.error.protocol_violation"]
```

- Client peer received second WELCOME message

```
[3, {"message":
        "Received WELCOME message after session was established."},
    "wamp.error.protocol_violation"]
```

4.2. Session Closing

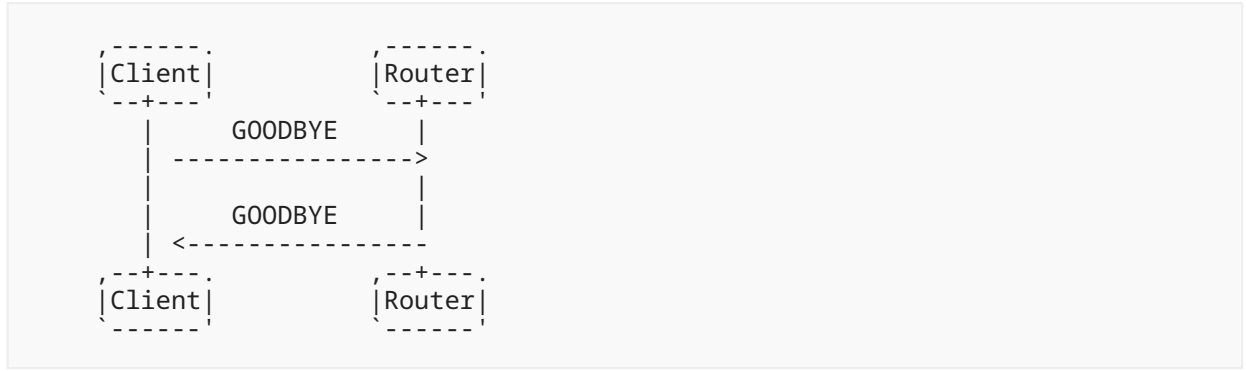
4.2.1. GOODBYE

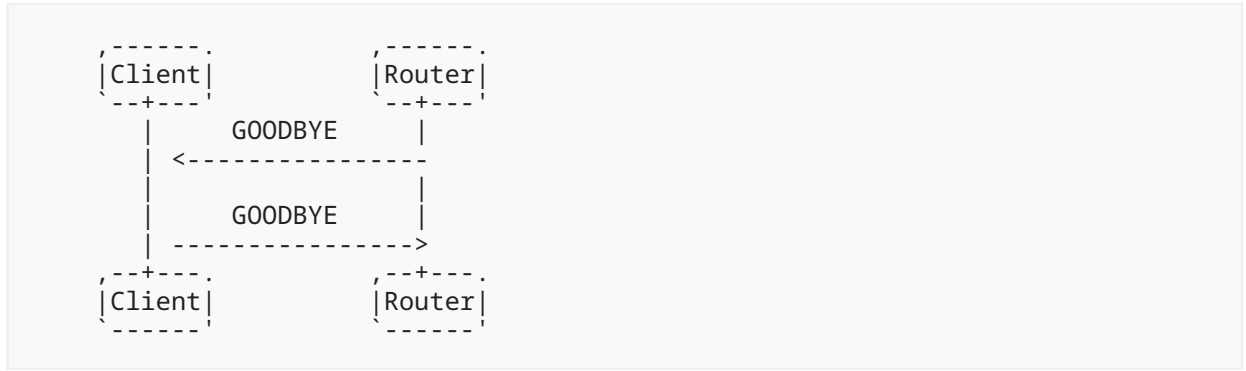
A WAMP session starts its lifetime with the Router sending a `WELCOME` message to the Client and ends when the underlying transport disappears or when the WAMP session is closed explicitly by a `GOODBYE` message sent by one Peer and a `GOODBYE` message sent from the other Peer in response.

```
[GOODBYE, Details|dict, Reason|uri]
```

where

- `Reason` MUST be a URI.
- `Details` MUST be a dictionary that allows to provide additional, optional closing information (see below).





Example. One Peer initiates closing

```
[6, {"message": "The host is shutting down now."},
    "wamp.close.system_shutdown"]
```

and the other peer replies

```
[6, {}, "wamp.close.goodbye_and_out"]
```

Example. One Peer initiates closing

```
[6, {}, "wamp.close.close_realm"]
```

and the other peer replies

```
[6, {}, "wamp.close.goodbye_and_out"]
```

Difference between ABORT and GOODBYE

The differences between `ABORT` and `GOODBYE` messages is that `ABORT` is never replied to by a Peer, whereas `GOODBYE` must be replied to by the receiving Peer.

> Though `ABORT` and `GOODBYE` are structurally identical, using different message types serves to reduce overloaded meaning of messages and simplify message handling code.

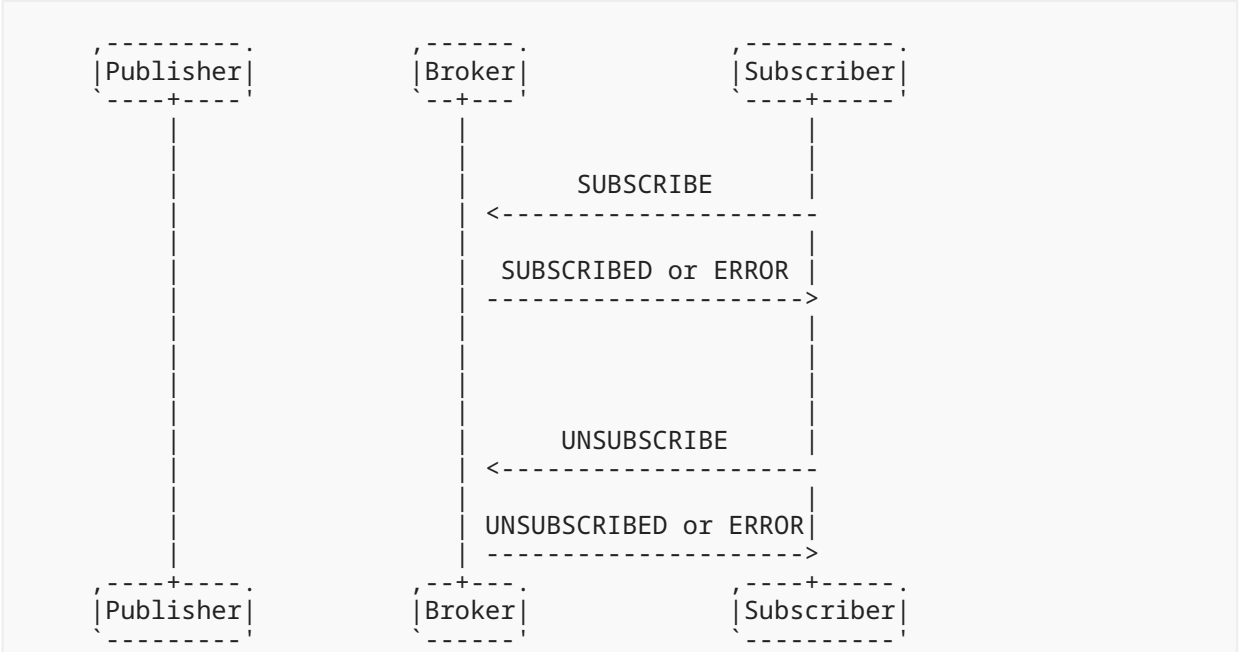
5. Publish and Subscribe

All of the following features for Publish & Subscribe are mandatory for WAMP Basic Profile implementations supporting the respective roles, i.e. *Publisher*, *Subscriber* and *Broker*.

5.1. Subscribing and Unsubscribing

The message flow between Clients implementing the role of Subscriber and Routers implementing the role of Broker for subscribing and unsubscribing involves the following messages:

1. `SUBSCRIBE`
2. `SUBSCRIBED`
3. `UNSUBSCRIBE`
4. `UNSUBSCRIBED`
5. `ERROR`



A Subscriber may subscribe to zero, one or more topics, and a Publisher publishes to topics without knowledge of subscribers.

Upon subscribing to a topic via the `SUBSCRIBE` message, a Subscriber will receive any future events published to the respective topic by Publishers, and will receive those events asynchronously.

A subscription lasts for the duration of a session, unless a Subscriber opts out from a previously established subscription via the `UNSUBSCRIBE` message.

> A Subscriber may have more than one event handler attached to the same subscription. This can be implemented in different ways: a) a Subscriber can recognize itself that it is already subscribed and just attach another handler to the subscription for incoming events, b) or it can send a new `SUBSCRIBE` message to broker (as it would be first) and upon receiving a `SUBSCRIBED.Subscription|id` it already knows about, attach the handler to the existing subscription

5.1.1. SUBSCRIBE

A Subscriber communicates its interest in a topic to a Broker by sending a `SUBSCRIBE` message:

```
[SUBSCRIBE, Request|id, Options|dict, Topic|uri]
```

where

- `Request` is a sequential ID in the *session scope*, incremented by the Subscriber and used to correlate the Broker's response with the request.
- `Options` is a dictionary that allows to provide additional subscription request details in a extensible way. This is described further below.
- `Topic` is the topic the Subscriber wants to subscribe to and is a URI.

Example

```
    [32, 713845233, {}, "com.myapp.mytopic1"]
```

A Broker, receiving a `SUBSCRIBE` message, can fullfill or reject the subscription, so it answers with `SUBSCRIBED` or `ERROR` messages.

5.1.2. SUBSCRIBED

If the Broker is able to fulfill and allow the subscription, it answers by sending a `SUBSCRIBED` message to the Subscriber

```
    [SUBSCRIBED, SUBSCRIBE.Request|id, Subscription|id]
```

where

- `SUBSCRIBE.Request` is the ID from the original subscription request.
- `Subscription` is an ID chosen by the Broker for the subscription.

Example

```
    [33, 713845233, 5512315355]
```

> Note. The `Subscription` ID chosen by the broker need not be unique to the subscription of a single Subscriber, but may be assigned to the `Topic`, or the combination of the `Topic` and some or all `Options`, such as the topic pattern matching method to be used. Then this ID may be sent to all Subscribers for the `Topic` or `Topic` / `Options` combination. This allows the Broker to serialize an event to be delivered only once for all actual receivers of the event.
>
> In case of receiving a `SUBSCRIBE` message from the same Subscriber and to already subscribed topic, Broker should answer with `SUBSCRIBED` message, containing the existing `Subscription|id`.

5.1.3. Subscribe ERROR

When the request for subscription cannot be fulfilled by the Broker, the Broker sends back an `ERROR` message to the Subscriber

```
[ERROR, SUBSCRIBE, SUBSCRIBE.Request|id, Details|dict, Error|uri]
```

where

- `SUBSCRIBE.Request` is the ID from the original request.
- `Error` is a URI that gives the error of why the request could not be fulfilled.

Example

```
[8, 32, 713845233, {}, "wamp.error.not_authorized"]
```

5.1.4. UNSUBSCRIBE

When a Subscriber is no longer interested in receiving events for a subscription it sends an `UNSUBSCRIBE` message

```
[UNSUBSCRIBE, Request|id, SUBSCRIBED.Subscription|id]
```

where

- `Request` is a sequential ID in the *session scope*, incremented by the Subscriber and used to correlate the Broker's response with the request.
- `SUBSCRIBED.Subscription` is the ID for the subscription to unsubscribe from, originally handed out by the Broker to the Subscriber.

Example

```
[34, 85346237, 5512315355]
```

5.1.5. UNSUBSCRIBED

Upon successful unsubscription, the Broker sends an `UNSUBSCRIBED` message to the Subscriber

```
[UNSUBSCRIBED, UNSUBSCRIBE.Request|id]
```

where

- `UNSUBSCRIBE.Request` is the ID from the original request.

Example

```
[35, 85346237]
```

5.1.6. Unsubscribe ERROR

When the request fails, the Broker sends an `ERROR`

```
[ERROR, UNSUBSCRIBE, UNSUBSCRIBE.Request|id, Details|dict, Error|uri]
```

where

- `UNSUBSCRIBE.Request` is the ID from the original request.
- `Error` is a URI that gives the error of why the request could not be fulfilled.

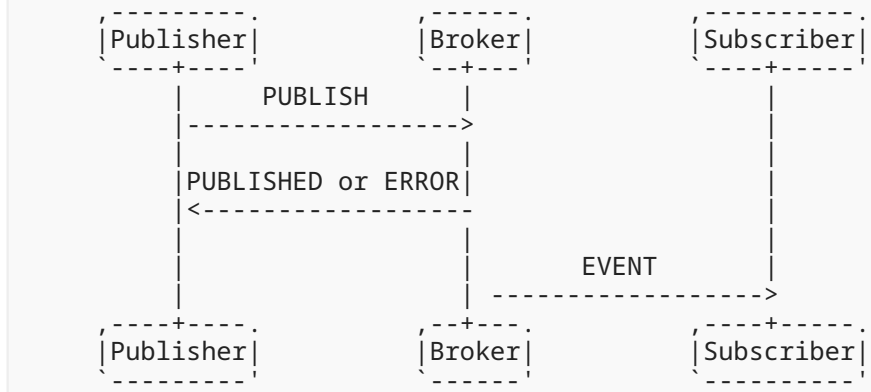
Example

```
[8, 34, 85346237, {}, "wamp.error.no_such_subscription"]
```

5.2. Publishing and Events

The message flow between Publishers, a Broker and Subscribers for publishing to topics and dispatching events involves the following messages:

1. `PUBLISH`
2. `PUBLISHED`
3. `EVENT`
4. `ERROR`



5.2.1. PUBLISH

When a Publisher requests to publish an event to some topic, it sends a `PUBLISH` message to a Broker:

```
[PUBLISH, Request|id, Options|dict, Topic|uri]
```

or

```
[PUBLISH, Request|id, Options|dict, Topic|uri, Arguments|list]
```

or

```
[PUBLISH, Request|id, Options|dict, Topic|uri, Arguments|list,
    ArgumentsKw|dict]
```

where

- `Request` is a sequential ID in the *session scope*, incremented by the Publisher and used to correlate the Broker's response with the request.
- `Options` is a dictionary that allows to provide additional publication request details in an extensible way. This is described further below.
- `Topic` is the topic published to.
- `Arguments` is a list of application-level event payload elements. The list may be of zero length.
- `ArgumentsKw` is an optional dictionary containing application-level event payload, provided as keyword arguments. The dictionary may be empty.

If the Broker allows and is able to fulfill the publication, the Broker will send the event to all current Subscribers of the topic of the published event.

By default, publications are unacknowledged, and the Broker will not respond, whether the publication was successful indeed or not. This behavior can be changed with the option `PUBLISH.Options.acknowledge|bool` (see below).

Example

```
[16, 239714735, {}, "com.myapp.mytopic1"]
```

Example

```
[16, 239714735, {}, "com.myapp.mytopic1", ["Hello, world!"]]
```

Example

```
[16, 239714735, {}, "com.myapp.mytopic1", [], {"color": "orange",
    "sizes": [23, 42, 7]}]
```

5.2.2. PUBLISHED

If the Broker is able to fulfill and allowing the publication, and `PUBLISH.Options.acknowledge == true`, the Broker replies by sending a `PUBLISHED` message to the Publisher:

```
    [PUBLISHED, PUBLISH.Request|id, Publication|id]
```

where

- `PUBLISH.Request` is the ID from the original publication request.
- `Publication` is an ID chosen by the Broker for the publication.

Example

```
    [17, 239714735, 4429313566]
```

5.2.3. Publish ERROR

When the request for publication cannot be fulfilled by the Broker, and `PUBLISH.Options.acknowledge == true`, the Broker sends back an `ERROR` message to the Publisher

```
    [ERROR, PUBLISH, PUBLISH.Request|id, Details|dict, Error|uri]
```

where

- `PUBLISH.Request` is the ID from the original publication request.
- `Error` is a URI that gives the error of why the request could not be fulfilled.

Example

```
    [8, 16, 239714735, {}, "wamp.error.not_authorized"]
```

5.2.4. EVENT

When a publication is successful and a Broker dispatches the event, it determines a list of receivers for the event based on Subscribers for the topic published to and, possibly, other information in the event.

Note that the Publisher of an event will never receive the published event even if the Publisher is also a Subscriber of the topic published to.

> The Advanced Profile provides options for more detailed control over publication.

When a Subscriber is deemed to be a receiver, the Broker sends the Subscriber an `EVENT` message:

```
    [EVENT, SUBSCRIBED.Subscription|id, PUBLISHED.Publication|id, Details|
dict]
```

or

```
    [EVENT, SUBSCRIBED.Subscription|id, PUBLISHED.Publication|id, Details|
dict,
        PUBLISH.Arguments|list]
```

or

```
    [EVENT, SUBSCRIBED.Subscription|id, PUBLISHED.Publication|id, Details|
dict,
        PUBLISH.Arguments|list, PUBLISH.ArgumentsKw|dict]
```

where

- `SUBSCRIBED.Subscription` is the ID for the subscription under which the Subscriber receives the event - the ID for the subscription originally handed out by the Broker to the Subscribe.
- `PUBLISHED.Publication` is the ID of the publication of the published event.
- `Details` is a dictionary that allows the Broker to provide additional event details in a extensible way. This is described further below.
- `PUBLISH.Arguments` is the application-level event payload that was provided with the original publication request.
- `PUBLISH.ArgumentsKw` is the application-level event payload that was provided with the original publication request.

Example

```
    [36, 5512315355, 4429313566, {}]
```

Example

```
    [36, 5512315355, 4429313566, {}, ["Hello, world!"]]
```

Example

```
    [36, 5512315355, 4429313566, {}, [], {"color": "orange", "sizes": [23,
42, 7]}]
```

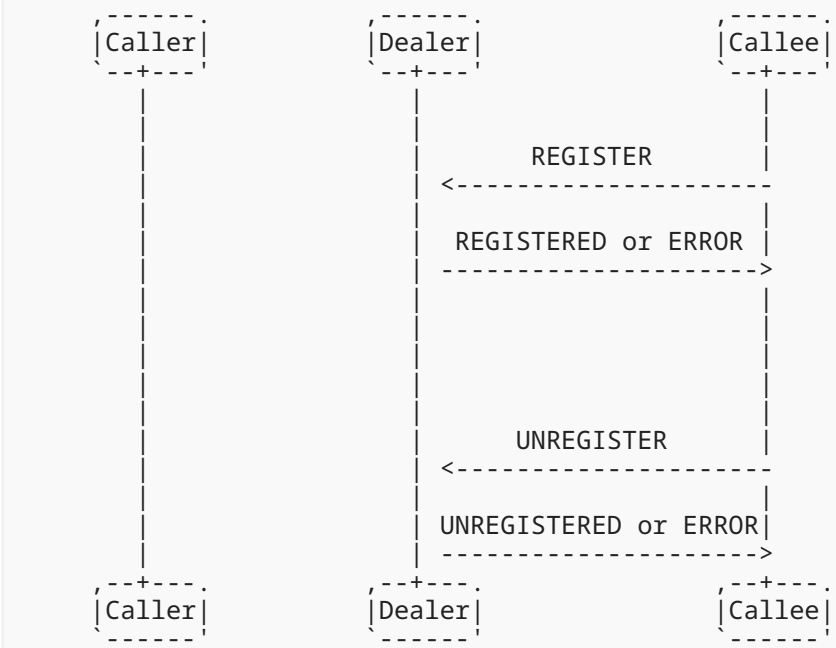
6. Remote Procedure Calls

All of the following features for Remote Procedure Calls are mandatory for WAMP Basic Profile implementations supporting the respective roles.

6.1. Registering and Unregistering

The message flow between Callees and a Dealer for registering and unregistering endpoints to be called over RPC involves the following messages:

1. `REGISTER`
2. `REGISTERED`
3. `UNREGISTER`
4. `UNREGISTERED`
5. `ERROR`



6.1.1. REGISTER

A Callee announces the availability of an endpoint implementing a procedure with a Dealer by sending a `REGISTER` message:

```
[REGISTER, Request|id, Options|dict, Procedure|uri]
```

where

- `Request` is a sequential ID in the *session scope*, incremented by the Callee and used to correlate the Dealer's response with the request.
- `Options` is a dictionary that allows to provide additional registration request details in a extensible way. This is described further below.
- `Procedure` is the procedure the Callee wants to register

Example

```
[64, 25349185, {}, "com.myapp.myprocedure1"]
```

6.1.2. REGISTERED

If the Dealer is able to fulfill and allowing the registration, it answers by sending a `REGISTERED` message to the `Callee`:

```
[REGISTERED, REGISTER.Request|id, Registration|id]
```

where

- `REGISTER.Request` is the ID from the original request.
- `Registration` is an ID chosen by the Dealer for the registration.

Example

```
[65, 25349185, 2103333224]
```

6.1.3. Register ERROR

When the request for registration cannot be fulfilled by the Dealer, the Dealer sends back an `ERROR` message to the Callee:

```
[ERROR, REGISTER, REGISTER.Request|id, Details|dict, Error|uri]
```

where

- `REGISTER.Request` is the ID from the original request.
- `Error` is a URI that gives the error of why the request could not be fulfilled.

Example

```
[8, 64, 25349185, {}, "wamp.error.procedure_already_exists"]
```

6.1.4. UNREGISTER

When a Callee is no longer willing to provide an implementation of the registered procedure, it sends an `UNREGISTER` message to the Dealer:

```
[UNREGISTER, Request|id, REGISTERED.Registration|id]
```

where

- `Request` is a sequential ID in the *session scope*, incremented by the Callee and used to correlate the Dealer's response with the request.
- `REGISTERED.Registration` is the ID for the registration to revoke, originally handed out by the Dealer to the Callee.

Example

```
[66, 788923562, 2103333224]
```

6.1.5. UNREGISTERED

Upon successful unregistration, the Dealer sends an `UNREGISTERED` message to the Callee:

```
[UNREGISTERED, UNREGISTER.Request|id]
```

where

- `UNREGISTER.Request` is the ID from the original request.

Example

```
[67, 788923562]
```

6.1.6. Unregister ERROR

When the unregistration request fails, the Dealer sends an `ERROR` message:

```
[ERROR, UNREGISTER, UNREGISTER.Request|id, Details|dict, Error|uri]
```

where

- `UNREGISTER.Request` is the ID from the original request.
- `Error` is a URI that gives the error of why the request could not be fulfilled.

Example

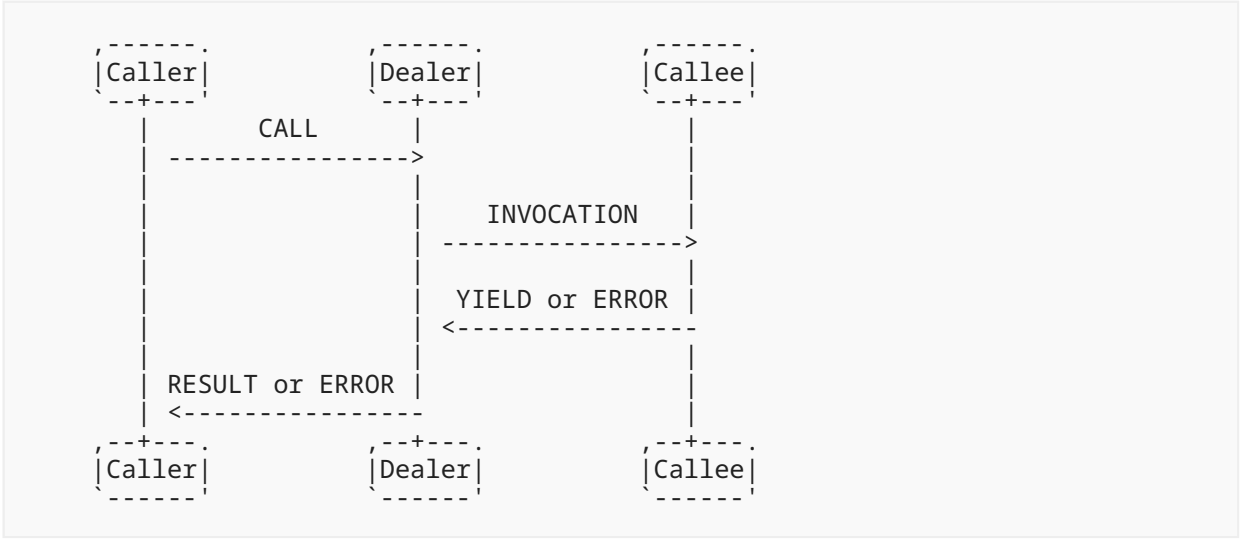
```
[8, 66, 788923562, {}, "wamp.error.no_such_registration"]
```

6.2. Calling and Invocations

The message flow between Callers, a Dealer and Callees for calling procedures and invoking endpoints involves the following messages:

1. `CALL`

2. `RESULT`
3. `INVOCATION`
4. `YIELD`
5. `ERROR`



The execution of remote procedure calls is asynchronous, and there may be more than one call outstanding. A call is called outstanding (from the point of view of the Caller), when a (final) result or error has not yet been received by the Caller.

6.2.1. CALL

When a Caller wishes to call a remote procedure, it sends a `CALL` message to a Dealer:

```
[CALL, Request|id, Options|dict, Procedure|uri]
```

or

```
[CALL, Request|id, Options|dict, Procedure|uri, Arguments|list]
```

or

```
[CALL, Request|id, Options|dict, Procedure|uri, Arguments|list,
    ArgumentsKw|dict]
```

where

- `Request` is a sequential ID in the *session scope*, incremented by the Caller and used to correlate the Dealer's response with the request.
- `Options` is a dictionary that allows to provide additional call request details in an extensible way. This is described further below.

- `Procedure` is the URI of the procedure to be called.
- `Arguments` is a list of positional call arguments (each of arbitrary type). The list may be of zero length.
- `ArgumentsKw` is a dictionary of keyword call arguments (each of arbitrary type). The dictionary may be empty.

Example

```
    [48, 7814135, {}, "com.myapp.ping"]
```

Example

```
    [48, 7814135, {}, "com.myapp.echo", ["Hello, world!"]]
```

Example

```
    [48, 7814135, {}, "com.myapp.add2", [23, 7]]
```

Example

```
    [48, 7814135, {}, "com.myapp.user.new", ["johnny"],
        {"firstname": "John", "surname": "Doe"}]
```

6.2.2. INVOCATION

If the Dealer is able to fulfill (mediate) the call and it allows the call, it sends a `INVOCATION` message to the respective Callee implementing the procedure:

```
    [INVOCATION, Request|id, REGISTERED.Registration|id, Details|dict]
```

or

```
    [INVOCATION, Request|id, REGISTERED.Registration|id, Details|dict,
        CALL.Arguments|list]
```

or

```
    [INVOCATION, Request|id, REGISTERED.Registration|id, Details|dict,
        CALL.Arguments|list, CALL.ArgumentsKw|dict]
```

where

- `Request` is a sequential ID in the *session scope*, incremented by the Dealer and used to correlate the *Callee's* response with the request.

- `REGISTERED.Registration` is the registration ID under which the procedure was registered at the Dealer.
- `Details` is a dictionary that allows to provide additional invocation request details in an extensible way. This is described further below.
- `CALL.Arguments` is the original list of positional call arguments as provided by the Caller.
- `CALL.ArgumentsKw` is the original dictionary of keyword call arguments as provided by the Caller.

Example

```
    [68, 6131533, 9823526, {}]
```

Example

```
    [68, 6131533, 9823527, {}, ["Hello, world!"]]
```

Example

```
    [68, 6131533, 9823528, {}, [23, 7]]
```

Example

```
    [68, 6131533, 9823529, {}, ["johnny"], {"firstname": "John", "surname":
"Doe"}]
```

6.2.3. YIELD

If the Callee is able to successfully process and finish the execution of the call, it answers by sending a `YIELD` message to the Dealer:

```
    [YIELD, INVOCATION.Request|id, Options|dict]
```

or

```
    [YIELD, INVOCATION.Request|id, Options|dict, Arguments|list]
```

or

```
    [YIELD, INVOCATION.Request|id, Options|dict, Arguments|list, ArgumentsKw|
dict]
```

where

- `INVOCATION.Request` is the ID from the original invocation request.
- `Options`is a dictionary that allows to provide additional options.
- `Arguments` is a list of positional result elements (each of arbitrary type). The list may be of zero length.
- `ArgumentsKw` is a dictionary of keyword result elements (each of arbitrary type). The dictionary may be empty.

Example

```
[70, 6131533, {}]
```

Example

```
[70, 6131533, {}, ["Hello, world!"]]
```

Example

```
[70, 6131533, {}, [30]]
```

Example

```
[70, 6131533, {}, [], {"userid": 123, "karma": 10}]
```

6.2.4. RESULT

The Dealer will then send a `RESULT` message to the original Caller:

```
[RESULT, CALL.Request|id, Details|dict]
```

or

```
[RESULT, CALL.Request|id, Details|dict, YIELD.Arguments|list]
```

or

```
[RESULT, CALL.Request|id, Details|dict, YIELD.Arguments|list,
    YIELD.ArgumentsKw|dict]
```

where

- `CALL.Request` is the ID from the original call request.

- `Details` is a dictionary of additional details.
- `YIELD.Arguments` is the original list of positional result elements as returned by the Callee.
- `YIELD.ArgumentsKw` is the original dictionary of keyword result elements as returned by the Callee.

Example

```
    [50, 7814135, {}]
```

Example

```
    [50, 7814135, {}, ["Hello, world!"]]
```

Example

```
    [50, 7814135, {}, [30]]
```

Example

```
    [50, 7814135, {}, [], {"userid": 123, "karma": 10}]
```

6.2.5. Invocation ERROR

If the Callee is unable to process or finish the execution of the call, or the application code implementing the procedure raises an exception or otherwise runs into an error, the Callee sends an `ERROR` message to the Dealer:

```
    [ERROR, INVOCATION, INVOCATION.Request|id, Details|dict, Error|uri]
```

or

```
    [ERROR, INVOCATION, INVOCATION.Request|id, Details|dict, Error|uri,
Arguments|list]
```

or

```
    [ERROR, INVOCATION, INVOCATION.Request|id, Details|dict, Error|uri,
Arguments|list,
        ArgumentsKw|dict]
```

where

- `INVOCATION.Request` is the ID from the original `INVOCATION` request previously sent by the Dealer to the Callee.
- `Details` is a dictionary with additional error details.
- `Error` is a URI that identifies the error of why the request could not be fulfilled.
- `Arguments` is a list containing arbitrary, application defined, positional error information. This will be forwarded by the Dealer to the Caller that initiated the call.
- `ArgumentsKw` is a dictionary containing arbitrary, application defined, keyword-based error information. This will be forwarded by the Dealer to the Caller that initiated the call.

Example

```
    [8, 68, 6131533, {}, "com.myapp.error.object_write_protected",
        ["Object is write protected."], {"severity": 3}]
```

6.2.6. Call ERROR

The Dealer will then send a `ERROR` message to the original Caller:

```
    [ERROR, CALL, CALL.Request|id, Details|dict, Error|uri]
```

or

```
    [ERROR, CALL, CALL.Request|id, Details|dict, Error|uri, Arguments|list]
```

or

```
    [ERROR, CALL, CALL.Request|id, Details|dict, Error|uri, Arguments|list,
        ArgumentsKw|dict]
```

where

- `CALL.Request` is the ID from the original `CALL` request sent by the Caller to the Dealer.
- `Details` is a dictionary with additional error details.
- `Error` is a URI identifying the type of error as returned by the Callee to the Dealer.
- `Arguments` is a list containing the original error payload list as returned by the Callee to the Dealer.
- `ArgumentsKw` is a dictionary containing the original error payload dictionary as returned by the Callee to the Dealer

Example

```
[8, 48, 7814135, {}, "com.myapp.error.object_write_protected",
    ["Object is write protected."], {"severity": 3}]
```

If the original call already failed at the Dealer **before** the call would have been forwarded to any Callee, the Dealer will send an `ERROR` message to the Caller:

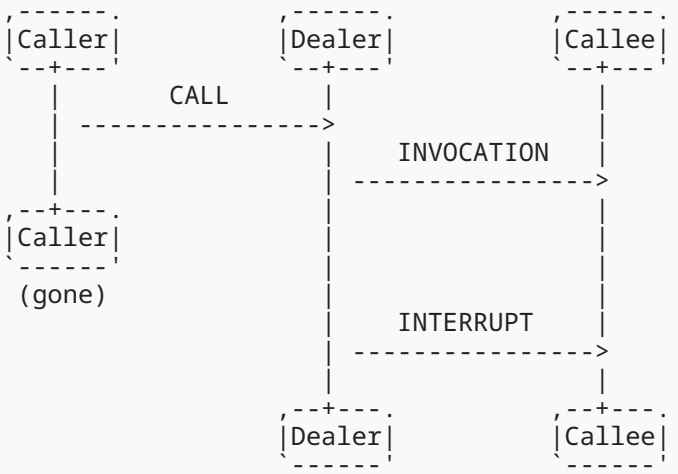
```
[ERROR, CALL, CALL.Request|id, Details|dict, Error|uri]
```

Example

```
[8, 48, 7814135, {}, "wamp.error.no_such_procedure"]
```

6.3. Caller Leaving During an RPC Invocation

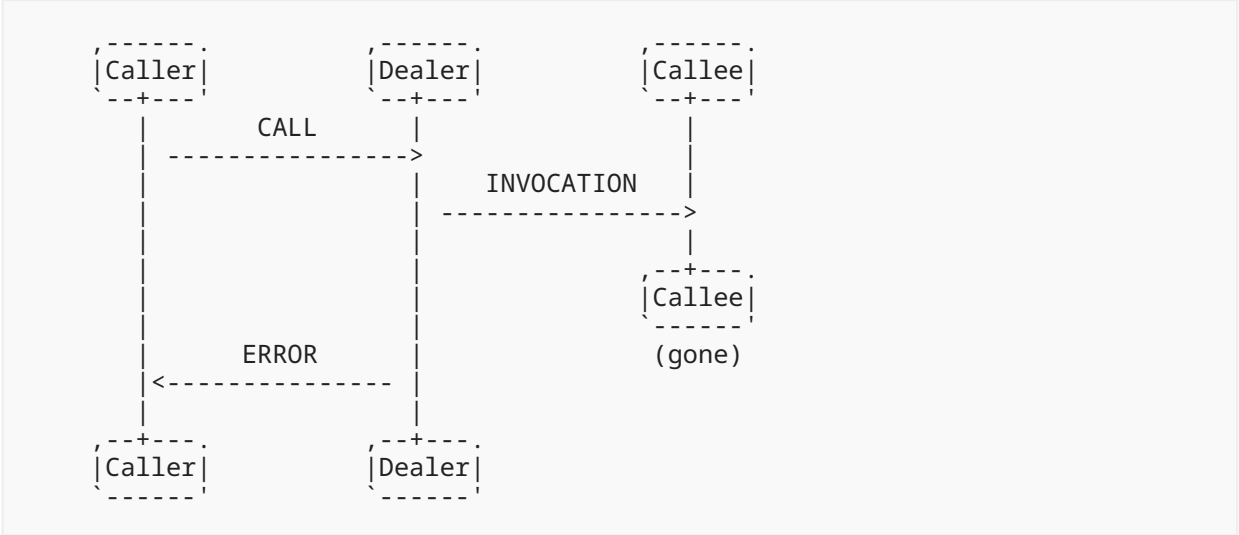
If, after the *Dealer* sends an INVOCATION but before it receives a YIELD or ERROR response, the *Dealer* detects the original *Caller* leaving or disconnecting, then the *Dealer* shall send an INTERRUPT to the *Callee* if both the *Dealer* and *Callee* support the *Call Canceling* advanced feature. That INTERRUPT message MUST have `Options.mode` set to `"killnowait"` to indicate to the *Callee* that no response should be sent for the INTERRUPT.



If either the *Dealer* or the *Callee* does not support the *Call Canceling* feature, then an INTERRUPT message shall NOT sent in this scenario. Whether or not call canceling is supported, the *Dealer* shall be prepared to discard a YIELD or ERROR response associated with that defunct call request.

6.4. Callee Leaving During an RPC Invocation

After sending an INVOCATION message, if a *Dealer* detects that the *Callee* has left/disconnected without sending a final YIELD or ERROR response, then the *Dealer* SHALL return an ERROR message back to the Caller with a `wamp.error.cancelled` URI. The *Dealer* MAY provide additional information via the ERROR payload arguments to clarify that the cancellation is due to the *Callee* leaving before the call could be completed.



7. Security Model

The following discusses the security model for the Basic Profile. Any changes or extensions to this for the Advanced Profile are discussed further on as part of the Advanced Profile definition.

All WAMP implementations, in particular Routers MUST support the following ordering guarantees.

A WAMP Advanced Profile may provide applications options to relax ordering guarantees, in particular with distributed calls.

7.1. Ordering Guarantees

Publish & Subscribe Ordering

Regarding **Publish & Subscribe**, the ordering guarantees are as follows:

If *Subscriber A* is subscribed to both **Topic 1** and **Topic 2**, and *Publisher B* first publishes an **Event 1** to **Topic 1** and then an **Event 2** to **Topic 2**, then *Subscriber A* will first receive **Event 1** and then **Event 2**. This also holds if **Topic 1** and **Topic 2** are identical.

In other words, WAMP guarantees ordering of events between any given *pair* of Publisher and Subscriber.

Further, if *Subscriber A* subscribes to **Topic 1**, the `SUBSCRIBED` message will be sent by the *Broker* to *Subscriber A* before any `EVENT` message for **Topic 1**.

There is no guarantee regarding the order of return for multiple subsequent subscribe requests. A subscribe request might require the *Broker* to do a time-consuming lookup in some database, whereas another subscribe request second might be permissible immediately.

Remote Procedure Call Ordering

Regarding **Remote Procedure Calls**, the ordering guarantees are as follows:

If *Callee A* has registered endpoints for both **Procedure 1** and **Procedure 2**, and *Caller B* first issues a **Call 1** to **Procedure 1** and then a **Call 2** to **Procedure 2**, and both calls are routed to *Callee A*, then *Callee A* will first receive an invocation corresponding to **Call 1** and then **Call 2**. This also holds if **Procedure 1** and **Procedure 2** are identical.

In other words, WAMP guarantees ordering of invocations between any given *pair* of Caller and Callee.

There are no guarantees on the order of call results and errors in relation to *different* calls, since the execution of calls upon different invocations of endpoints in Callees are running independently. A first call might require an expensive, long-running computation, whereas a second, subsequent call might finish immediately.

Further, if *Callee A* registers for **Procedure 1**, the `REGISTERED` message will be sent by *Dealer* to *Callee A* before any `INVOCATION` message for **Procedure 1**.

There is no guarantee regarding the order of return for multiple subsequent register requests. A register request might require the *Broker* to do a time-consuming lookup in some database, whereas another register request second might be permissible immediately.

7.2. Transport Encryption and Integrity

WAMP transports may provide (optional) transport-level encryption and integrity verification. If so, encryption and integrity is point-to-point: between a Client and the Router it is connected to.

Transport-level encryption and integrity is solely at the transport-level and transparent to WAMP. WAMP itself deliberately does not specify any kind of transport-level encryption.

Implementations that offer TCP based transport such as WAMP-over-WebSocket or WAMP-over-RawSocket SHOULD implement Transport Layer Security (TLS).

WAMP deployments are encouraged to stick to a TLS-only policy with the TLS code and setup being hardened.

Further, when a Client connects to a Router over a local-only transport such as Unix domain sockets, the integrity of the data transmitted is implicit (the OS kernel is trusted), and the privacy of the data transmitted can be assured using file system permissions (no one can tap a Unix domain socket without appropriate permissions or being root).

7.3. Router Authentication

To authenticate Routers to Clients, deployments MUST run TLS and Clients MUST verify the Router server certificate presented. WAMP itself does not provide mechanisms to authenticate a Router (only a Client).

The verification of the Router server certificate can happen

1. against a certificate trust database that comes with the Clients operating system
2. against an issuing certificate/key hard-wired into the Client
3. by using new mechanisms like DNS-based Authentication of Named Enitities (DNSSEC)/ TLSA

Further, when a Client connects to a Router over a local-only transport such as Unix domain sockets, the file system permissions can be used to create implicit trust. E.g. if only the OS user under which the Router runs has the permission to create a Unix domain socket under a specific path, Clients connecting to that path can trust in the router authenticity.

7.4. Client Authentication

Authentication of a Client to a Router at the WAMP level is not part of the basic profile.

When running over TLS, a Router MAY authenticate a Client at the transport level by doing a *client certificate based authentication*.

7.5. Routers are trusted

Routers are *trusted* by Clients. In particular, Routers can read (and modify) any application payload transmitted in events, calls, call results and call errors (the `Arguments` or `ArgumentsKw` message fields).

Hence, Routers do not provide confidentiality with respect to application payload, and also do not provide authenticity or integrity of application payloads that could be verified by a receiving Client.

Routers need to read the application payloads in cases of automatic conversion between different serialization formats.

Further, Routers are trusted to **actually perform** routing as specified. E.g. a Client that publishes an event has to trust a Router that the event is actually dispatched to all (eligible) Subscribers by the Router.

A rogue Router might deny normal routing operation without a Client taking notice.

8. Basic Profile URIs

WAMP pre-defines the following error URIs for the **Basic Profile**. WAMP peers SHOULD only use the defined error messages.

Incorrect URIs

When a Peer provides an incorrect URI for any URI-based attribute of a WAMP message (e.g. realm, topic), then the other Peer MUST respond with an `ERROR` message and give the following *Error URI*:

```
    wamp.error.invalid_uri
```

Interaction

Peer provided an incorrect URI for any URI-based attribute of WAMP message, such as realm, topic or procedure

```
    wamp.error.invalid_uri
```

A Dealer could not perform a call, since no procedure is currently registered under the given URI.

```
    wamp.error.no_such_procedure
```

A procedure could not be registered, since a procedure with the given URI is already registered.

```
    wamp.error.procedure_already_exists
```

A Dealer could not perform an unregister, since the given registration is not active.

```
    wamp.error.no_such_registration
```

A Broker could not perform an unsubscribe, since the given subscription is not active.

```
    wamp.error.no_such_subscription
```

A call failed since the given argument types or values are not acceptable to the called procedure. In this case the Callee may throw this error. Alternatively a Router may throw this error if it performed *payload validation* of a call, call result, call error or publish, and the payload did not conform to the requirements.

```
    wamp.error.invalid_argument
```

A *Dealer* or *Callee* canceled a call previously issued

```
    wamp.error.canceled
```

A message could not be delivered due to transport payload size limits.

```
wamp.error.payload_size_exceeded
```

Session Close

The Peer is shutting down completely - used as a `GOODBYE` (or `ABORT`) reason.

```
wamp.close.system_shutdown
```

The Peer want to leave the realm - used as a `GOODBYE` reason.

```
wamp.close.close_realm
```

A Peer acknowledges ending of a session - used as a `GOODBYE` reply reason.

```
wamp.close.goodbye_and_out
```

A Peer received invalid WAMP protocol message (e.g. `HELLO` message after session was already established) - used as a `ABORT` reply reason. More detailed information may be provided by using the `Arguments|list` or `ArgumentsKw|dict`.

```
wamp.error.protocol_violation
```

Authorization

A join, call, register, publish or subscribe failed, since the Peer is not authorized to perform the operation.

```
wamp.error.not_authorized
```

A Dealer or Broker could not determine if the Peer is authorized to perform a join, call, register, publish or subscribe, since the authorization operation *itself* failed. E.g. a custom authorizer did run into an error.

```
wamp.error.authorization_failed
```

Peer wanted to join a non-existing realm (and the Router did not allow to auto-create the realm).

```
wamp.error.no_such_realm
```

A Peer was to be authenticated under a Role that does not (or no longer) exists on the Router. For example, the Peer was successfully authenticated, but the Role configured does not exists - hence there is some misconfiguration in the Router.

```
wamp.error.no_such_role
```

9. WAMP Advanced Profile

While all implementations MUST implement the subset of the Basic Profile necessary for the particular set of WAMP roles they provide, they MAY implement any subset of features from the Advanced Profile. Implementers SHOULD implement the maximum of features possible considering the aims of an implementation.

> Note: Features listed here may be experimental or underspecced and yet unimplemented in any implementation. This part of the specification is very much a work in progress. An approximate status of each feature is given at the beginning of the feature section.

9.1. Feature Announcement

Support for advanced features must be announced by the peers which implement them. The following is a complete list of advanced features currently defined or proposed.

Advanced RPC Features

Feature	Status	Caller	Dealer	Callee
Progressive Call Results	stable	X	X	X
Progressive Call Invocations	alpha	X	X	X
Call Timeout	alpha	X	X	X
Call Canceling	alpha	X	X	X
Caller Identification	stable	X	X	X
Call Trustlevels	alpha		X	X
Registration Meta API	beta		X	
Pattern-based Registration	stable		X	X
Shared Registration	beta		X	X
Sharded Registration	alpha		X	X
Registration Revocation	alpha		X	X
(Interface) Procedure Reflection	sketch		X	

Table 3

Advanced PubSub Features

Feature	Status	Publisher	Broker	Subscriber
Subscriber Blackwhite Listing	stable	X	X	
Publisher Exclusion	stable	X	X	
Publisher Identification	stable	X	X	X
Publication Trustlevels	alpha		X	X
Subscription Meta API	beta		X	
Pattern-based Subscription	stable		X	X
Sharded Subscription	alpha		X	X
Event History	beta		X	
(Interface) Topic Reflection	sketch		X	

Table 4

Other Advanced Features

Feature	Status
Challenge-response Authentication	stable
Ticket authentication	beta
Cryptosign authentication	beta
RawSocket transport	stable
Batched WebSocket transport	sketch
HTTP Longpoll transport	beta
Session Meta API	beta
Call Rerouting	sketch
Payload Passthru Mode	sketch

Table 5

The status of the respective AP feature is marked as follows:

Status	Description
sketch	There is a rough description of an itch to scratch, but the feature use case isn't clear, and there is no protocol proposal at all.
alpha	The feature use case is still fuzzy and/or the feature definition is unclear, but there is at least a protocol level proposal.
beta	The feature use case is clearly defined and the feature definition in the spec is sufficient to write a prototype implementation. The feature definition and details may still be incomplete and change.
stable	The feature definition in the spec is complete and stable and the feature use case is field proven in real applications. There are multiple, interoperable implementations.

Table 6

9.2. Additional Messages

The Advanced Profile defines additional WAMP-level messages which are explained in detail in separate sections. The following 4 additional message types MAY be used in the Advanced Profile and their direction between peer roles. Here, "Tx" ("Rx") means the message is sent (received) by a peer of the respective role.

Code	Message	Publisher	Broker	Subscriber	Caller	Dealer	Callee
4	CHALLENGE	Rx	Tx	Rx	Rx	Tx	Rx
5	AUTHENTICATE	Tx	Rx	Tx	Tx	Rx	Tx
49	CANCEL				Tx	Rx	
69	INTERRUPT					Tx	Rx

Table 7

9.2.1. CHALLENGE

The `CHALLENGE` message is used with certain Authentication Methods. During authenticated session establishment, a **Router** sends a challenge message.

```
[CHALLENGE, AuthMethod|string, Extra|dict]
```

9.2.2. AUTHENTICATE

The `AUTHENTICATE` message is used with certain Authentication Methods. A **Client** having received a challenge is expected to respond by sending a signature or token.

```
[AUTHENTICATE, Signature|string, Extra|dict]
```

9.2.3. CANCEL

The `CANCEL` message is used with the Call Canceling advanced feature. A *Caller* can cancel an issued call actively by sending a cancel message to the *Dealer*.

```
[CANCEL, CALL.Request|id, Options|dict]
```

9.2.4. INTERRUPT

The `INTERRUPT` message is used with the Call Canceling advanced feature. Upon receiving a cancel for a pending call, a *Dealer* will issue an interrupt to the *Callee*.

```
[INTERRUPT, INVOCATION.Request|id, Options|dict]
```

10. Meta API

10.1. Session Meta API

WAMP enables the monitoring of when sessions join a realm on the router or when they leave it via **Session Meta Events**. It also allows retrieving information about currently connected sessions via **Session Meta Procedures**.

Meta events are created by the router itself. This means that the events, as well as the data received when calling a meta procedure, can be accorded the same trust level as the router.

> Note that an implementation that only supports a *Broker* or *Dealer* role, not both at the same time, essentially cannot offer the **Session Meta API**, as it requires both roles to support this feature.

The following sections contain an informal, easy to digest description of the WAMP procedures and topics available in (this part of) the WAMP Meta API. A formal definition of the WAMP Meta API in terms of available WAMP procedures and topics including precise and complete type definitions of the application payloads, that is procedure arguments and results or event payloads is contained in

- Compiled Binary Schema: `<WAMP API Catalog>/schema/wamp-meta.bfbs`
- FlatBuffers Schema Source: `<WAMP API Catalog>/src/wamp-meta.fbs`

which uses FlatBuffers IDL to describe the API. The method of using FlatBuffers IDL and type definitions to formally define WAMP procedures and topics is detailed in section WAMP IDL.

Feature Announcement

Support for this feature MUST be announced by **both** *Dealers* and *Brokers* via:

```
    HELLO.Details.roles.<role>.features.
        session_meta_api|bool := true
```

Here is a `WELCOME` message from a *Router* with support for both the *Broker* and *Dealer* role, and with support for **Session Meta API**:

```
[
    2,
    4580268554656113,
    {
        "authid":"OL3AeppwDLXiAAPbqm9IVhnw",
        "authrole": "anonymous",
        "authmethod": "anonymous",
        "roles": {
            "broker": {
                "features": {
                    "session_meta_api": true
                }
            },
            "dealer": {
                "features": {
                    "session_meta_api": true
                }
            }
        }
    }
]
```

Note in particular that the feature is announced on both the *Broker* and the *Dealer* roles.

10.1.1. Events

A client can subscribe to the following session meta-events, which cover the lifecycle of a session:

- `wamp.session.on_join`: Fired when a session joins a realm on the router.
- `wamp.session.on_leave`: Fired when a session leaves a realm on the router or is disconnected.

Session Meta Events MUST be dispatched by the *Router* to the same realm as the WAMP session which triggered the event.

10.1.1.1. wamp.session.on_join

Fired when a session joins a realm on the router. The event payload consists of a single positional argument `details|dict`:

- `session|id` - The session ID of the session that joined
- `authid|string` - The authentication ID of the session that joined
- `authrole|string` - The authentication role of the session that joined
- `authmethod|string` - The authentication method that was used for authentication the session that joined
- `authprovider|string`- The provider that performed the authentication of the session that joined

- `transport|dict` - Optional, implementation defined information about the WAMP transport the joined session is running over.

> See **Authentication** for a description of the `authid`, `authrole`, `authmethod` and `authprovider` properties.

10.1.1.2. wamp.session.on_leave

Fired when a session leaves a realm on the router or is disconnected. The event payload consists of three positional arguments:

- `session|id` - The session ID of the session that left
- `authid|string` - The authentication ID of the session that left
- `authrole|string` - The authentication role of the session that left

10.1.2. Procedures

A client can actively retrieve information about sessions, or forcefully close sessions, via the following meta-procedures:

- `wamp.session.count`: Obtains the number of sessions currently attached to the realm.
- `wamp.session.list`: Retrieves a list of the session IDs for all sessions currently attached to the realm.
- `wamp.session.get`: Retrieves information on a specific session.
- `wamp.session.kill`: Kill a single session identified by session ID.
- `wamp.session.kill_by_authid`: Kill all currently connected sessions that have the specified authid.
- `wamp.session.kill_by_authrole`: Kill all currently connected sessions that have the specified authrole.
- `wamp.session.kill_all`: Kill all currently connected sessions in the caller's realm.

Session meta procedures MUST be registered by the *Router* on the same realm as the WAMP session about which information is retrieved.

10.1.2.1. wamp.session.count

Obtains the number of sessions currently attached to the realm.

Positional arguments

1. `filter_authroles|list[string]` - Optional filter: if provided, only count sessions with an `authrole` from this list.

Positional results

1. `count|int` - The number of sessions currently attached to the realm.

10.1.2.2. wamp.session.list

Retrieves a list of the session IDs for all sessions currently attached to the realm.

Positional arguments

1. `filter_authroles|list[string]` - Optional filter: if provided, only count sessions with an `authrole` from this list.

Positional results

1. `session_ids|list` - List of WAMP session IDs (order undefined).

10.1.2.3. wamp.session.get

Retrieves information on a specific session.

Positional arguments

1. `session|id` - The session ID of the session to retrieve details for.

Positional results

1. `details|dict` - Information on a particular session:
   - `session|id` - The session ID of the session that joined
   - `authid|string` - The authentication ID of the session that joined
   - `authrole|string` - The authentication role of the session that joined
   - `authmethod|string` - The authentication method that was used for authentication the session that joined
   - `authprovider|string`- The provider that performed the authentication of the session that joined
   - `transport|dict` - Optional, implementation defined information about the WAMP transport the joined session is running over.

> See **Authentication** for a description of the `authid`, `authrole`, `authmethod` and `authprovider` properties.

Errors

- `wamp.error.no_such_session` - No session with the given ID exists on the router.

10.1.2.4. wamp.session.kill

Kill a single session identified by session ID.

The caller of this meta procedure may only specify session IDs other than its own session. Specifying the caller's own session will result in a `wamp.error.no_such_session` since no *other* session with that ID exists.

The keyword arguments are optional, and if not provided the reason defaults to `wamp.close.killed` and the message is omitted from the `GOODBYE` sent to the closed session.

Positional arguments

1. `session|id` - The session ID of the session to close.

Keyword arguments

1. `reason|uri` - reason for closing session, sent to client in `GOODBYE.Reason`.
2. `message|string` - additional information sent to client in `GOODBYE.Details` under the key "message".

Errors

- `wamp.error.no_such_session` - No session with the given ID exists on the router.
- `wamp.error.invalid_uri` - A `reason` keyword argument has a value that is not a valid non-empty URI.

10.1.2.5. wamp.session.kill_by_authid

Kill all currently connected sessions that have the specified `authid`.

If the caller's own session has the specified `authid`, the caller's session is excluded from the closed sessions.

The keyword arguments are optional, and if not provided the reason defaults to `wamp.close.killed` and the message is omitted from the `GOODBYE` sent to the closed session.

Positional arguments

1. `authid|string` - The authentication ID identifying sessions to close.

Keyword arguments

1. `reason|uri` - reason for closing sessions, sent to clients in `GOODBYE.Reason`
2. `message|string` - additional information sent to clients in `GOODBYE.Details` under the key "message".

Positional results

1. `sessions|list` - The list of WAMP session IDs of session that were killed.

Errors

- `wamp.error.invalid_uri` - A `reason` keyword argument has a value that is not a valid non-empty URI.

10.1.2.6. wamp.session.kill_by_authrole

Kill all currently connected sessions that have the specified `authrole`.

If the caller's own session has the specified `authrole`, the caller's session is excluded from the closed sessions.

The keyword arguments are optional, and if not provided the reason defaults to `wamp.close.killed` and the message is omitted from the `GOODBYE` sent to the closed session.

Positional arguments

1. `authrole|string` - The authentication role identifying sessions to close.

Keyword arguments

1. `reason|uri` - reason for closing sessions, sent to clients in `GOODBYE.Reason`
2. `message|string` - additional information sent to clients in `GOODBYE.Details` under the key "message".

Positional results

1. `count|int` - The number of sessions closed by this meta procedure.

Errors

- `wamp.error.invalid_uri` - A `reason` keyword argument has a value that is not a valid non-empty URI.

10.1.2.7. wamp.session.kill_all

Kill all currently connected sessions in the caller's realm.

The caller's own session is excluded from the closed sessions. Closing all sessions in the realm will not generate session meta events or testament events, since no subscribers would remain to receive these events.

The keyword arguments are optional, and if not provided the reason defaults to `wamp.close.killed` and the message is omitted from the `GOODBYE` sent to the closed session.

Keyword arguments

1. `reason|uri` - reason for closing sessions, sent to clients in `GOODBYE.Reason`
2. `message|string` - additional information sent to clients in `GOODBYE.Details` under the key "message".

Positional results

1. `count|int` - The number of sessions closed by this meta procedure.

Errors

- `wamp.error.invalid_uri` - A `reason` keyword argument has a value that is not a valid non-empty URI.

10.2. Registration Meta API

Registration Meta Events are fired when registrations are first created, when *Callees* are attached (removed) to (from) a registration, and when registrations are finally destroyed.

Furthermore, WAMP allows actively retrieving information about registrations via **Registration Meta Procedures**.

Meta-events are created by the router itself. This means that the events as well as the data received when calling a meta-procedure can be accorded the same trust level as the router.

> Note that an implementation that only supports a *Broker* or *Dealer* role, not both at the same time, essentially cannot offer the **Registration Meta API**, as it requires both roles to support this feature.

The following sections contain an informal, easy to digest description of the WAMP procedures and topics available in (this part of) the WAMP Meta API. A formal definition of the WAMP Meta API in terms of available WAMP procedures and topics including precise and complete type definitions of the application payloads, that is procedure arguments and results or event payloads is contained in:

- Compiled Binary Schema: `<WAMP API Catalog>/schema/wamp-meta.bfbs`
- FlatBuffers Schema Source: `<WAMP API Catalog>/src/wamp-meta.fbs`

which uses FlatBuffers IDL to describe the API. The method of using FlatBuffers IDL and type definitions to formally define WAMP procedures and topics is detailed in section WAMP IDL.

Feature Announcement

Support for this feature MUST be announced by a *Dealers* (`role := "dealer"`) via:

```
HELLO.Details.roles.<role>.features.
    registration_meta_api|bool := true
```

Here is a `WELCOME` message from a *Router* with support for both the *Broker* and *Dealer* role, and with support for **Registration Meta API**:

```
[
    2,
    4580268554656113,
    {
        "authid":"OL3AeppwDLXiAAPbqm9IVhnw",
        "authrole": "anonymous",
        "authmethod": "anonymous",
        "roles": {
            "broker": {
                "features": {
                }
            },
            "dealer": {
                "features": {
                    "registration_meta_api": true
                }
            }
        }
    }
]
```

10.2.1. Events

A client can subscribe to the following registration meta-events, which cover the lifecycle of a registration:

- `wamp.registration.on_create`: Fired when a registration is created through a registration request for a URI which was previously without a registration.
- `wamp.registration.on_register`: Fired when a *Callee* session is added to a registration.
- `wamp.registration.on_unregister`: Fired when a *Callee* session is removed from a registration.
- `wamp.registration.on_delete`: Fired when a registration is deleted after the last *Callee* session attached to it has been removed.

A `wamp.registration.on_register` event MUST be fired subsequent to a `wamp.registration.on_create` event, since the first registration results in both the creation of the registration and the addition of a session.

Similarly, the `wamp.registration.on_delete` event MUST be preceded by a `wamp.registration.on_unregister` event.

Registration Meta Events MUST be dispatched by the router to the same realm as the WAMP session which triggered the event.

10.2.1.1. wamp.registration.on_create

Fired when a registration is created through a registration request for a URI which was previously without a registration. The event payload consists of positional arguments:

- `session|id`: The session ID performing the registration request.
- `RegistrationDetails|dict`: Information on the created registration.

Object Schemas

```
    RegistrationDetails :=
    {
        "id": registration|id,
        "created": time_created|iso_8601_string,
        "uri": procedure|uri,
        "match": match_policy|string,
        "invoke": invocation_policy|string
    }
```

See Pattern-based Registrations for a description of `match_policy`.

> NOTE: invocation_policy IS NOT YET DESCRIBED IN THE ADVANCED SPEC

10.2.1.2. wamp.registration.on_register

Fired when a session is added to a registration. The event payload consists of positional arguments:

- `session|id`: The ID of the session being added to a registration.
- `registration|id`: The ID of the registration to which a session is being added.

10.2.1.3. wamp.registration.on_unregister

Fired when a session is removed from a registration. The event payload consists of positional arguments:

- `session|id`: The ID of the session being removed from a registration.
- `registration|id`: The ID of the registration from which a session is being removed.

10.2.1.4. wamp.registration.on_delete

Fired when a registration is deleted after the last session attached to it has been removed. The event payload consists of positional arguments:

- `session|id`: The ID of the last session being removed from a registration.
- `registration|id`: The ID of the registration being deleted.

10.2.2. Procedures

A client can actively retrieve information about registrations via the following meta-procedures:

- `wamp.registration.list`: Retrieves registration IDs listed according to match policies.
- `wamp.registration.lookup`: Obtains the registration (if any) managing a procedure, according to some match policy.
- `wamp.registration.match`: Obtains the registration best matching a given procedure URI.
- `wamp.registration.get`: Retrieves information on a particular registration.

- `wamp.registration.list_callees`: Retrieves a list of session IDs for sessions currently attached to the registration.
- `wamp.registration.count_callees`: Obtains the number of sessions currently attached to the registration.

10.2.2.1. wamp.registration.list

Retrieves registration IDs listed according to match policies.

Arguments

- None

Results

- `RegistrationLists|dict`: A dictionary with a list of registration IDs for each match policy.

Object Schemas

```
RegistrationLists :=
{
    "exact": registration_ids|list,
    "prefix": registration_ids|list,
    "wildcard": registration_ids|list
}
```

See Pattern-based Registrations for a description of match policies.

10.2.2.2. wamp.registration.lookup

Obtains the registration (if any) managing a procedure, according to some match policy.

Arguments

- `procedure|uri`: The procedure to lookup the registration for.
- (Optional) `options|dict`: Same options as when registering a procedure.

Results

- (Nullable) `registration|id`: The ID of the registration managing the procedure, if found, or null.

10.2.2.3. wamp.registration.match

Obtains the registration best matching a given procedure URI.

Arguments

- `procedure|uri`: The procedure URI to match

Results

- (Nullable) `registration|id`: The ID of best matching registration, or null.

10.2.2.4. wamp.registration.get

Retrieves information on a particular registration.

Arguments

- `registration|id`: The ID of the registration to retrieve.

Results

- `RegistrationDetails|dict`: Details on the registration.

Error URIs

- `wamp.error.no_such_registration`: No registration with the given ID exists on the router.

Object Schemas

```
RegistrationDetails :=
{
    "id": registration|id,
    "created": time_created|iso_8601_string,
    "uri": procedure|uri,
    "match": match_policy|string,
    "invoke": invocation_policy|string
}
```

See Pattern-based Registrations for a description of match policies.

NOTE: invocation_policy IS NOT YET DESCRIBED IN THE ADVANCED SPEC

10.2.2.5. wamp.registration.list_callees

Retrieves a list of session IDs for sessions currently attached to the registration.

Arguments

- `registration|id`: The ID of the registration to get callees for.

Results

- `callee_ids|list`: A list of WAMP session IDs of callees currently attached to the registration.

Error URIs

- `wamp.error.no_such_registration`: No registration with the given ID exists on the router.

10.2.2.6. wamp.registration.count_callees

Obtains the number of sessions currently attached to a registration.

Arguments

- `registration|id`: The ID of the registration to get the number of callees for.

Results

- `count|int`: The number of callees currently attached to a registration.

Error URIs

- `wamp.error.no_such_registration`: No registration with the given ID exists on the router.

10.3. Subscriptions Meta API

Within an application, it may be desirable for a publisher to know whether a publication to a specific topic currently makes sense, i.e. whether there are any subscribers who would receive an event based on the publication. It may also be desirable to keep a current count of subscribers to a topic to then be able to filter out any subscribers who are not supposed to receive an event.

Subscription *meta-events* are fired when topics are first created, when clients subscribe/unsubscribe to them, and when topics are deleted. WAMP allows retrieving information about subscriptions via subscription *meta-procedures*.

Support for this feature MUST be announced by Brokers via

```
    HELLO.Details.roles.broker.features.subscription_meta_api|
        bool := true
```

Meta-events are created by the router itself. This means that the events as well as the data received when calling a meta-procedure can be accorded the same trust level as the router.

The following sections contain an informal, easy to digest description of the WAMP procedures and topics available in (this part of) the WAMP Meta API. A formal definition of the WAMP Meta API in terms of available WAMP procedures and topics including precise and complete type definitions of the application payloads, that is procedure arguments and results or event payloads is contained in

- Compiled Binary Schema: `<WAMP API Catalog>/schema/wamp-meta.bfbs`
- FlatBuffers Schema Source: `<WAMP API Catalog>/src/wamp-meta.fbs`

which uses FlatBuffers IDL to describe the API. The method of using FlatBuffers IDL and type definitions to formally define WAMP procedures and topics is detailed in section WAMP IDL.

Feature Announcement

Support for this feature MUST be announced by a *Brokers* (`role := "nroker"`) via:

```
    HELLO.Details.roles.<role>.features.
        subscription_meta_api|bool := true
```

Here is a `WELCOME` message from a *Router* with support for both the *Broker* and *Dealer* role, and with support for **Subscription Meta API**:

```
    [
        2,
        4580268554656113,
        {
            "authid":"OL3AeppwDLXiAAPbqm9IVhnw",
            "authrole": "anonymous",
            "authmethod": "anonymous",
            "roles": {
                "broker": {
                    "features": {
                        "subscription_meta_api": true
                    }
                },
                "dealer": {
                    "features": {
                    }
                }
            }
        }
    ]
```

10.3.1. Events

A client can subscribe to the following session meta-events, which cover the lifecycle of a subscription:

- `wamp.subscription.on_create`: Fired when a subscription is created through a subscription request for a topic which was previously without subscribers.
- `wamp.subscription.on_subscribe`: Fired when a session is added to a subscription.
- `wamp.subscription.on_unsubscribe`: Fired when a session is removed from a subscription.
- `wamp.subscription.on_delete`: Fired when a subscription is deleted after the last session attached to it has been removed.

A `wamp.subscription.on_subscribe` event MUST always be fired subsequent to a `wamp.subscription.on_create` event, since the first subscribe results in both the creation of the subscription and the addition of a session. Similarly, the `wamp.subscription.on_delete` event MUST always be preceded by a `wamp.subscription.on_unsubscribe` event.

The WAMP subscription meta events shall be dispatched by the router to the same realm as the WAMP session which triggered the event.

10.3.1.1. wamp.subscription.on_create

Fired when a subscription is created through a subscription request for a topic which was previously without subscribers. The event payload consists of positional arguments:

- `session|id`: ID of the session performing the subscription request.
- `SubscriptionDetails|dict`: Information on the created subscription.

Object Schemas

```
SubscriptionDetails :=
{
    "id": subscription|id,
    "created": time_created|iso_8601_string,
    "uri": topic|uri,
    "match": match_policy|string
}
```

See Pattern-based Subscriptions for a description of `match_policy`.

10.3.1.2. wamp.subscription.on_subscribe

Fired when a session is added to a subscription. The event payload consists of positional arguments:

- `session|id`: ID of the session being added to a subscription.
- `subscription|id`: ID of the subscription to which the session is being added.

10.3.1.3. wamp.subscription.on_unsubscribe

Fired when a session is removed from a subscription. The event payload consists of positional arguments:

- `session|id`: ID of the session being removed from a subscription.
- `subscription|id`: ID of the subscription from which the session is being removed.

10.3.1.4. wamp.subscription.on_delete

Fired when a subscription is deleted after the last session attached to it has been removed. The event payload consists of positional arguments:

- `session|id`: ID of the last session being removed from a subscription.
- `subscription|id`: ID of the subscription being deleted.

10.3.2. Procedures

A client can actively retrieve information about subscriptions via the following meta-procedures:

- `wamp.subscription.list`: Retrieves subscription IDs listed according to match policies.
- `wamp.subscription.lookup`: Obtains the subscription (if any) managing a topic, according to some match policy.

- `wamp.subscription.match`: Retrieves a list of IDs of subscriptions matching a topic URI, irrespective of match policy.
- `wamp.subscription.get`: Retrieves information on a particular subscription.
- `wamp.subscription.list_subscribers`: Retrieves a list of session IDs for sessions currently attached to the subscription.
- `wamp.subscription.count_subscribers`: Obtains the number of sessions currently attached to the subscription.

10.3.2.1. wamp.subscription.list

Retrieves subscription IDs listed according to match policies.

Arguments - None

Results

The result consists of one positional argument:

- `SubscriptionLists|dict`: A dictionary with a list of subscription IDs for each match policy.

Object Schemas

```
SubscriptionLists :=
{
    "exact": subscription_ids|list,
    "prefix": subscription_ids|list,
    "wildcard": subscription_ids|list
}
```

See Pattern-based Subscriptions for information on match policies.

10.3.2.2. wamp.subscription.lookup

Obtains the subscription (if any) managing a topic, according to some match policy.

Arguments

- `topic|uri`: The URI of the topic.
- (Optional) `options|dict`: Same options as when subscribing to a topic.

Results

The result consists of one positional argument:

- (Nullable) `subscription|id`: The ID of the subscription managing the topic, if found, or null.

10.3.2.3. wamp.subscription.match

Retrieves a list of IDs of subscriptions matching a topic URI, irrespective of match policy.

Arguments

- `topic|uri`: The topic to match.

Results

The result consists of positional arguments:

- (Nullable) `subscription_ids|list`: A list of all matching subscription IDs, or null.

10.3.2.4. wamp.subscription.get

Retrieves information on a particular subscription.

Arguments

- `subscription|id`: The ID of the subscription to retrieve.

Results

The result consists of one positional argument:

- `SubscriptionDetails|dict`: Details on the subscription.

Error URIs

- `wamp.error.no_such_subscription`: No subscription with the given ID exists on the router.

Object Schemas

```
SubscriptionDetails :=
{
    "id": subscription|id,
    "created": time_created|iso_8601_string,
    "uri": topic|uri,
    "match": match_policy|string
}
```

See Pattern-based Subscriptions for information on match policies.

10.3.2.5. wamp.subscription.list_subscribers

Retrieves a list of session IDs for sessions currently attached to the subscription.

Arguments - `subscription|id`: The ID of the subscription to get subscribers for.

Results

The result consists of positional arguments:

- `subscribers_ids|list`: A list of WAMP session IDs of subscribers currently attached to the subscription.

Error URIs

- `wamp.error.no_such_subscription`: No subscription with the given ID exists on the router.

10.3.2.6. wamp.subscription.count_subscribers

Obtains the number of sessions currently attached to a subscription.

Arguments

- `subscription|id`: The ID of the subscription to get the number of subscribers for.

Results

The result consists of one positional argument:

- `count|int`: The number of sessions currently attached to a subscription.

Error URIs

- `wamp.error.no_such_subscription`: No subscription with the given ID exists on the router.

11. Advanced RPC

11.1. Progressive Call Results

A procedure implemented by a *Callee* and registered at a *Dealer* may produce progressive results. Progressive results can e.g. be used to return partial results for long-running operations, or to chunk the transmission of larger results sets.

Feature Announcement

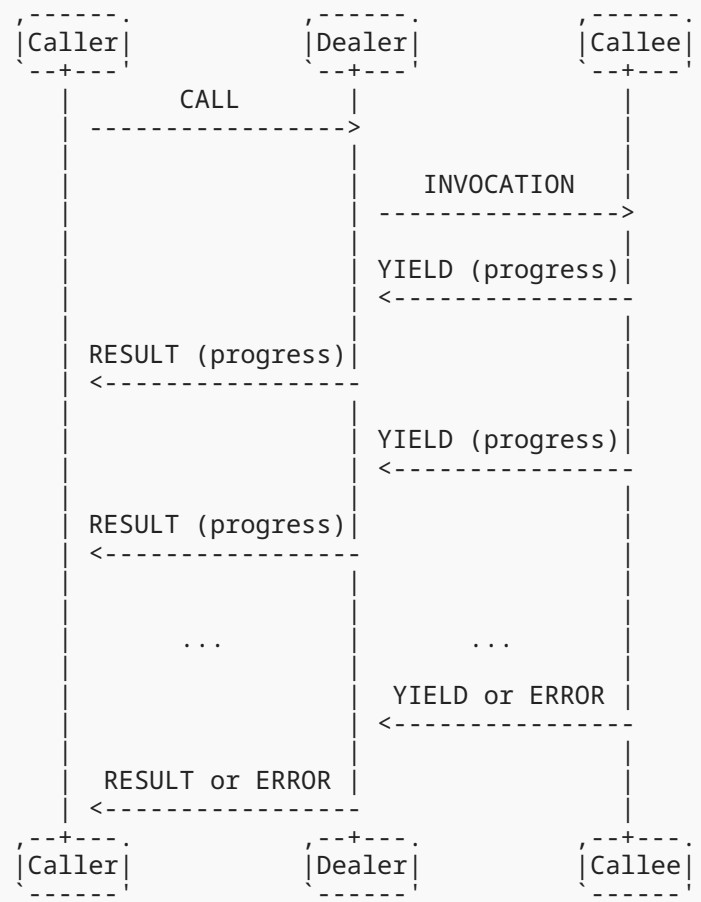
Support for this advanced feature MUST be announced by *Callers* (`role := "caller"`), *Callees* (`role := "callee"`) and *Dealers* (`role := "dealer"`) via

```
    HELLO.Details.roles.<role>.features.
         progressive_call_results|bool := true
```

Additionally, *Callees* and *Dealers* MUST support Call Canceling, which is required for canceling progressive results if the original *Caller* leaves the realm. If a *Callee* supports Progressive Call Results, but not Call Canceling, then the *Dealer* disregards the *Callees* Progressive Call Results feature.

Message Flow

The message flow for progressive results involves:



A *Caller* indicates its willingness to receive progressive results by setting

```
CALL.Options.receive_progress|bool := true
```

Example. Caller-to-Dealer `CALL`

```
[
    48,
    77133,
    {
        "receive_progress": true
    },
    "com.myapp.compute_revenue",
    [2010, 2011, 2012]
]
```

If the *Callee* supports Progressive Call Results, the *Dealer* will forward the *Caller's* willingness to receive progressive results by setting

```
INVOCATION.Details.receive_progress|bool := true
```

Example. Dealer-to-Callee `INVOCATION`

```
[
    68,
    87683,
    324,
    {
        "receive_progress": true
    },
    [2010, 2011, 2012]
]
```

An endpoint implementing the procedure produces progressive results by sending `YIELD` messages to the *Dealer* with

```
YIELD.Options.progress|bool := true
```

Example. Callee-to-Dealer progressive `YIELD`s

```
[
    70,
    87683,
    {
        "progress": true
    },
    ["Y2010", 120]
]
```

```
[
    70,
    87683,
    {
        "progress": true
    },
    ["Y2011", 205]
]
```

Upon receiving an `YIELD` message from a *Callee* with `YIELD.Options.progress == true` (for a call that is still ongoing), the *Dealer* will **immediately** send a `RESULT` message to the original *Caller* with

```
RESULT.Details.progress|bool := true
```

Example. Dealer-to-Caller progressive `RESULT`s

```
    [
        50,
        77133,
        {
            "progress": true
        },
        ["Y2010", 120]
    ]
```

```
    [
        50,
        77133,
        {
            "progress": true
        },
        ["Y2011", 205]
    ]
```

and so on...

An invocation MUST *always* end in either a *normal* `RESULT` or `ERROR` message being sent by the *Callee* and received by the *Dealer*.

Example. Callee-to-Dealer final `YIELD`

```
    [
        70,
        87683,
        {},
        ["Total", 490]
    ]
```

Example. Callee-to-Dealer final `ERROR`

```
    [
        8,
        68,
        87683,
        {},
        "com.myapp.invalid_revenue_year",
        [1830]
    ]
```

A call MUST *always* end in either a *normal* `RESULT` or `ERROR` message being sent by the *Dealer* and received by the *Caller*.

Example. Dealer-to-Caller final `RESULT`

```
[
    50,
    77133,
    {},
    ["Total", 490]
]
```

Example. Dealer-to-Caller final `ERROR`

```
[
    8,
    68,
    77133,
    {},
    "com.myapp.invalid_revenue_year",
    [1830]
]
```

In other words: `YIELD` with `YIELD.Options.progress == true` and `RESULT` with `RESULT.Details.progress == true` messages may only be sent *during* a call or invocation is still ongoing.

The final `YIELD` and final `RESULT` may also be empty, e.g. when all actual results have already been transmitted in progressive result messages.

Example. Callee-to-Dealer `YIELD`s

```
[70, 87683, {"progress": true}, ["Y2010", 120]]
[70, 87683, {"progress": true}, ["Y2011", 205]]
 ...
[70, 87683, {"progress": true}, ["Total", 490]]
[70, 87683, {}]
```

Example. Dealer-to-Caller `RESULT`s

```
[50, 77133, {"progress": true}, ["Y2010", 120]]
[50, 77133, {"progress": true}, ["Y2011", 205]]
 ...
[50, 77133, {"progress": true}, ["Total", 490]]
[50, 77133, {}]
```

The progressive `YIELD` and progressive `RESULT` may also be empty, e.g. when those messages are only used to signal that the procedure is still running and working, and the actual result is completely delivered in the final `YIELD` and `RESULT`:

Example. Callee-to-Dealer `YIELD`s

```
[70, 87683, {"progress": true}]
[70, 87683, {"progress": true}]
...
[70, 87683, {}, [["Y2010", 120], ["Y2011", 205], ...,
    ["Total", 490]]]
```

Example. Dealer-to-Caller `RESULT`s

```
[50, 77133, {"progress": true}]
[50, 77133, {"progress": true}]
...
[50, 77133, {}, [["Y2010", 120], ["Y2011", 205], ...,
    ["Total", 490]]]
```

> Note that intermediate, progressive results and/or the final result MAY have different structure. The WAMP peer implementation is responsible for mapping everything into a form suitable for consumption in the host language.

Example. Callee-to-Dealer `YIELD`s

```
[70, 87683, {"progress": true}, ["partial 1", 10]]
[70, 87683, {"progress": true}, [], {"foo": 10,
    "bar": "partial 1"}]
 ...
[70, 87683, {}, [1, 2, 3], {"moo": "hello"}]
```

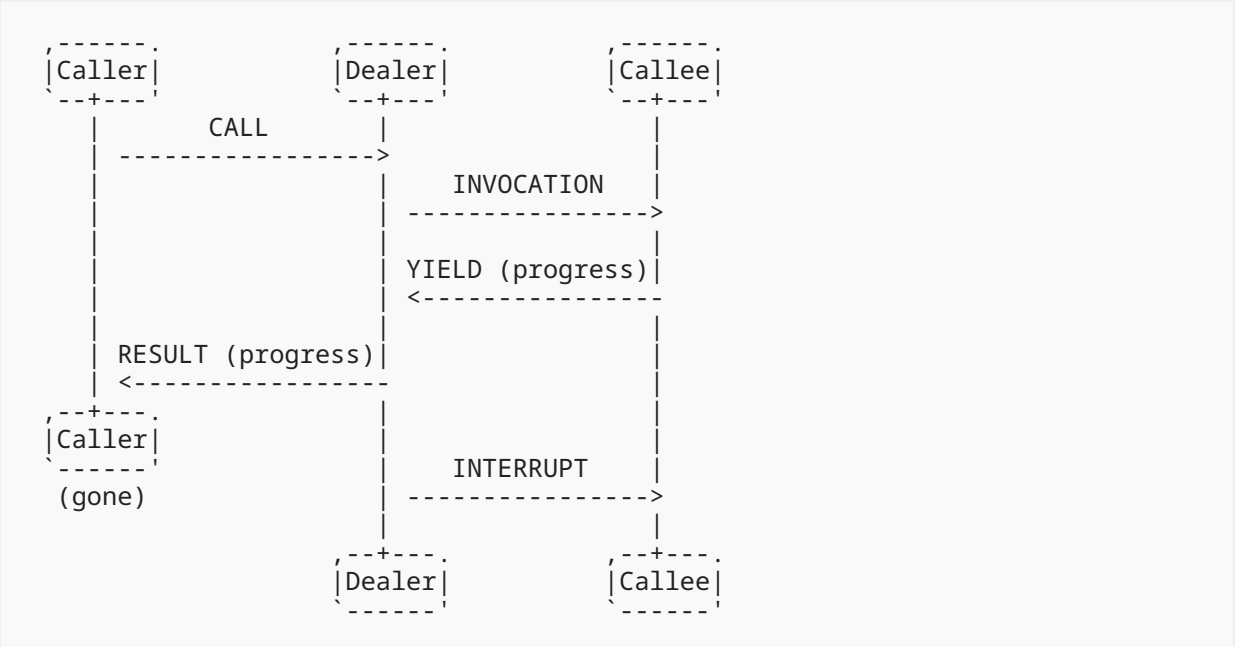
Example. Dealer-to-Caller `RESULT`s

```
[50, 77133, {"progress": true}, ["partial 1", 10]]
[50, 77133, {"progress": true}, [], {"foo": 10,
    "bar": "partial 1"}]
 ...
[50, 77133, {}, [1, 2, 3], {"moo": "hello"}]
```

Even if a *Caller* has indicated its expectation to receive progressive results by setting `CALL.Options.receive_progress|bool := true`, a *Callee* is **not required** to produce progressive results. `CALL.Options.receive_progress` and `INVOCATION.Details.receive_progress` are simply indications that the *Caller* is prepared to process progressive results, should there be any produced. In other words, *Callees* are free to ignore such `receive_progress` hints at any time.

Caller Leaving

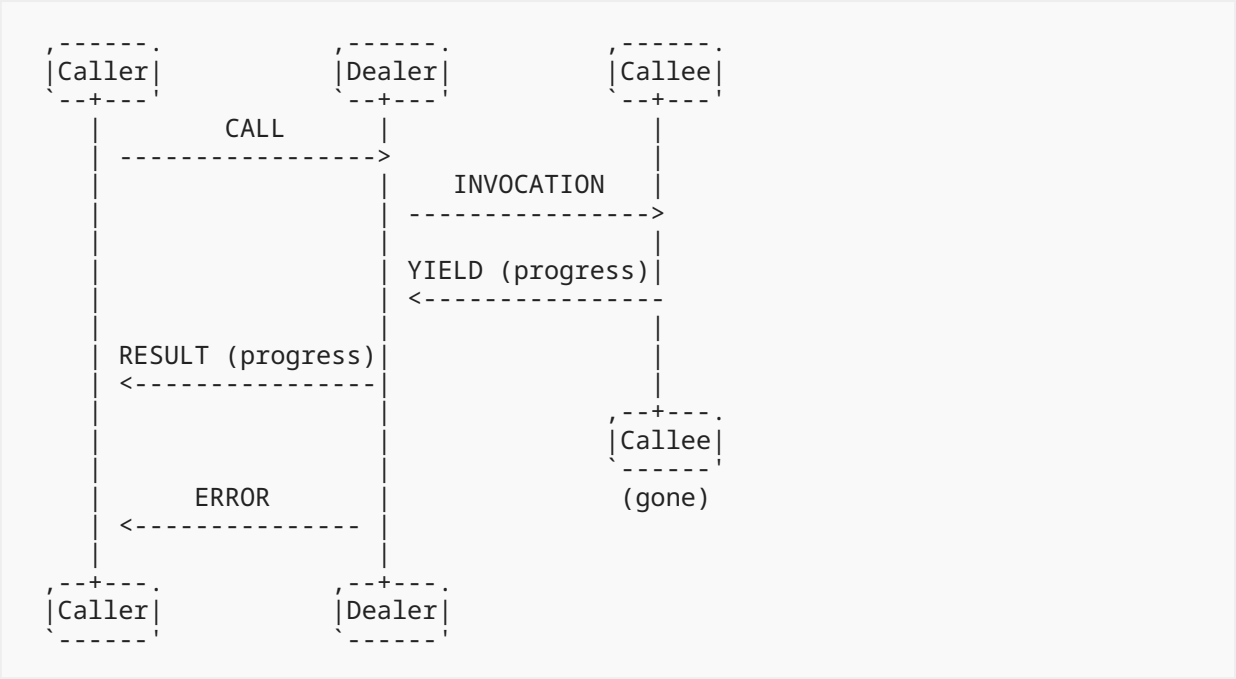
The *Dealer*'s behavior for when a *Caller* leaves or disconnects during a progressive results call shall be the same as in a basic, non-progressive call. That is, the *Dealer* sends an INTERRUPT to the *Callee* with `mode="killnowait"`. See Caller Leaving During RPC Invocation under the Basic Profile.



Such cancellation when the caller leaves addresses a potential security vulnerability: In cases where progressive results are used to stream data to *Callers*, and network connectivity is unreliable, *Callers* may often get disconnected in the middle of receiving such progressive results. Without the mandated cancellation behavior, recurring connect-call-disconnect cycles by a *Caller* would result in a rapidly growing backlog of unprocessed streaming results, overloading the router and further degrading network connectivity.

Callee Leaving

The *Dealer*'s behavior for when a *Callee* leaves or disconnects during a progressive results call shall be the same as in a basic, non-progressive call. That is, the *Dealer* sends an ERROR message to the *Caller* with the `wamp.error.canceled` URI. See Callee Leaving During an RPC Invocation under the Basic Profile.



Ignoring Requests for Progressive Call Results

A *Callee* that does not support progressive results SHOULD ignore any `INVOCATION.Details.receive_progress` flag.

A *Callee* that supports progressive results, but does not support call canceling is considered by the *Dealer* to not support progressive results.

Timeouts

When the *Call Timeouts* feature is used in combination with Progressive Call Results, the `CALL.Options.timeout|integer` option shall represent the time limit between the initial call and the first result, and between results thereafter.

For Dealer-initiated timeouts, this corresponds to - the time between receipt of the `CALL` message and receipt of the first `YIELD` message, and, - the time between received `YIELD` messages thereafter.

For Callee-initiated timeouts, this corresponds to - the time between receipt of the `INVOCATION` message and acquisition of the first result, and, - the time between acquisition of successive results thereafter.

Note that for progressive results, the timeout value does *not* correspond to the duration of the complete call from initiation to the final result. The rationale for this is that it would be unfeasible to compute a reasonable timeout value for a call having a non-deterministic number of progressive results.

11.2. Progressive Call Invocations

A *Caller* may issue a call having progressive invocations. This can be useful in a few cases:

- Payload is too big to send it whole in one request, e.g., uploading a file.
- Long-term data transfer that needs to be consumed early, such as a media stream.
- RPC is called too often and overall processing can be optimized: avoid the need to generate a new id for requests, initiate data structures for a new call, etc.

In such cases, a procedure implemented by a *Callee* and registered at a *Dealer* may be made to receive progressive call invocations, where the *Callee* may start processing the incoming data without awaiting the entire set of payload chunks.

Feature Announcement

Support for this advanced feature MUST be announced by *Callers* (`role := "caller"`), *Callees* (`role := "callee"`) and *Dealers* (`role := "dealer"`) via

```
    HELLO.Details.roles.<role>.features.progressive_call_invocations|bool :=
true
```

Progressive call invocations can work only if all three peers support and announce this feature. In addition, *Callees* MUST announce support of the *Call Cancelling* feature via

```
    HELLO.Details.roles.callee.features.call_cancelling|bool := true
```

As a consequence, *Dealers* MUST also announce support of the *Call Cancelling* feature via

```
    WELCOME.Details.roles.dealer.features.call_cancelling|bool := true
```

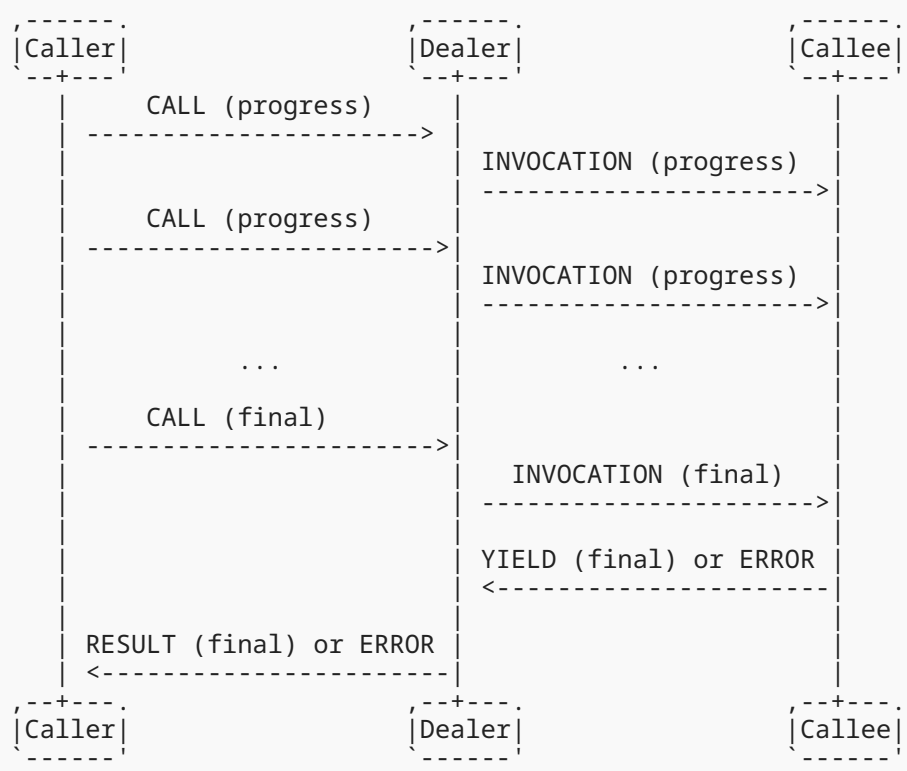
The following cases, where a *Caller* sends a `CALL` message with `progress := true`, MUST be treated as *protocol errors* with the underlying WAMP sessions being aborted:

- The *Caller* did not announce the progressive call invocations feature during the `HELLO` handshake.
- The *Dealer* did not announce the progressive call invocations feature during the `HELLO` handshake.

Otherwise, in cases where the *Caller* sends a `CALL` message with `progress := true` but the *Callee* does not support progressive call invocations or call cancelling, the call MUST be treated as an *application error* with the *Dealer* responding to the *Caller* with the `wamp.error.feature_not_supported` error message.

Message Flow

The message flow for a progressive call when a *Callee* waits for all chunks before processing and sending a single result:



As a `progressive` call chunks are part of the same overall call, the *Caller* must send the same `Request|id` for every `CALL` and the *Dealer* must also use the same `Request|id` with every `INVOCATION` to the *Callee*.

A *Caller* indicates its willingness to issue a progressive call by setting

```
CALL.Options.progress|bool := true
```

Example. Caller-to-Dealer `CALL`

```
[
    48,
    77245,
    {
        "progress": true
    },
    "com.myapp.get_country_by_coords",
    [50.450001, 30.523333]
]
```

If the *Callee* supports progressive call invocations, the *Dealer* shall forward the *Caller's* willingness to send progressive call invocations by setting

```
    INVOCATION.Details.progress|bool := true
```

Example. Dealer-to-Callee `INVOCATION`

```
    [
        68,
        35224,
        379,
        {
            "progress": true
        },
        [50.450001, 30.523333]
    ]
```

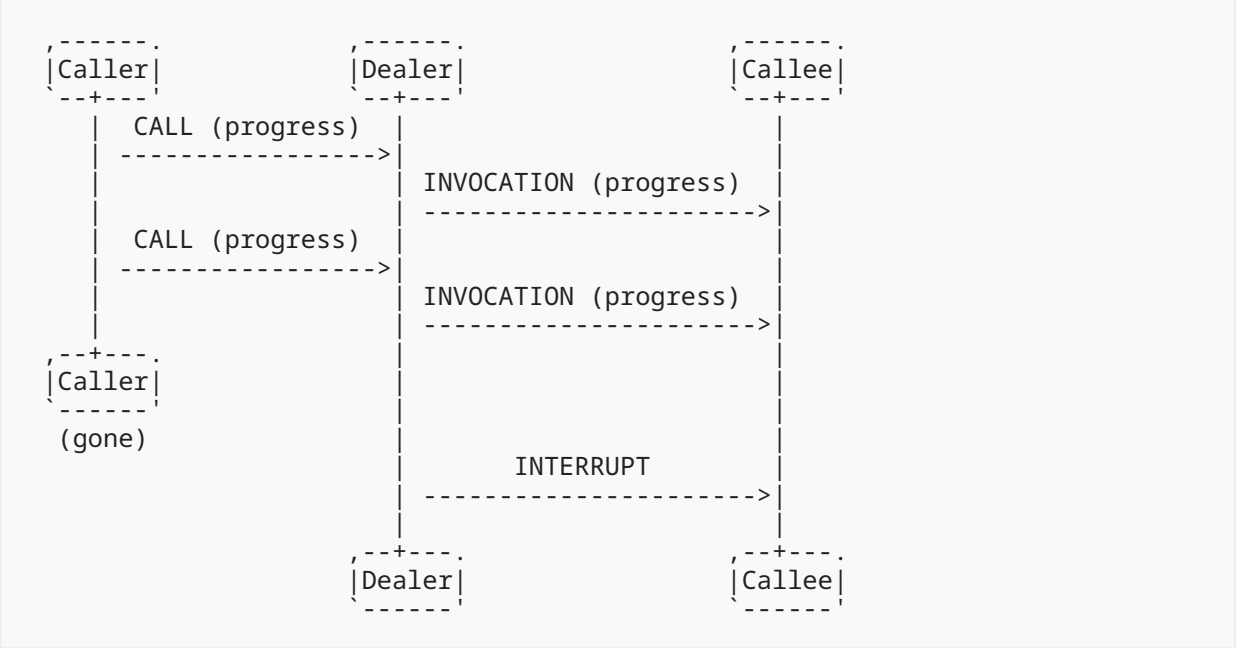
A call invocation MUST *always* end in a *normal* `CALL` without the `"progress": true` option, or explicitly set `"progress": false` which is the default.

Progressive Call Invocations and Shared Registration

RPCs can have a multiple registrations (see `Shared Registration` feature) with different `<invocation_policies>`. However, allowing progressive `CALL` messages to be routed to different *Callees* would lead to unexpected results. To prevent this the *Dealer* MUST make a first `INVOCATION` based on `<invocation_policy>` and then route all subsequent `progressive` calls to the same *Callee*.

Caller Leaving

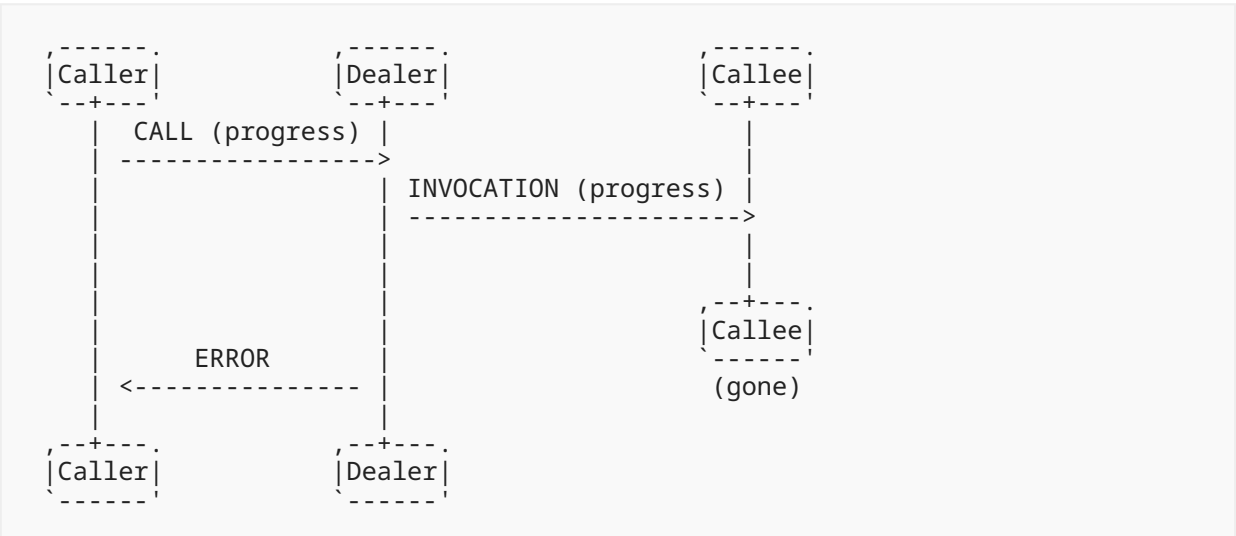
The *Dealer*'s behavior for when a *Caller* leaves or disconnects during a call with progressive invocations shall be the same as in a basic, non-progressive call. That is, the *Dealer* sends an INTERRUPT to the *Callee* with `mode="killnowait"`. See Caller Leaving During RPC Invocation under the Basic Profile.



As in progressive call results, such cancellation when the caller leaves addresses a potential security vulnerability: In cases where progressive call invocations are used to stream data from a *Caller*, and network connectivity is unreliable, the *Caller* may often get disconnected in the middle of sending progressive data. This can lead to unneeded memory consumption for the *Dealer* and *Callee*, due to the need to store temporary metadata about ongoing calls.

Callee Leaving

The *Dealer*'s behavior for when a *Callee* leaves or disconnects during a call with progressive invocations shall be the same as in a basic, non-progressive call. That is, the *Dealer* sends an ERROR message to the *Caller* with the `wamp.error.canceled` URI. See Callee Leaving During an RPC Invocation under the Basic Profile.



Continuations on Completed Calls

A call with progressive invocations is considered completed by the *Dealer* when the latter receives a correlated final YIELD or ERROR message from the *Callee*, or when the *Callee* leaves the realm.

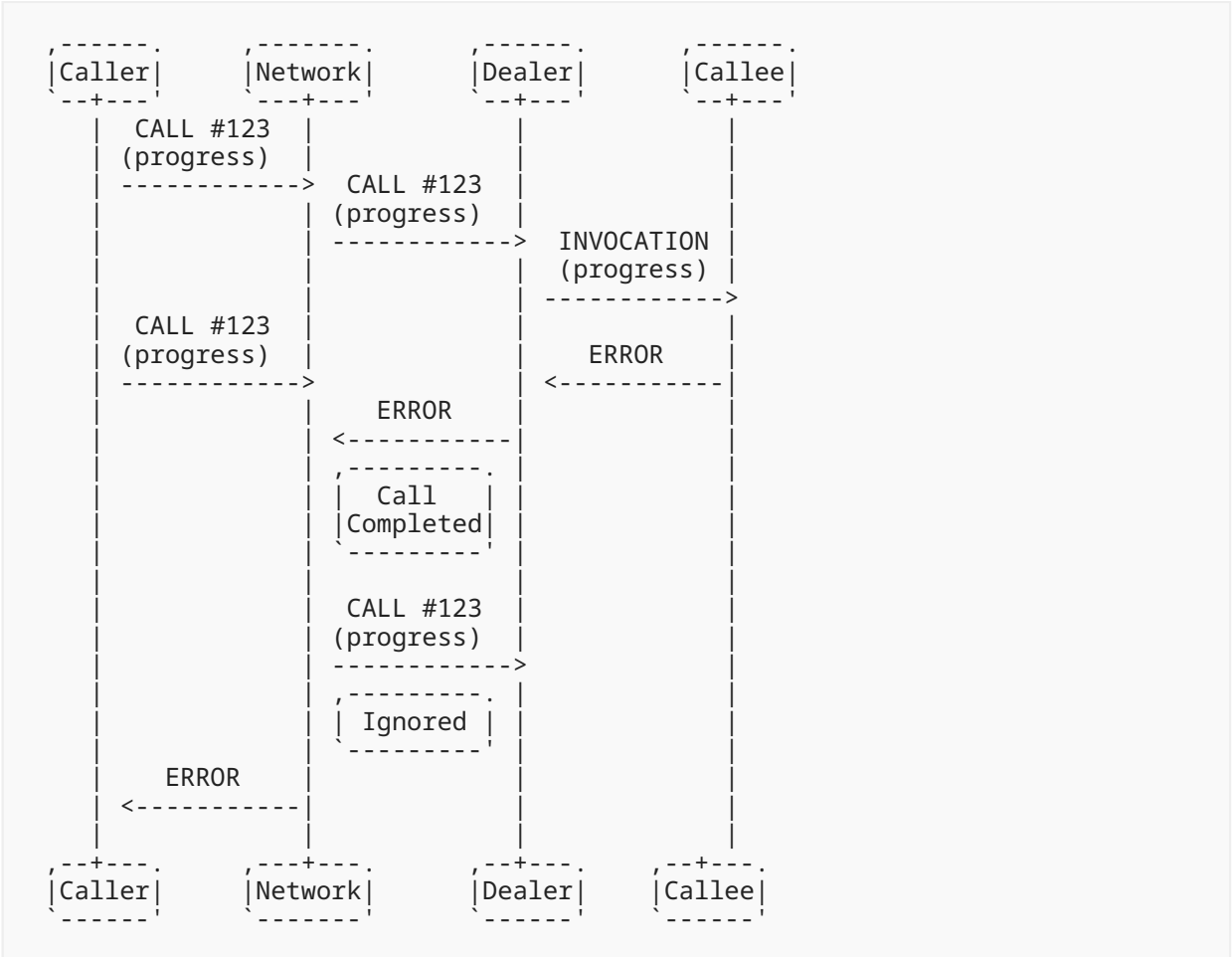
Due to network delays, the *Caller* may be unaware that the call is completed by time it sends another progressive CALL continuation.

When a *Dealer* receives a CALL under the following conditions:

- the *Dealer* supports Progressive Call Invocations,
- the CALL request ID does not correspond to a new call request, and,
- the CALL request ID does not match any RPC invocation in progress,

then it MUST ignore and discard that CALL message without any further correlated response to the *Caller*. The rationale for this is that the *Caller* will eventually receive a correlated RESULT or ERROR message from the previous call completion and will be able to handle the call completion accordingly.

The sequence diagram below illustrates this sitation, where the Network actor models network delay.



From the *Callee*'s perspective, a call with progressive invocations is considered completed when the *Callee* sends a correlated final YIELD or ERROR message.

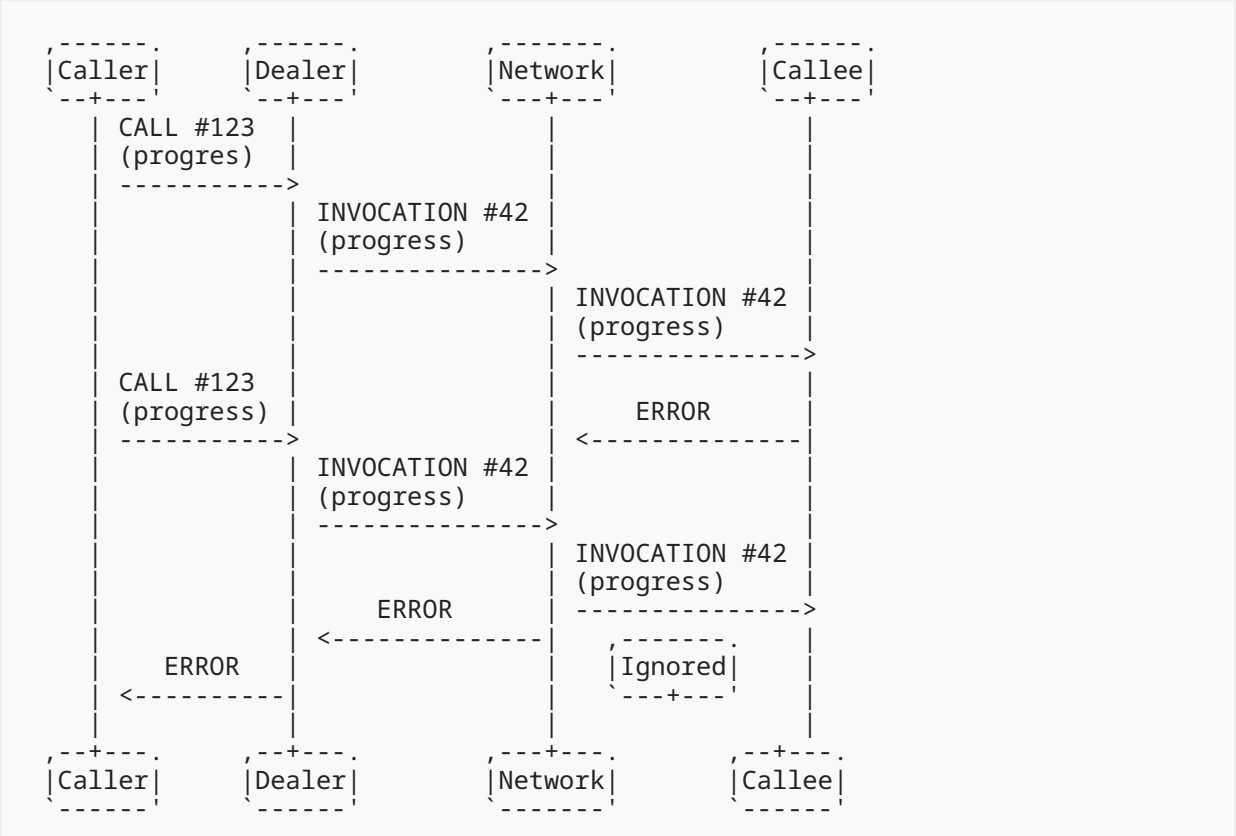
Due to network delays, the *Dealer* may be unaware that the call is completed by time it sends another progressive INVOCATION.

When a *Callee* receives an INVOCATION under the following conditions:

- the *Callee* supports Progressive Call Invocations,
- the INVOCATION request ID does not correspond to a new call request, and,
- the INVOCATION request ID does not match any RPC invocation in progress,

then it MUST ignore and discard that INVOCATION message without any further correlated response to the *Dealer*. The rationale for this is that the *Dealer* will eventually receive a correlated YIELD or ERROR message from the *Callee* and then send a correlated RESULT or ERROR message to the *Caller*, thus the *Caller* and the *Dealer* will both be able to handle the call completion accordingly.

The sequence diagram below illustrates this sitation, where the Network actor models network delay.



Verification of CALL Request IDs

When sending a CALL that continues a progressive call, its request ID is the same as the CALL that initiated the progressive call invocation. Therefore, when progressive call invocations are enabled, request IDs from CALL messages may not appear sequential. For example:

1. `[CALL, 1, {"progress":true}, "foo"]`
2. `[CALL, 2, {}, "bar"]`
3. `[CALL, 1, {}, "foo"]` (OK, continues CALL request #1)

The requirement for CALL request IDs to be sequential session scope (see Protocol Violations) must therefore only apply to **new** RPC transactions:

1. `[CALL, 1, {"progress":true}, "foo"]`
2. `[CALL, 2, {}, "bar"]`
3. `[CALL, 4, {}, "baz"]` (protocol violation, request ID 3 expected)

Let us define *watermark* as the maximum valid request ID of all received CALL messages during a *Dealer*'s run time.

Let us also define *continuation candiate* as a CALL with a request ID that is equal to or less than the *watermark*.

When a *Dealer* receives a CALL with a request ID that is exactly one above the watermark, then it shall be considered a new RPC transaction, and the requirement for sequential session scope IDs is verified.

When a *Dealer* receives a CALL with a request ID that is greater than one above the watermark, then this corresponds to a gap in the session scope ID sequence and MUST always be treated as a protocol violation.

When a *Dealer* receives a CALL with a request ID that is equal to or less than the watermark, then it is considered as a *continuation candidate* for a progressive invocation transfer.

As discussed in the previous section, a *Caller* may be unaware that a progressive invocation transfer is completed while sending a CALL continuation for that progressive invocation. Therefore, the *Dealer* cannot simply just check against progressive transfers in progress when verifying the validity of continuation candidates. It must also consider past progressive transfers that have been completed.

In order to validate the request ID of continuation candidates, it is suggested that a *Dealer* maintain a table of request IDs of completed progressive invocation transfers, where each entry is kept for a limited *grace period*. When a *Dealer* receives a continuation candidate with a request ID that is not in that table, nor in the list of active progressive invocations, then it is considered a protocol violation. Care must be taken in choosing the grace period: too short and data races can occur with slow networks, too long and memory usage of the table may become excessive when frequent progressive calls are made.

Due to resource constraints, it may not be desireable to implement such a grace period table, so *Dealers* MAY instead discard continuation candidates with request IDs that cannot be found in the list of active progressive invocation transfers.

The following pseudocode summarizes the algorithm for verifying CALL request IDs when progressive call invocations are enabled:

```
if (request_id == watermark + 1)
    watermark = watermak + 1
    initiate_new_call()
else if (request_id > watermark + 1)
    // Gap in request IDs
    abort_session("wamp.error.protocol_violation")
else
    // Continuation candidate
    if (active_call_records.contains(request_id, procedure_uri))
        record = active_call_records.at(request_id)
        if (record.is_progressive_invocation_call())
            continue_call()
        else
            abort_session("wamp.error.protocol_violation")
        endif
    else if (strict_request_id_verification_enabled)
        if (grace_period_table.contains(request_id, procedure_uri))
            discard_call()
        else
            abort_session("wamp.error.protocol_violation")
        endif
    else
        discard_call()
    endif
endif
```

Ignoring Progressive Call Invocations

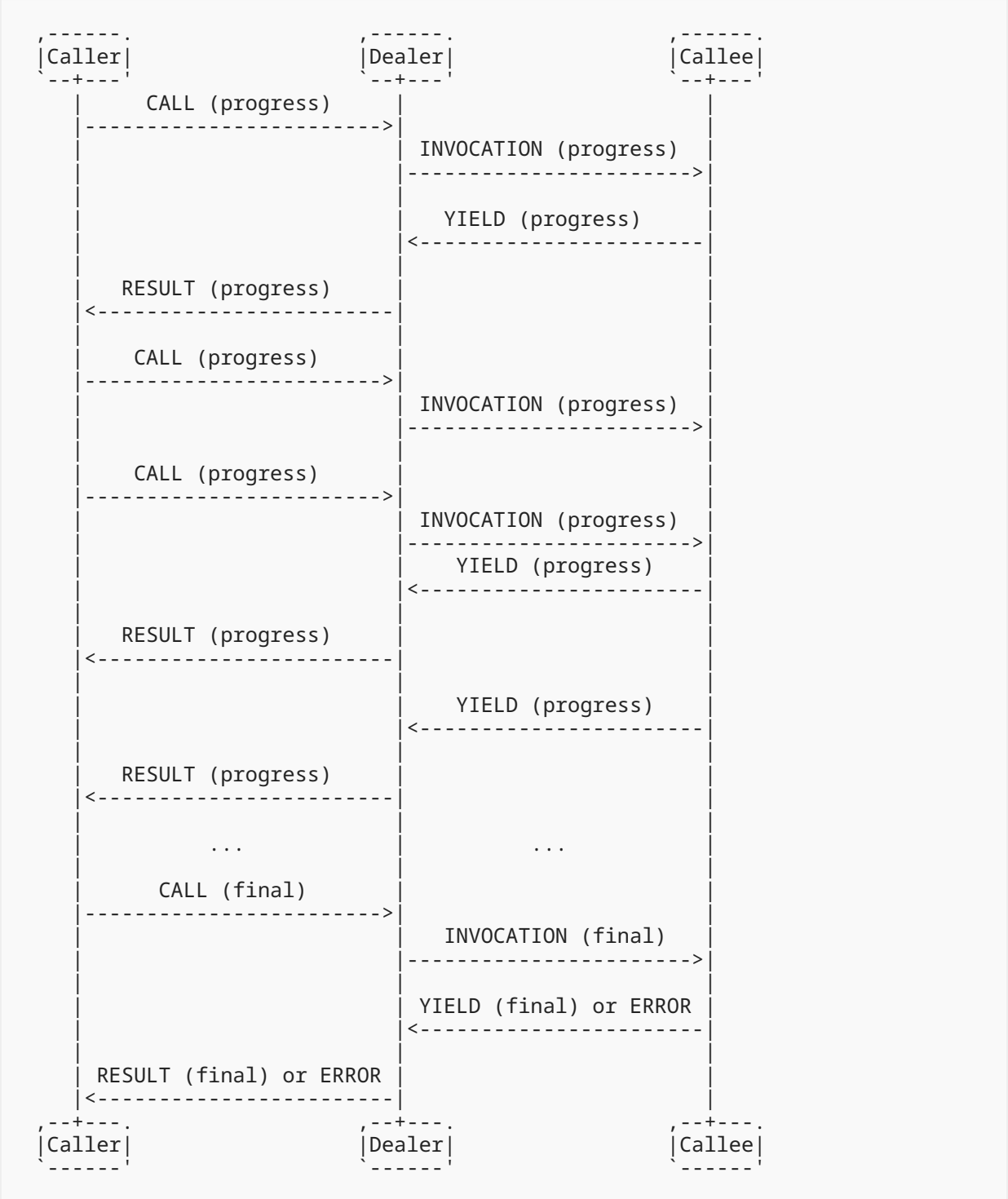
Unlike some other advanced features, a *Callee* cannot be unaware of progressive call invocations. Therefore, if a *Callee* doesn't support this feature, the *Dealer* MUST respond to the *Caller* with an `wamp.error.feature_not_supported` error message.

A *Callee* that supports progressive call invocations, but does not support call canceling, shall be considered by the *Dealer* as not supporting progressive call invocations.

Progressive Call Invocations with Progressive Call Results

Progressive Call Invocations may be used in conjunction with *Progressive Call Results* if the *Caller*, *Dealer*, and *Callee* all support both features. This allows the *Callee* to start sending partial results back to the *Caller* after receiving one or more initial payload chunks. Efficient two-way streams between a *Caller* and *Callee* can be implemented this way.

The following message flow illustrates a call using progressive call invocations when a *Callee* starts sending progressive call results immediately. Note that `YIELD` messages don't need to be matched with `CALL`/`INVOCATION` messages. For example, the *caller* can send a few `CALL` messages before starting to receive `RESULT` messages; they do not need to be matched pairs.



Because they are part of the same call operation, the request ID is the same in all `CALL`, `INVOCATION`, `YIELD`, and `ERROR` messages in the above exchange.

Freezing of Options in Progressive Call Invocations

Except for `progress`, items in the `Options` dictionary of the **initiating** progressive `CALL` shall be effective for the entirety of the progressive call request. Only the `progress` option shall be considered by the *Dealer* in subsequent progressive call invocations (within the same overall request). Except for `progress`, items in the `Details` dictionary of corresponding `INVOCATION` messages shall be based on the initiating progressive `CALL` only.

For example, if `disclose_me=true` was specified in the initiating progressive call, all subsequent progressive call invocations (within the same call) shall be considered by the *Dealer* to implictly have `disclose_me=true` as well. That is, all `INVOCATION` messages associated with the overall request shall contain caller identify information.

Any option besides `progress` SHOULD be omitted altogether by the *Caller* in subsequent progressive call invocations. Not having to repeat (and not being able to change) options is more in tune with the concept of a media stream where options are set up initially, and the source (*Caller*) only needs to keep uploading more data thereafter.

In subsequent call invocations:

- *Dealers* and *Callees* MUST ignore any option except for `progress`
- *Dealers* SHOULD NOT propagate any option passed by the *Caller*, except for `progress`

Dealers and *Callees* are responsible for remembering the options of the initiating progressive `CALL`.

For reference, here is a list of options that are frozen upon the initial progressive call invocations:

- `CALL.Options.disclose_me|bool`
- `CALL.Options.ppt_cipher|string`
- `CALL.Options.ppt_keyid|string`
- `CALL.Options.ppt_scheme|string`
- `CALL.Options.ppt_serializer|string`
- `CALL.Options.receive_progress|bool`
- `CALL.Options.rkey`
- `CALL.Options.runmode|string`
- `CALL.Options.timeout|integer`

These rules concerning progressive call invocation options also apply to custom options that are not part of this specification.

11.3. Call Timeouts

The Call Timeouts feature allows for **automatic** cancellation of a remote procedure call by the *Dealer* or *Callee* after a specified time duration.

A *Caller* specifies a timeout by providing

```
    CALL.Options.timeout|integer
```

in milliseconds. Automatic call timeouts are deactivated if there is no `timeout` option, or if its value is `0`.

Dealer-Initiated Timeouts

If the *Callee* does not support Call Timeouts, a *Dealer* supporting this feature MUST start a timeout timer upon receiving a `CALL` message with a `timeout` option. The message flow for call timeouts is identical to Call Canceling, except that there is no `CANCEL` message that originates from the *Caller*. The cancellation mode is implicitly `killnowait` if the *Callee* supports call cancellation, otherwise the cancellation mode is `skip`.

The error message that is returned to the *Caller* MUST use `wamp.error.timeout` as the reason URI.

Callee-Initiated Timeouts

If the *Callee* supports Call Timeouts, the *Dealer* MAY propagate the `CALL.Options.timeout|integer` option via the `INVOCATION` message and allow the *Callee* to handle the timeout logic. If the operation times out, the *Callee* MUST return an `ERROR` message with `wamp.error.timeout` as the reason URI.

Callees wanting to handle the timeout logic MAY specify this intention via the `REGISTER.Options.forward_timeout|boolean` option. The *Dealer*, upon receiving a CALL with the `timeout` option set, checks if the matching RPC registration had the `forward_timeout` option set, then accordingly either forwards the timeout value or handles the timeout logic locally without forwarding the timeout value.

Dealers MAY choose to override the `REGISTER.Options.forward_timeout|boolean` option based on router configuration. For example, if a *Dealer* is resource-constrained and does not wish to maintain a queue of pending call timeouts, it may decide to always forward the CALL timeout option to *Callees*.

Caller-Initiated Timeouts

Callers may run their own timeout timer and send a `CANCEL` message upon timeout. This is permitted if the *Dealer* supports Call Canceling and is not considered to be a usage of the Call Timeouts feature.

Feature Announcement

Support for this feature MUST be announced by *Dealers* (`role := "dealer"`) and MAY be announced by *Callees* (`role := "callee"`) via

```
    HELLO.Details.roles.<role>.features.call_timeout|bool := true
```

If a *Callee* does not support Call Timeouts, it may optionally announce support for Call Cancellation via

```
    HELLO.Details.roles.<role>.features.call_canceling|bool := true
```

11.4. Call Canceling

A *Caller* might want to actively cancel a call that was issued, but not has yet returned. An example where this is useful could be a user triggering a long running operation and later changing his mind or no longer willing to wait.

Feature Announcement

Support for this feature MUST be announced by *Callers* (`role := "caller"`), *Callees* (`role := "callee"`) and *Dealers* (`role := "dealer"`) via

```
    HELLO.Details.roles.<role>.features.call_canceling|bool := true
```

Message Flow

The message flow between *Callers*, a *Dealer* and *Callees* for canceling remote procedure calls involves the following messages:

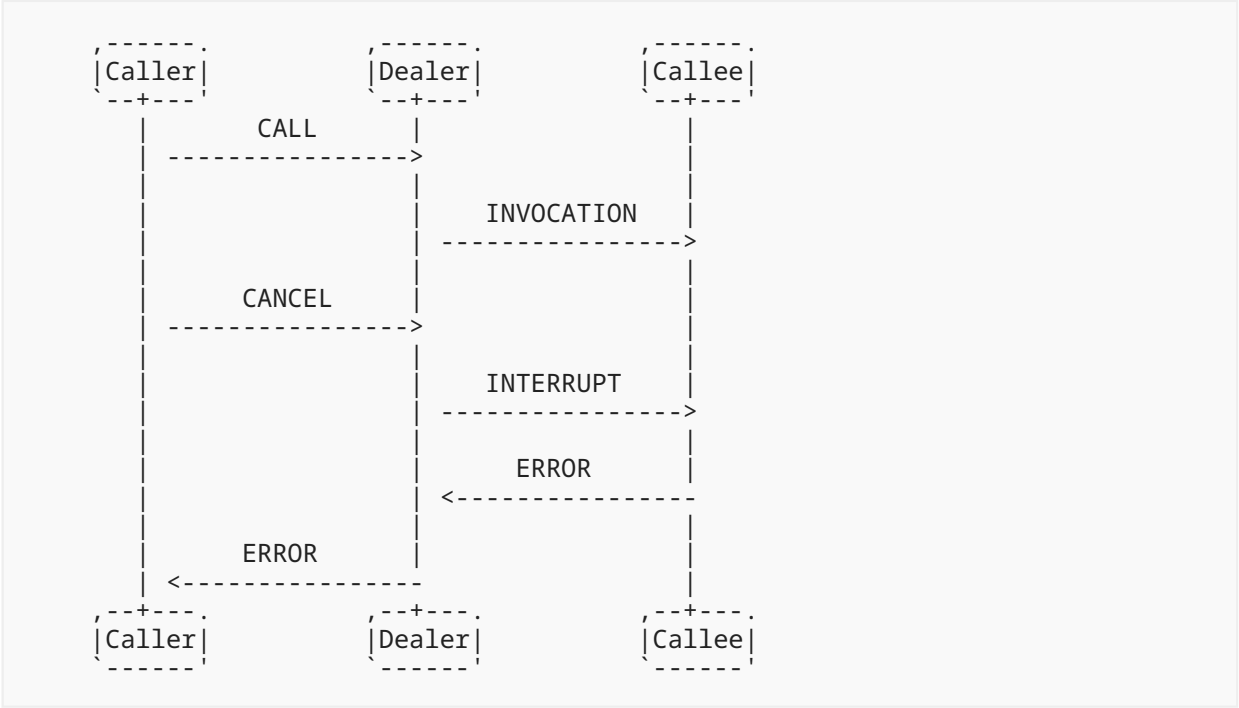
- `CANCEL`
- `INTERRUPT`
- `ERROR`

A call may be canceled at the *Callee* or at the *Dealer* side. Cancellation behaves differently depending on the mode:

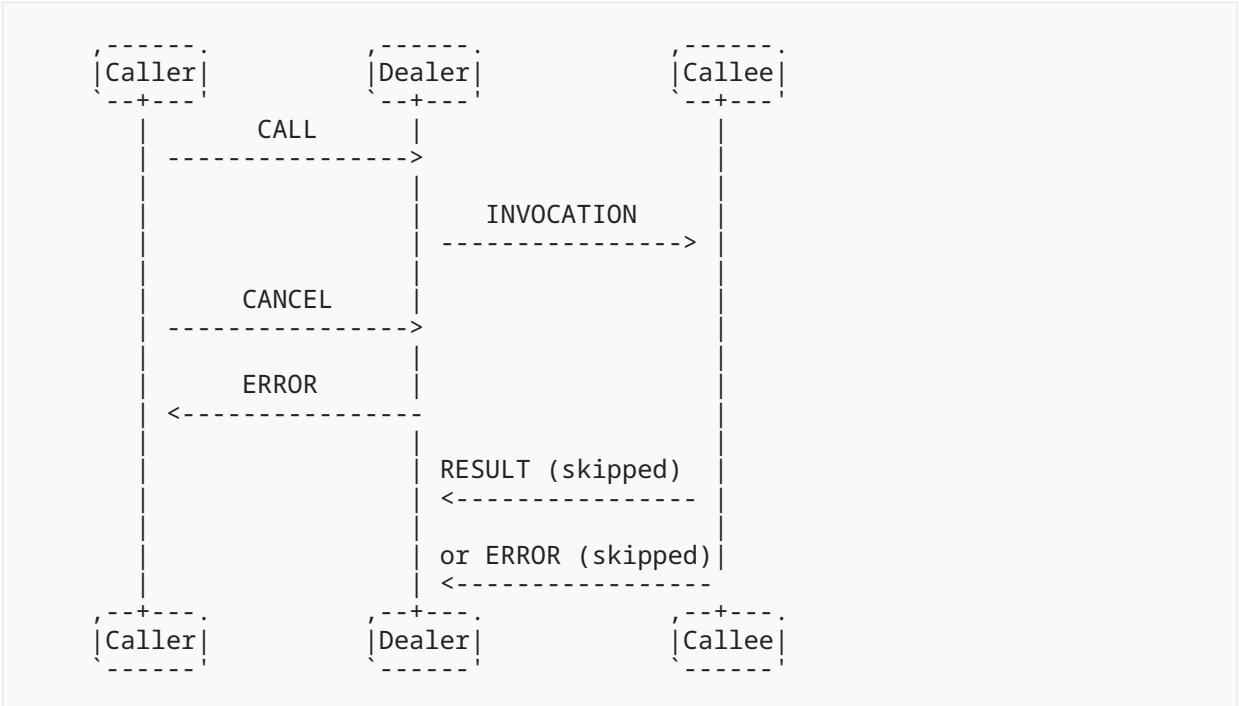
- **skip**: The pending call is canceled and `ERROR` is sent immediately back to the caller. No `INTERRUPT` is sent to the callee and the result is discarded when received.
- **kill**: `INTERRUPT` is sent to the callee, but `ERROR` is not returned to the caller until after the callee has responded to the canceled call. In this case the caller may receive `RESULT` or `ERROR` depending whether the callee finishes processing the invocation or the interrupt first.
- **killnowait**: The pending call is canceled and `ERROR` is sent immediately back to the caller. `INTERRUPT` is sent to the callee and any response to the invocation or interrupt from the callee is discarded when received.

If the callee does not support call canceling, then behavior is **skip**.

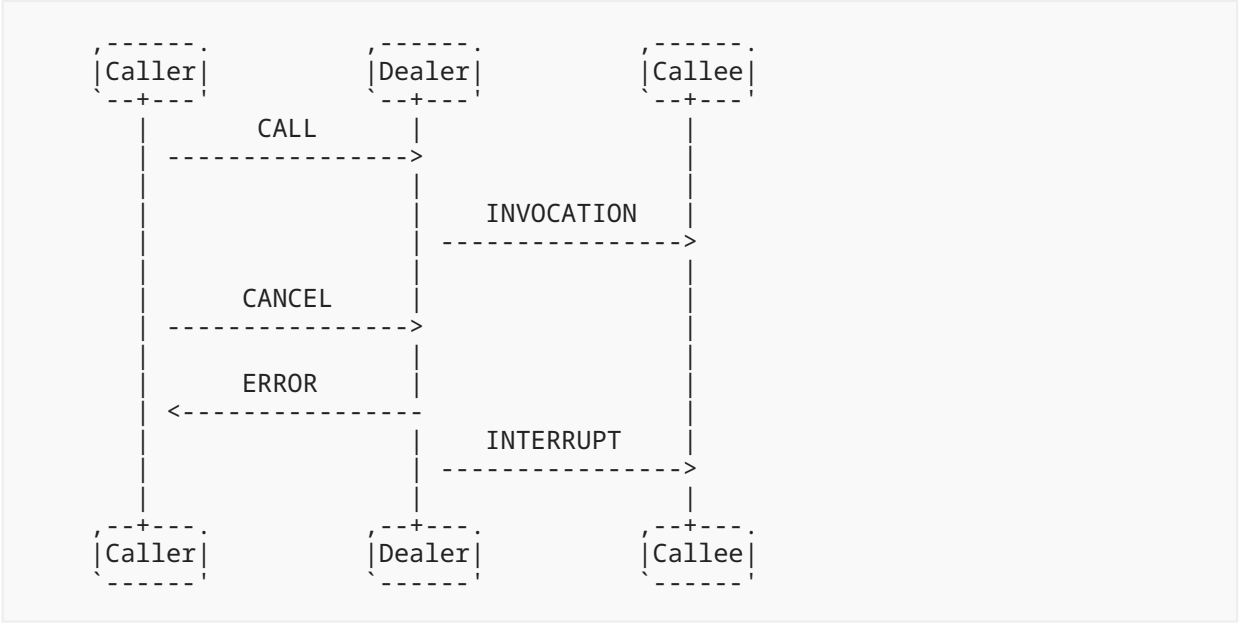
Message flow during call canceling when *Callee* supports this feature and mode is `kill`



Message flow during call canceling when *Callee* does not support this feature or mode is `skip`



Message flow during call canceling when *Callee* supports this feature and mode is `killnowait`



A *Caller* cancels a remote procedure call initiated (but not yet finished) by sending a `CANCEL` message to the *Dealer*:

```
[CANCEL, CALL.Request|id, Options|dict]
```

A *Dealer* cancels an invocation of an endpoint initiated (but not yet finished) by sending a `INTERRUPT` message to the *Callee*:

```
[INTERRUPT, INVOCATION.Request|id, Options|dict]
```

Options:

```
CANCEL.Options.mode|string == "skip" | "kill" | "killnowait"
```

Ignoring Results after Cancel

After the *Dealer* sends an `INTERRUPT` when mode="killnowait", any responses from the *Callee* are ignored. This means that it is not necessary for the *Callee* to respond with an `ERROR` message, when mode="killnowait", since the *Dealer* ignores it.

11.5. Call Re-Routing

A *CALLEE* might not be able to attend to a call. This may be due to a multitude of reasons including, but not limited to:

- *CALLEE* is busy handling other requests and is not able to attend
- *CALLEE* has dependency issues which prevent it from being able to fulfil the request

- In a HA environment, the *Callee* knows that it is scheduled to be taken off the HA cluster and as such should not handle the request.

A *unavailable* response allows for **automatic** reroute of a call by the *Dealer* without the *CALLER* ever having to know about it.

When such a situation occurs, the *Callee* responds to a `INVOCATION` message with the error uri:

wamp.error.unavailable

When the *Dealer* receives the `wamp.error.unavailable` message in response to an `INVOCATION`, it will reroute the `CALL` to another *registration* according to the rerouting rules of the `invocation_policy` of the `procedure`, as given below.

Feature Announcement

Support for this feature MUST be announced by *Callees* (`role := "callee"`) and *Dealers* (`role := "dealer"`) via

HELLO.Details.roles.<role>.features.call_reroute|bool := true

Rerouting Rules

The *Dealer* MUST adhere to the `invocation policy` of the `procedure` when rerouting the `CALL`, while assuming that the `unavailable` registration virtually does not exist.

For different `invocation policy` the *Dealer* MUST follow:

Invocation Policy	Operation
`single`	Responds with a `wamp.error.no_available_callee` error message to the *CALLER*
`roundrobin`	Picks the next `registration` from the *Registration Queue* of the *Procedure*
`random`	Picks another `registration` at random from the Registration Queue of the *Procedure*, as long as it is not the same `registration`
`first`	Picks the `registration` which was registered after the *called* registration was registered
`last`	Picks the `registration` which was registered right before the *called* registration was registered

Table 8

Failure Scenario

In case all available registrations of a *Procedure* responds with a `wamp.error.unavailable` for a *CALL*, the *Dealer* MUST respond with a `wamp.error.no_available_callee` to the *CALLER*

11.6. Caller Identification

A *Caller* MAY **request** the disclosure of its identity (its WAMP session ID) to endpoints of a routed call via

```
CALL.Options.disclose_me|bool := true
```

Example

```
[48, 7814135, {"disclose_me": true}, "com.myapp.echo",
    ["Hello, world!"]]
```

If above call is issued by a *Caller* with WAMP session ID `3335656`, the *Dealer* sends an `INVOCATION` message to *Callee* with the *Caller*'*s* WAMP session ID in `INVOCATION.Details.caller`:

Example

```
[68, 6131533, 9823526, {"caller": 3335656}, ["Hello, world!"]]
```

Note that a *Dealer* MAY disclose the identity of a *Caller* even without the *Caller* having explicitly requested to do so when the *Dealer* configuration (for the called procedure) is setup to do so.

Identity Information

When *Caller* disclosure is allowed for a particular CALL message, the corresponding `INVOCATION` message MUST contain an `INVOCATION.Details.caller|integer` property set to the *Caller*'s session ID.

The following additional properties MAY also be included::

- `INVOCATION.Details.caller_authid|string`
- `INVOCATION.Details.caller_authrole|string`

Feature Announcement

Support for this feature MUST be announced by *Callers* (`role := "caller"`), *Callees* (`role := "callee"`) and *Dealers* (`role := "dealer"`) via

```
HELLO.Details.roles.<role>.features.
     caller_identification|bool := true
```

Request Identification

A *Dealer* MAY deny a *Caller*'*s* request to disclose its identity:

Example

```
[8, 48, 7814135, "wamp.error.disclose_me.not_allowed"]
```

A *Callee* MAY **request** the disclosure of caller identity via

```
REGISTER.Options.disclose_caller|bool := true
```

Example

```
[64, 927639114088448, {"disclose_caller":true},
    "com.maypp.add2"]
```

With the above registration, the registered procedure is called with the caller's sessionID as part of the call details object.

11.7. Call Trust Levels

A *Dealer* may be configured to automatically assign *trust levels* to calls issued by *Callers* according to the *Dealer* configuration on a per-procedure basis and/or depending on the application defined role of the (authenticated) *Caller*.

A *Dealer* supporting trust level will provide

```
INVOCATION.Details.trustlevel|integer
```

in an `INVOCATION` message sent to a *Callee*. The trustlevel `0` means lowest trust, and higher integers represent (application-defined) higher levels of trust.

Example

```
[68, 6131533, 9823526, {"trustlevel": 2}, ["Hello, world!"]]
```

In above event, the *Dealer* has (by configuration and/or other information) deemed the call (and hence the invocation) to be of trustlevel `2`.

Feature Announcement

Support for this feature MUST be announced by *Callees* (`role := "callee"`) and *Dealers* (`role := "dealer"`) via

```
HELLO.Details.roles.<role>.features.call_trustlevels|bool := true
```

11.8. Pattern-based Registrations

By default, *Callees* register procedures with **exact matching policy**. That is a call will only be routed to a *Callee* by the *Dealer* if the procedure called (`CALL.Procedure`) *exactly* matches the endpoint registered (`REGISTER.Procedure`).

A *Callee* might want to register procedures based on a *pattern*. This can be useful to reduce the number of individual registrations to be set up or to subscribe to a open set of topics, not known beforehand by the *Subscriber*.

If the *Dealer* and the *Callee* support **pattern-based registrations**, this matching can happen by

- **prefix-matching policy**
- **wildcard-matching policy**

Feature Announcement

Support for this feature MUST be announced by *Callees* (`role := "callee"`) and *Dealers* (`role := "dealer"`) via

```
    HELLO.Details.roles.<role>.features.
        pattern_based_registration|bool := true
```

11.8.1. Prefix Matching

A *Callee* requests **prefix-matching policy** with a registration request by setting

```
    REGISTER.Options.match|string := "prefix"
```

Example

```
    [
        64,
        612352435,
        {
            "match": "prefix"
        },
        "com.myapp.myobject1"
    ]
```

When a **prefix-matching policy** is in place, any call with a procedure that has `REGISTER.Procedure` as a *prefix* will match the registration, and potentially be routed to *Callees* on that registration.

In above example, the following calls with `CALL.Procedure`

- `com.myapp.myobject1.myprocedure1`

- `com.myapp.myobject1-mysubobject1`
- `com.myapp.myobject1.mysubobject1.myprocedure1`
- `com.myapp.myobject1`

will all apply for call routing. A call with one of the following `CALL.Procedure`

- `com.myapp.myobject2`
- `com.myapp.myobject`

will not apply.

11.8.2. Wildcard Matching

A *Callee* requests **wildcard-matching policy** with a registration request by setting

```
    REGISTER.Options.match|string := "wildcard"
```

Wildcard-matching allows to provide wildcards for **whole** URI components.

Example

```
    [
        64,
        612352435,
        {
            "match": "wildcard"
        },
        "com.myapp..myprocedure1"
    ]
```

In the above registration request, the 3rd URI component is empty, which signals a wildcard in that URI component position. In this example, calls with `CALL.Procedure` e.g.

- `com.myapp.myobject1.myprocedure1`
- `com.myapp.myobject2.myprocedure1`

will all apply for call routing. Calls with `CALL.Procedure` e.g.

- `com.myapp.myobject1.myprocedure1.mysubprocedure1`
- `com.myapp.myobject1.myprocedure2`
- `com.myapp2.myobject1.myprocedure1`

will not apply for call routing.

When a single call matches more than one of a *Callees* registrations, the call MAY be routed for invocation on multiple registrations, depending on call settings.

11.8.3. Design Aspects

No set semantics

Since each *Callee*'s' registrations "stands on its own", there is no *set semantics* implied by pattern-based registrations.

E.g. a *Callee* cannot register to a broad pattern, and then unregister from a subset of that broad pattern to form a more complex registration. Each registration is separate.

Calls matching multiple registrations

There can be situations, when some call URI matches more then one registration. In this case a call is routed to one and only one best matched RPC registration, or fails with ERROR `wamp.error.no_such_procedure`.

The following algorithm MUST be applied to find a single RPC registration to which a call is routed:

1. Check for exact matching registration. If this match exists — (U+2014) use it.
2. If there are prefix-based registrations, find the registration with the longest prefix match. Longest means it has more URI components matched, e.g. for call URI `a1.b2.c3.d4` registration `a1.b2.c3` has higher priority than registration `a1.b2`. If this match exists — (U+2014) use it.
3. If there are wildcard-based registrations, find the registration with the longest portion of URI components matched before each wildcard. E.g. for call URI `a1.b2.c3.d4` registration `a1.b2..d4` has higher priority than registration `a1...d4`, see below for more complex examples. If this match exists — (U+2014) use it.
4. If there is no exact match, no prefix match, and no wildcard match, then *Dealer* MUST return ERROR `wamp.error.no_such_procedure`.

Examples

```
Registered RPCs:
    1. 'a1.b2.c3.d4.e55' (exact),
    2. 'a1.b2.c3' (prefix),
    3. 'a1.b2.c3.d4' (prefix),
    4. 'a1.b2..d4.e5',
    5. 'a1.b2.c33..e5',
    6. 'a1.b2..d4.e5..g7',
    7. 'a1.b2..d4..f6.g7'

Call request RPC URI: 'a1.b2.c3.d4.e55' →
    exact match. Use RPC 1
Call request RPC URI: 'a1.b2.c3.d98.e74' →
    no exact match, single prefix match. Use RPC 2
Call request RPC URI: 'a1.b2.c3.d4.e325' →
    no exact match, 2 prefix matches (2,3), select longest one.
    Use RPC 3
Call request RPC URI: 'a1.b2.c55.d4.e5' →
    no exact match, no prefix match, single wildcard match.
    Use RPC 4
Call request RPC URI: 'a1.b2.c33.d4.e5' →
    no exact match, no prefix match, 2 wildcard matches (4,5),
    but RPC 5 has longer first portion (a1.b2.c33). Use RPC 5
Call request RPC URI: 'a1.b2.c88.d4.e5.f6.g7' →
    no exact match, no prefix match, 2 wildcard matches (6,7),
    both having equal first portions (a1.b2), but RPC 6 has longer
    second portion (d4.e5). Use RPC 6
Call request RPC URI: 'a2.b2.c2.d2.e2' →
    no exact match, no prefix match, no wildcard match.
    Return wamp.error.no_such_procedure
```

Concrete procedure called

If an endpoint was registered with a pattern-based matching policy, a *Dealer* MUST supply the original `CALL.Procedure` as provided by the *Caller* in

```
    INVOCATION.Details.procedure
```

to the *Callee*.

Example

```
    [
        68,
        6131533,
        9823527,
        {
            "procedure": "com.myapp.procedure.proc1"
        },
        ["Hello, world!"]
    ]
```

11.9. Shared Registration

Feature status: **alpha**

As a default, only a single **Callee** may register a procedure for a URI.

There are use cases where more flexibility is required. As an example, for an application component with a high computing load, several instances may run, and load balancing of calls across these may be desired. As another example, in an application a second or third component providing a procedure may run, which are only to be called in case the primary component is no longer reachable (hot standby).

When shared registrations are supported, then the first **Callee** to register a procedure for a particular URI MAY determine that additional registrations for this URI are allowed, and what **Invocation Rules** to apply in case such additional registrations are made.

This is done through setting

```
    REGISTER.Options.invoke|string := <invocation_policy>
```

where `<invocation_policy>` is one of

- 'single'
- 'roundrobin'
- 'random'
- 'first'
- 'last'

If the option is not set, 'single' is applied as a default.

With 'single', the **Dealer** MUST fail all subsequent attempts to register a procedure for the URI while the registration remains in existence.

With the other values, the **Dealer** MUST fail all subsequent attempts to register a procedure for the URI where the value for this option does not match that of the initial registration.

Feature Announcement

Support for this feature MUST be announced by *Callees* (`role := "callee"`) and *Dealers* (`role := "dealer"`) via

```
    HELLO.Details.roles.<role>.features.
        shared_registration|bool := true
```

11.9.1. Load Balancing

For sets of registrations registered using either 'roundrobin' or 'random', load balancing is performed across calls to the URI.

For 'roundrobin', callees are picked subsequently from the list of registrations (ordered by the order of registration), with the picking looping back to the beginning of the list once the end has been reached.

For 'random' a callee is picked randomly from the list of registrations for each call.

11.9.2. Hot Stand-By

For sets of registrations registered using either 'first' or 'last', the first respectively last callee on the current list of registrations (ordered by the order of registration) is called.

11.10. Sharded Registration

Feature status: **sketch**

Sharded Registrations are intended to allow calling a procedure which is offered by a sharded database, by routing the call to a single shard.

Feature Announcement

Support for this feature MUST be announced by *Callers* (`role := "caller"`), *Callees* (`role := "callee"`) and *Dealers* (`role := "dealer"`) via

```
HELLO.Details.roles.<role>.features.sharded_registration|bool := true
```

11.10.1. "All" Calls

Write me.

11.10.2. "Partitioned" Calls

If `CALL.Options.runmode == "partition"`, then `CALL.Options.rkey` MUST be present.

The call is then routed to all endpoints that were registered ..

The call is then processed as for "All" Calls.

11.11. Registration Revocation

Feature status: **alpha**

This feature allows a *Dealer* to actively revoke a previously granted registration. To achieve this, the existing UNREGISTERED message is extended as described below.

Feature Announcement

Support for this feature MUST be announced by *Callees* (role := "callee") and *Dealers* (role := "dealer") via

```
    HELLO.Details.roles.<role>.features.
         registration_revocation|bool := true
```

If the *Callee* does not support registration_revocation, the *Dealer* may still revoke a registration to support administrative functionality. In this case, the *Dealer* MUST NOT send an **UNREGISTERED** message to the *Callee*. The *Callee* MAY use the registration meta event `wamp.registration.on_unregister` to determine whether a session is removed from a registration.

Extending UNREGISTERED

When revoking a registration, the router has no request ID to reply to. So it's set to zero and another argument is appended to indicate which registration to revoke. Optionally, a reason why the registration was revoked is also appended.

```
    [UNREGISTERED, 0, Details|dict]
```

where

- `Details.registration|bool` MUST be a previously issued registration ID.
- `Details.reason|string` MAY provide a reason as to why the registration was revoked.

Example

```
    [67, 0, {"registration": 1293722, "reason": "moving endpoint to other
callee"}]
```

12. Advanced PubSub

12.1. Subscriber Black- and Whitelisting

Subscriber Black- and Whitelisting is an advanced *Broker* feature where a *Publisher* is able to restrict the set of receivers of a published event.

Under normal Publish & Subscriber event dispatching, a *Broker* will dispatch a published event to all (authorized) *Subscribers* other than the *Publisher* itself. This set of receivers can be further reduced on a per-publication basis by the *Publisher* using **Subscriber Black- and Whitelisting**.

The *Publisher* can explicitly **exclude** *Subscribers* based on WAMP `sessionid`, `authid` or `authrole`. This is referred to as **Blacklisting**.

A *Publisher* may also explicitly define a **eligible** list of *Subscribers* based on WAMP `sessionid`, `authid` or `authrole`. This is referred to as **Whitelisting**.

Use Cases include the following.

Avoiding Callers from being self-notified

Consider an application that exposes a procedure to update a product price. The procedure might not only actually update the product price (e.g. in a backend database), but additionally publish an event with the updated product price, so that **all** application components get notified actively of the new price.

However, the application might want to exclude the originator of the product price update (the **Caller** of the price update procedure) from receiving the update event - as the originator naturally already knows the new price, and might get confused when it receives an update the **Caller** has triggered himself.

The product price update procedure can use `PUBLISH.Options.exclude|list[int]` to exclude the **Caller** of the procedure.

> Note that the product price update procedure needs to know the session ID of the **Caller** to be able to exclude him. For this, please see **Caller Identification**.

A similar approach can be used for other CRUD-like procedures.

Restricting receivers of sensitive information

Consider an application with users that have different `authroles`, such as "manager" and "staff" that publishes events with updates to "customers". The topics being published to could be structured like

```
com.example.myapp.customer.<customer ID>
```

The application might want to restrict the receivers of customer updates depending on the `authrole` of the user. E.g. a user authenticated under `authrole` "manager" might be allowed to receive any kind of customer update, including personal and business sensitive information. A user under `authrole` "staff" might only be allowed to receive a subset of events.

The application can publish **all** customer updates to the **same** topic `com.example.myapp.customer.<customer ID>` and use `PUBLISH.Options.eligible_authrole|list[string]` to safely restrict the set of actual receivers as desired.

Feature Definition

A *Publisher* may restrict the actual receivers of an event from the set of *Subscribers* through the use of

- Blacklisting Options
  - `PUBLISH.Options.exclude|list[int]`
  - `PUBLISH.Options.exclude_authid|list[string]`
  - `PUBLISH.Options.exclude_authrole|list[string]`
- Whitelisting Options
  - `PUBLISH.Options.eligible|list[int]`
  - `PUBLISH.Options.eligible_authid|list[string]`
  - `PUBLISH.Options.eligible_authrole|list[string]`

`PUBLISH.Options.exclude` is a list of integers with WAMP `sessionids` providing an explicit list of (potential) *Subscribers* that won't receive a published event, even though they may be subscribed. In other words, `PUBLISH.Options.exclude` is a **blacklist** of (potential) *Subscribers*.

`PUBLISH.Options.eligible` is a list of integers with WAMP WAMP `sessionids` providing an explicit list of (potential) *Subscribers* that are allowed to receive a published event. In other words, `PUBLISH.Options.eligible` is a **whitelist** of (potential) *Subscribers*.

The `exclude_authid`, `exclude_authrole`, `eligible_authid` and `eligible_authrole` options work similar, but not on the basis of WAMP `sessionid`, but `authid` and `authrole`.

An (authorized) *Subscriber* to topic T will receive an event published to T if and only if all of the following statements hold true:

1. if there is an `eligible` attribute present, the *Subscriber*'s `sessionid` is in this list
2. if there is an `eligible_authid` attribute present, the *Subscriber*'s `authid` is in this list
3. if there is an `eligible_authrole` attribute present, the *Subscriber*'s `authrole` is in this list
4. if there is an `exclude attribute` present, the *Subscriber*'s `sessionid` is NOT in this list
5. if there is an `exclude_authid` attribute present, the *Subscriber*'s `authid` is NOT in this list
6. if there is an `exclude_authrole` attribute present, the *Subscriber*'s `authrole` is NOT in this list

For example, if both `PUBLISH.Options.exclude` and `PUBLISH.Options.eligible` are present, the *Broker* will dispatch events published only to *Subscribers* that are not explicitly excluded in `PUBLISH.Options.exclude` **and** which are explicitly eligible via `PUBLISH.Options.eligible`.

Example

```
[
   16,
   239714735,
   {
      "exclude": [
         7891255,
         1245751
      ]
   },
   "com.myapp.mytopic1",
   [
      "Hello, world!"
   ]
]
```

The above event will get dispatched to all *Subscribers* of `com.myapp.mytopic1`, but not WAMP sessions with IDs `7891255` or `1245751` (and also not the publishing session).

Example

```
[
   16,
   239714735,
   {
      "eligible": [
         7891255,
         1245751
      ]
   },
   "com.myapp.mytopic1",
   [
      "Hello, world!"
   ]
]
```

The above event will get dispatched to WAMP sessions with IDs `7891255` or `1245751` only - but only if those are actually subscribed to the topic `com.myapp.mytopic1`.

Example

```
[
   16,
   239714735,
   {
      "eligible": [
         7891255,
         1245751,
         9912315
      ],
      "exclude": [
         7891255
      ]
   },
   "com.myapp.mytopic1",
   [
      "Hello, world!"
   ]
]
```

The above event will get dispatched to WAMP sessions with IDs `1245751` or `9912315` only, since `7891255` is excluded - but only if those are actually subscribed to the topic `com.myapp.mytopic1`.

Feature Announcement

Support for this feature MUST be announced by *Publishers* (`role := "publisher"`) and *Brokers* (`role := "broker"`) via

```
HELLO.Details.roles.<role>.features.
    subscriber_blackwhite_listing|bool := true
```

12.2. Publisher Exclusion

By default, a *Publisher* of an event will **not** itself receive an event published, even when subscribed to the `Topic` the *Publisher* is publishing to. This behavior can be overridden using this feature.

To override the exclusion of a publisher from its own publication, the `PUBLISH` message must include the following option:

```
PUBLISH.Options.exclude_me|bool
```

When publishing with `PUBLISH.Options.exclude_me := false`, the *Publisher* of the event will receive that event, if it is subscribed to the `Topic` published to.

Example

```
[
    16,
    239714735,
    {
        "exclude_me": false
    },
    "com.myapp.mytopic1",
    ["Hello, world!"]
]
```

In this example, the *Publisher* will receive the published event, if it is subscribed to `com.myapp.mytopic1`.

Feature Announcement

Support for this feature MUST be announced by *Publishers* (`role := "publisher"`) and *Brokers* (`role := "broker"`) via

```
HELLO.Details.roles.<role>.features.
    publisher_exclusion|bool := true
```

12.3. Publisher Identification

A *Publisher* may request the disclosure of its identity (its WAMP session ID) to receivers of a published event by setting

```
PUBLISH.Options.disclose_me|bool := true
```

Example

```
[16, 239714735, {"disclose_me": true}, "com.myapp.mytopic1",
    ["Hello, world!"]]
```

If above event is published by a *Publisher* with WAMP session ID `3335656`, the *Broker* would send an `EVENT` message to *Subscribers* with the *Publisher*'*s* WAMP session ID in `EVENT.Details.publisher`:

Example

```
[36, 5512315355, 4429313566, {"publisher": 3335656},
    ["Hello, world!"]]
```

Note that a *Broker* may deny a *Publisher*'*s* request to disclose its identity:

Example

```
[8, 16, 239714735, {}, "wamp.error.option_disallowed.disclose_me"]
```

A *Broker* may also (automatically) disclose the identity of a *Publisher* even without the *Publisher* having explicitly requested to do so when the *Broker* configuration (for the publication topic) is set up to do so.

Additional Identity Information

When publisher disclosure is allowed for a particular PUBLISH message, the corresponding `EVENT` message MAY also contain the following additional properties in its `Details` dictionary:

- `EVENT.Details.publisher_authid|string`
- `EVENT.Details.publisher_authrole|string`

Feature Announcement

Support for this feature MUST be announced by *Publishers* (`role := "publisher"`), *Brokers* (`role := "broker"`) and *Subscribers* (`role := "subscriber"`) via

```
HELLO.Details.roles.<role>.features.
    publisher_identification|bool := true
```

12.4. Publication Trust Levels

A *Broker* may be configured to automatically assign *trust levels* to events published by *Publishers* according to the *Broker* configuration on a per-topic basis and/or depending on the application defined role of the (authenticated) *Publisher*.

A *Broker* supporting trust level will provide

```
    EVENT.Details.trustlevel|integer
```

in an `EVENT` message sent to a *Subscriber*. The trustlevel `0` means lowest trust, and higher integers represent (application-defined) higher levels of trust.

Example

```
    [36, 5512315355, 4429313566, {"trustlevel": 2},
        ["Hello, world!"]]
```

In above event, the *Broker* has (by configuration and/or other information) deemed the event publication to be of trustlevel `2`.

Feature Announcement

Support for this feature MUST be announced by *Subscribers* (`role := "subscriber"`) and *Brokers* (`role := "broker"`) via

```
    HELLO.Details.roles.<role>.features.
        publication_trustlevels|bool := true
```

12.5. Pattern-based Subscription

By default, *Subscribers* subscribe to topics with **exact matching policy**. That is an event will only be dispatched to a *Subscriber* by the *Broker* if the topic published to (`PUBLISH.Topic`) *exactly* matches the topic subscribed to (`SUBSCRIBE.Topic`).

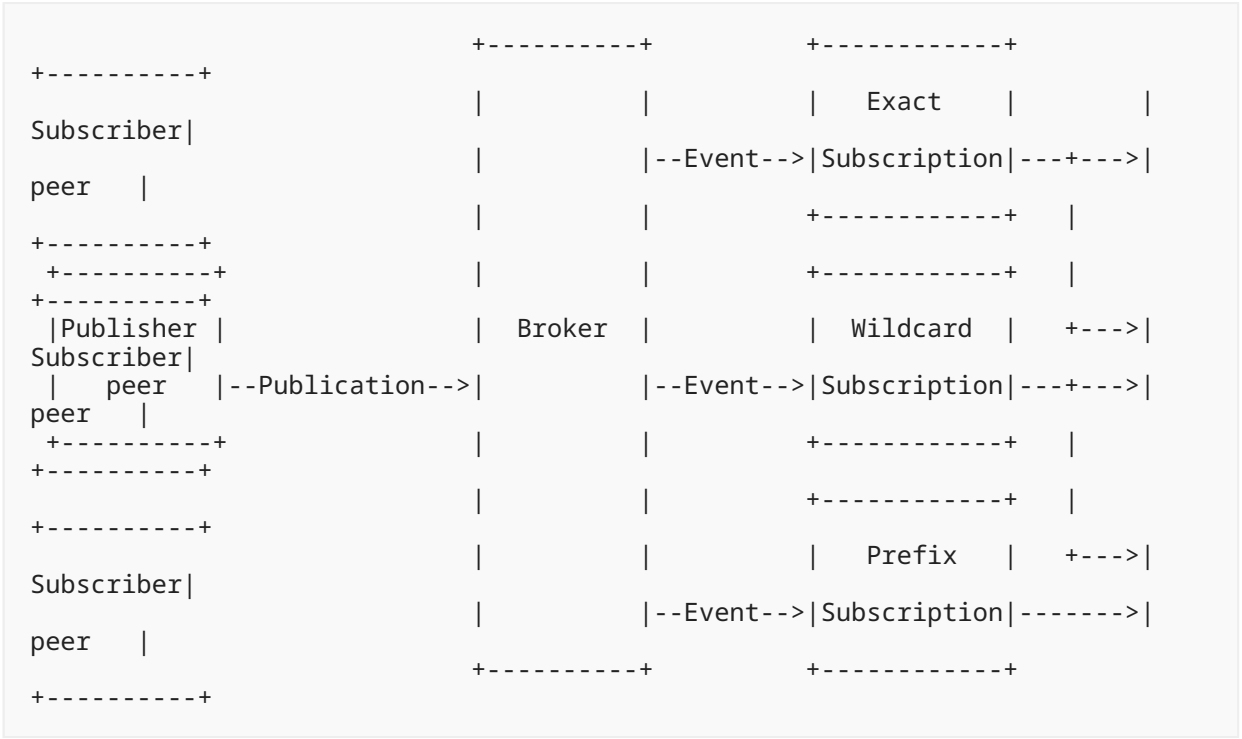
A *Subscriber* might want to subscribe to topics based on a *pattern*. This can be useful to reduce the number of individual subscriptions to be set up and to subscribe to topics the *Subscriber* is not aware of at the time of subscription, or which do not yet exist at this time.

Let's review the event publication flow. When one peer decides to publish a message to a topic, it results in a `PUBLISH` WAMP message with fields for the `Publication` id, `Details` dictionary, and, optionally, the payload arguments.

A given event received by the router from a publisher via a `PUBLISH` message will match one or more subscriptions:

- zero or one exact subscription
- zero or more prefix subscriptions
- zero or more wildcard subscriptions

The same published event is then forwarded to subscribers for every matching subscription. Thus, a given event might be sent multiple times to the same client under different subscriptions. Every subscription instance, based on a topic URI and some options, has a unique ID. All subscribers of the same subscription are given the same subscription ID.



If the *Broker* and the *Subscriber* support **pattern-based subscriptions**, this matching can happen by

- prefix-matching policy
- wildcard-matching policy

Feature Announcement

Support for this feature MUST be announced by *Subscribers* (`role := "subscriber"`) and *Brokers* (`role := "broker"`) via

```
    HELLO.Details.roles.<role>.features.
        pattern_based_subscription|bool := true
```

12.5.1. Prefix Matching

A *Subscriber* requests **prefix-matching policy** with a subscription request by setting

```
    SUBSCRIBE.Options.match|string := "prefix"
```

Example

```
    [
        32,
        912873614,
        {
            "match": "prefix"
        },
        "com.myapp.topic.emergency"
    ]
```

When a **prefix-matching policy** is in place, any event with a topic that has `SUBSCRIBE.Topic` as a *prefix* will match the subscription, and potentially be delivered to *Subscribers* on the subscription.

In the above example, events with `PUBLISH.Topic`

- `com.myapp.topic.emergency.11`
- `com.myapp.topic.emergency-low`
- `com.myapp.topic.emergency.category.severe`
- `com.myapp.topic.emergency`

will all apply for dispatching. An event with `PUBLISH.Topic` e.g. `com.myapp.topic.emerge` will not apply.

12.5.2. Wildcard Matching

A *Subscriber* requests **wildcard-matching policy** with a subscription request by setting

```
    SUBSCRIBE.Options.match|string := "wildcard"
```

Wildcard-matching allows to provide wildcards for **whole** URI components.

Example

```
    [
        32,
        912873614,
        {
            "match": "wildcard"
        },
        "com.myapp..userevent"
    ]
```

In above subscription request, the 3rd URI component is empty, which signals a wildcard in that URI component position. In this example, events with `PUBLISH.Topic`

- `com.myapp.foo.userevent`
- `com.myapp.bar.userevent`
- `com.myapp.a12.userevent`

will all apply for dispatching. Events with `PUBLISH.Topic`

- `com.myapp.foo.userevent.bar`
- `com.myapp.foo.user`
- `com.myapp2.foo.userevent`

will not apply for dispatching.

12.5.3. Design Aspects

No set semantics

Since each *Subscriber*'*s* subscription "stands on its own", there is no *set semantics* implied by pattern-based subscriptions.

E.g. a *Subscriber* cannot subscribe to a broad pattern, and then unsubscribe from a subset of that broad pattern to form a more complex subscription. Each subscription is separate.

Events matching multiple subscriptions

When a single event matches more than one of a *Subscriber*'*s* subscriptions, the event will be delivered for each subscription.

The *Subscriber* can detect the delivery of that same event on multiple subscriptions via `EVENT.PUBLISHED.Publication`, which will be identical.

Concrete topic published to

If a subscription was established with a pattern-based matching policy, a *Broker* MUST supply the original `PUBLISH.Topic` as provided by the *Publisher* in

```
EVENT.Details.topic|uri
```

to the *Subscribers*.

Example

```
[
    36,
    5512315355,
    4429313566,
    {
        "topic": "com.myapp.topic.emergency.category.severe"
    },
    ["Hello, world!"]
]
```

12.6. Sharded Subscription

Feature status: **alpha**

Support for this feature MUST be announced by *Publishers* (`role := "publisher"`), *Subscribers* (`role := "subscriber"`) and *Brokers* (`role := "broker"`) via

```
HELLO.Details.roles.<role>.features.shareded_subscriptions|
    bool := true
```

Resource keys: `PUBLISH.Options.rkey|string` is a stable, technical **resource key**.

> E.g. if your sensor has a unique serial identifier, you can use that.

Example

```
[16, 239714735, {"rkey": "sn239019"}, "com.myapp.sensor.sn239019.
    temperature", [33.9]]
```

Node keys: `SUBSCRIBE.Options.nkey|string` is a stable, technical **node key**.

> E.g. if your backend process runs on a dedicated host, you can use its hostname.

Example

```
[32, 912873614, {"match": "wildcard", "nkey": "node23"},
    "com.myapp.sensor..temperature"]
```

12.7. Event History

Instead of complex QoS for message delivery, a *Broker* may provide *Event History*. With event history, a *Subscriber* is responsible for handling overlaps (duplicates) when it wants "exactly-once" message processing across restarts.

The event history may be transient, or it may be persistent where it survives *Broker* restarts.

The *Broker* implementation may allow for configuration of event history on a per-topic or per-topic-pattern basis. Such configuration could enable/disable the feature, set the event history storage location, set parameters for sub-features such as compression, or set the event history data retention policy.

Event History saves events published to discrete subscriptions, in the chronological order received by the broker. Let us examine an example.

Subscriptions:

1. Subscription to exact match 'com.mycompany.log.auth' topic
2. Subscription to exact match 'com.mycompany.log.basket' topic

3. Subscription to prefix-based 'com.mycompany.log' topic

Publication messages:

1. Publication to topic 'com.mycompany.log.auth'. Forwarded as events to subscriptions 1 and 3.
2. Publication to topic 'com.mycompany.log.basket'. Delivered as event to subscriptions 2 and 3.
3. Publication to topic 'com.mycompany.log.basket'. Delivered as events to subscriptions 2 and 3.
4. Publication to topic 'com.mycompany.log.basket'. Delivered as events subscriptions 2 and 3.
5. Publication to topic 'com.mycompany.log.checkout'. Delivered as an event to subscription 3 only.

Event History:

- Event history for subscription 1 contains publication 1 only.
- Event history for subscription 2 contains publications 2, 3, and 4.
- Event history for subscription 3 contains all publications.

Feature Announcement

A *Broker* that implements *event history* must indicate `HELLO.roles.broker.features.event_history = true`, must announce the role `HELLO.roles.callee`, and must provide the meta procedures described below.

Receiving Event History

A *Caller* can request message history by calling the *Broker* meta procedure

```
    wamp.subscription.get_events
```

With payload:

- `Arguments = [subscription|id]`. The subscription id for which to retrieve event history
- `ArgumentsKw`:
  - `reverse`. Boolean. Optional. Traverses events in reverse order of occurrence. The default is to traverse events in order of occurrence.
  - `limit`. Positive integer. Optional. Indicates the maximum number of events to retrieve. Can be used for pagination.
  - `from_time`. RFC3339-formatted timestamp string. Optional. Only include publications occurring at the given timestamp or after (using >= comparison).
  - `after_time`. RFC3339-formatted timestamp string. Optional. Only include publications occurring after the given timestamp (using > comparison).
  - `before_time`. RFC3339-formatted timestamp string. Optional. Only include publications occurring before the given timestamp (using < comparison).

- `until_time`. RFC3339-formatted timestamp string. Optional. Only include publications occurring before the given timestamp including date itself (using `<=` comparison).
- `topic`. WAMP URI. Optional. For pattern-based subscriptions, only include publications to the specified topic.
- `from_publication`. Positive integer. Optional. Events in the results must have occurred at or following the event with the given `publication|id` (includes the event with the given `publication|id` in the results).
- `after_publication`. Positive integer. Optional. Events in the results must have occurred following the event with the given `publication_id` (excludes the event with the given `publication|id` in the results). Useful for pagination: pass the `publication|id` attribute of the last event returned in the previous page of results when navigating in order of occurrence (`reverse` argument absent or false).
- `before_publication`. Positive integer. Optional. Events in the results must have occurred previously to the event with the given `publication|id` (excludes the event with the given `publication|id` in the results). Useful for pagination: pass the `publication|id` attribute of the last event returned in the previous page of results when navigating in reverse order of occurrence (`reverse=true`).
- `until_publication`. Positive integer. Optional. Events in the results must have occurred at or previously to the event with the given `publication|id` (includes the event with the given `publication|id` in the results).

It is possible to pass multiple options at the same time. In this case they will be treated as conditions with logical `AND`. Note that the `publication|id` event attribute is not ordered as it belongs to the Global scope. But since events are stored in the order they are received by the broker, it is possible to find an event with the specified `publication|id` and then return events including or excluding the matched one depending on the `*_publication` filter attributes.

The `arguments` payload field returned by the above RPC uses the same schema: an array of `Event` objects containing an additional `timestamp` string attribute in RFC3339 format. It can also be an empty array in the case where there were no publications to the specified subscription, or all events were filtered out by the specified criteria. Additional general information about the query may be returned via the `argumentsKw` payload field.

```
  [
    {
        "timestamp": "yyyy-MM-ddThh:mm:ss.SSSZ", // string with event date/
time in RFC3339 format
        "subscription": 2342423, // The subscription ID of the event
        "publication": 32445235, // The original publication ID of the event
        "details": {},           // The original details of the event
        "args": [],              // The original list arguments payload of
the event. May be ommited
        "kwargs": {}             // The original key-value arguments payload
of the event. May be ommited
    }
  ]
```

Clients should not rely on timestamps being unique and monotonic. When events occur in quick succession, it's possible for some of them to have the same timestamp. When a router in an IoT system is deployed off-grid and is not synchronized to an NTP server, it's possible for the timestamps to jump backwards when the router's wall clock time or time zone is manually adjusted.

In cases where the events list is too large to send as a single RPC result, router implementations may provide additional options, such as pagination or returning progressive results.

As the Event History feature operates on `subscription|id`, there can be situations when there are not yet any subscribers to a topic of interest, but publications to the topic occur. In this situation, the *Broker* cannot predict that events under that topic should be stored. If the *Broker* implementation allows configuration on a per-topic basis, it may overcome this situations by preinitializing history-enabled topics with "dummy" subscriptions even if there are not yet any real subscribers to those topics.

Sometimes, a client may not be willing to subscribe to a topic just for the purpose of obtaining a subscription id. In that case a client may use other Subscriptions Meta API RPC for retrieving subscription IDs by topic URIs if the router supports it.

Security Aspects

TODO/FIXME: This part of Event History needs more discussion and clarification. But at least provides some basic information to take into account.

In order to request event history, a peer must be allowed to subscribe to a desired subscription first. Thus, if a peer cannot subscribe to a topic resulting in a subscription, it means that it cannot receive events history for that topic either. To sidestep this problem, a peer must be allowed to call related meta procedures for obtaining the event history as described above. Prohibited Event History meta procedure calls must fail with the `wamp.error.not_authorized` error URI.

Original publications may include additional options, such as `black-white-listing` that triggers special event processing. These same rules must also apply to event history requests. For example, if the original publication contains `eligible_authrole = 'admin'`, but the request for history came from a peer with `authrole = 'user'`, then even if `user` is authorized to subscribe to the topic (and thus is authorized to ask for event history), this publication must be filtered out from the results of this specific request, by the router side.

The `black-white-listing` feature also allows the filtering of event delivery on a `session ID` basis. In the context of event history, this can result in unexpected behaviour: session ids are generated randomly at runtime for every session establishment, so newly connected sessions asking for event history may receive events that were originally excluded, or, vice versa, may not receive expected events due to session ID mismatch. To prevent this unexpected behaviour, all events published with `Options.exclude|list[int]` or `Options.eligible|list[int]` should be ignored by the Event History mechanism and not be saved at all.

Finally, Event History should only filter according to attributes that do not change during the run time of the router, which are currently `authrole` and `authid`. Filtering based on ephemeral attributes like `session ID` – (U+2013) and perhaps other future custom attributes – (U+2013) should result in the event not being stored in the history at all, to avoid unintentional leaking of event information.

12.8. Event Retention

Event Retention is where a particular topic has an event associated with it which is delivered upon an opting-in client subscribing to the topic.

It can be used for topics that generally have single or very few Publishers notifying Subscribers of a single updating piece of data -- for example, a topic where a sensor publishes changes of temperature & humidity in a data center. It may do this every time the data changes (making the time between updates potentially very far apart), which causes an issue for new Subscribers who may need the last-known value upon connection, rather than waiting an unknown period of time until it is updated. Event Retention covers this use case by allowing the Publisher to mark a event as 'retained', bound to the topic it was sent to, which can be delivered upon a new client subscription that asks for it. It is similar to Event History, but allows the publisher to decide what the most important recent event is on the topic, even if other events are being delivered.

A *Broker* that advertises support MAY provide *event retention* on topics it provides. This event retention SHOULD be provided on a best-effort basis, and MUST NOT be interpreted as permanent or reliable storage by clients. This event retention is limited to one event that all subscribers would receive, and MAY include other supplemental events that have limited distribution (for example, a event published with subscriber black/whitelisting).

A *Publisher* can request storage of a new Retained Event by setting `Publish.Options.retain|bool` to `true`. Lack of the key in `Publish.Options` MUST be interpreted as a `false` value. A Broker MAY decline to provide event retention on certain topics by ignoring the `Publish.Options.retain` value. Brokers that allow event retention on the given topic MUST set the topic Retained Event to this if it were eligible to be published on the topic.

Subscribers may request access to the Retained Event by setting `Subscribe.Options.get_retained|bool` to `true`. Lack of the key in `Subscribe.Options` MUST be interpreted as a `false` value. When they opt-in to receiving the Retained Event, the Broker MUST send the Subscriber the **most recent** Retained Event that they would have received if they were subscribing when it was published. The Broker MUST NOT send the Subscriber a Retained Event that they would not be eligible to receive if they were subscribing when it was published. The *Retained Event*, as sent to the subscribing client, MUST have `Event.Details.retained|bool` set to `true`, to inform subscribers that it is not an immediately new message.

Feature Announcement

Support for this feature MUST be announced by *Brokers* (`role := "broker"`) via

```
    Welcome.Details.roles.broker.features.event_retention|bool := true
```

12.9. Subscription Revocation

Feature status: **alpha**

This feature allows a *Broker* to actively revoke a previously granted subscription. To achieve this, the existing UNSUBSCRIBED message is extended as described below.

Feature Announcement

Support for this feature MUST be announced by *Subscribers* (role := "subscriber") and *Brokers* (role := "broker") via

```
HELLO.Details.roles.<role>.features.
     subscription_revocation|bool := true
```

If the *Subscriber* does not support subscription_revocation, the *Broker* MAY still revoke a subscription to support administrative functionality. In this case, the *Broker* MUST NOT send an **UNSUBSCRIBED** message to the *Subscriber*. The *Subscriber* MAY use the subscription meta event `wamp.subscription.on_unsubscribe` to determine whether a session is removed from a subscription.

Extending UNSUBSCRIBED

When revoking a subscription, the router has no request ID to reply to. So it's set to zero and another argument is appended to indicate which subscription to revoke. Optionally, a reason why the subscription was revoked is also appended.

```
[UNSUBSCRIBED, 0, Details|dict]
```

where

- `Details.subscription|bool` MUST be a previously issued subscription ID.
- `Details.reason|string` MAY provide a reason as to why the subscription was revoked.

Example

```
[35, 0, {"subscription": 1293722, "reason": "no longer authorized"}]
```

12.10. Session Testament

When a WAMP client disconnects, or the WAMP session is destroyed, it may want to notify other subscribers or publish some fixed data. Since a client may disconnect uncleanly, this can't be done reliably by them. A *Testament*, however, set on the server, can be reliably sent by the *Broker* once either the WAMP session has detached or the client connection has been lost, and allows this functionality. It can be triggered when a Session is either detached (the client has disconnected from it, or frozen it, in the case of Session Resumption) or destroyed (when the WAMP session no longer exists on the server).

This allows clients that otherwise would not be able to know when other clients disconnect get a notification (for example, by using the WAMP Session Meta API) with a format the disconnected client chose.

Feature Announcement

Support for this feature MUST be announced by *Dealers* (`role := "dealer"`) via

```
    HELLO.Details.roles.dealer.features.
        testament_meta_api|bool := true
```

Testament Meta Procedures

A *Client* can call the following procedures to set/flush Testaments:

- `wamp.session.add_testament` to add a Testament which will be published on a particular topic when the Session is detached or destroyed.
- `wamp.session.flush_testaments` to remove the Testaments for that Session, either for when it is detached or destroyed.

wamp.session.add_testament

Adds a new testament:

Positional arguments

1. `topic|uri` - the topic to publish the event on
2. `args|list` - positional arguments for the event
3. `kwargs|dict` - keyword arguments for the event

Keyword arguments

1. `publish_options|dict` - options for the event when it is published -- see `Publish.Options`. Not all options may be honoured (for example, `acknowledge`). By default, there are no options.
2. `scope|string` - When the testament should be published. Valid values are `detached` (when the WAMP session is detached, for example, when using Event Retention) or `destroyed` (when the WAMP session is finalized and destroyed on the Broker). Default MUST be `destroyed`.

`wamp.session.add_testament` does not return a value.

wamp.session.flush_testaments

Removes testaments for the given scope:

Keyword arguments

1. `scope|string` - Which set of testaments to be removed. Valid values are the same as `wamp.session.add_testament`, and the default MUST be `destroyed`.

`wamp.session.flush_testaments` does not return a value.

Testaments in Use

A *Client* that wishes to send some form of data when their *Session* ends unexpectedly or their *Transport* becomes lost can set a testament using the WAMP Testament Meta API, when a *Router* supports it. For example, a client may call `add_testament` (this example uses the implicit `scope` option of `destroyed`):

```
yield self.call('wamp.session.add_testament',
                'com.myapp.mytopic', ['Seeya!'], {'my_name': 'app1'})
```

The *Router* will then store this information on the WAMP Session, either in a `detached` or `destroyed` bucket, in the order they were added. A client MUST be able to set multiple testaments per-scope. If the *Router* does not support Session Resumption (therefore removing the distinction between a detached and destroyed session), it MUST still use these two separate buckets to allow `wamp.session.flush_testaments` to work.

When a *Session* is *detached*, the *Router* will inspect it for any Testaments in the `detached` scope, and publish them in the order that the Router received them, on the specified topic, with the specified arguments, keyword arguments, and publish options. The *Router* MAY ignore publish options that do not make sense for a Testament (for example, acknowledged publishes).

When a *Session* is going to be *destroyed*, the *Router* will inspect it for any Testaments in the `destroyed` scope, and publish them in the same way as it would for the `detached` scope, in the order that they were received.

A *Router* that does not allow Session Resumption MUST send `detached`-scope Testaments before `destroyed`-scope Testaments.

A *Client* can also clear testaments if the information is no longer relevant (for example, it is shutting down completely cleanly). For example, a client may call `wamp.session.flush_testaments`:

```
yield self.call('wamp.session.flush_testaments', scope='detached')
yield self.call('wamp.session.flush_testaments', scope='destroyed')
```

The *Router* will then flush all Testaments stored for the given scope.

13. Authentication Methods

Authentication is a complex area. Some applications might want to leverage authentication information coming from the transport underlying WAMP, e.g. HTTP cookies or TLS certificates.

Some transports might imply trust or implicit authentication by their very nature, e.g. Unix domain sockets with appropriate file system permissions in place.

Other application might want to perform their own authentication using external mechanisms (completely outside and independent of WAMP).

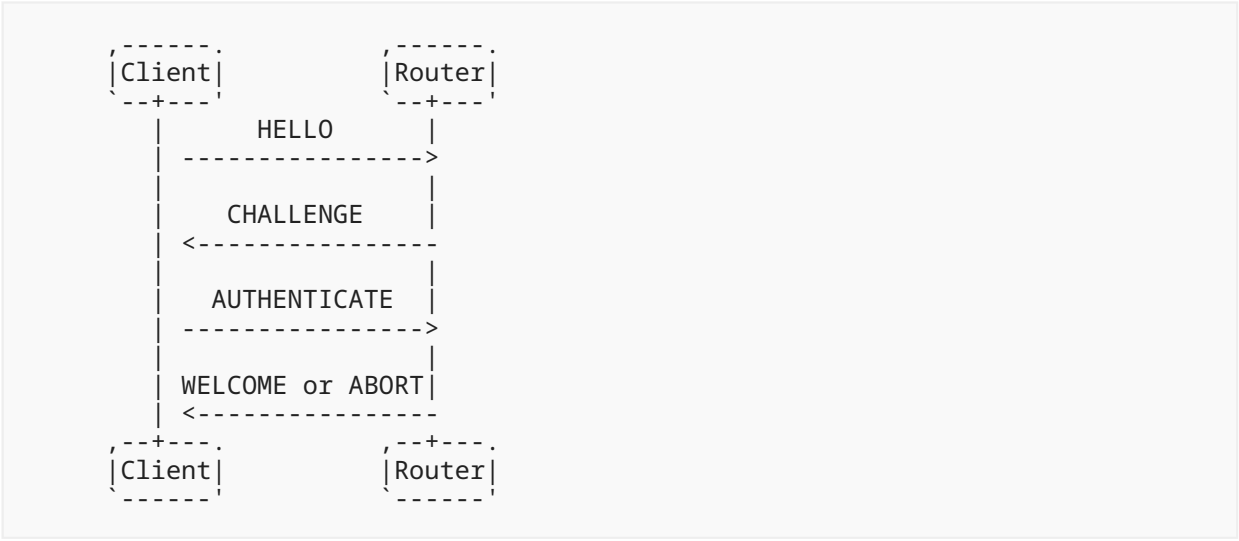
Some applications might want to perform their own authentication schemes by using basic WAMP mechanisms, e.g. by using application-defined remote procedure calls.

And some applications might want to use a transport independent scheme, nevertheless predefined by WAMP.

WAMP-level Authentication

The message flow between Clients and Routers for establishing and tearing down sessions MAY involve the following messages which authenticate a session:

1. `CHALLENGE`
2. `AUTHENTICATE`



Concrete use of `CHALLENGE` and `AUTHENTICATE` messages depends on the specific authentication method.

See WAMP Challenge-Response Authentication or ticket authentication for the use in these authentication methods.

If two-factor authentication is desired, then two subsequent rounds of `CHALLENGE` and `RESPONSE` may be employed.

CHALLENGE

An authentication MAY be required for the establishment of a session. Such requirement MAY be based on the `Realm` the connection is requested for.

To request authentication, the Router MUST send a `CHALLENGE` message to the *Endpoint*.

```
[CHALLENGE, AuthMethod|string, Extra|dict]
```

AUTHENTICATE

In response to a `CHALLENGE` message, the Client MUST send an `AUTHENTICATE` message.

```
[AUTHENTICATE, Signature|string, Extra|dict]
```

If the authentication succeeds, the Router MUST send a `WELCOME` message, else it MUST send an `ABORT` message.

Transport-level Authentication

Cookie-based Authentication

When running WAMP over WebSocket, the transport provides HTTP client cookies during the WebSocket opening handshake. The cookies can be used to authenticate one peer (the client) against the other (the server). The other authentication direction cannot be supported by cookies.

This transport-level authentication information may be forwarded to the WAMP level within `HELLO.Details.transport.auth|any` in the client-to-server direction.

TLS Certificate Authentication

When running WAMP over a TLS (either secure WebSocket or raw TCP) transport, a peer may authenticate to the other via the TLS certificate mechanism. A server might authenticate to the client, and a client may authenticate to the server (TLS client-certificate based authentication).

This transport-level authentication information may be forwarded to the WAMP level within `HELLO.Details.transport.auth|any` in both directions (if available).

13.1. Ticket-based Authentication

With *Ticket-based authentication*, the client needs to present the server an authentication "ticket" - some magic value to authenticate itself to the server.

This "ticket" could be a long-lived, pre-agreed secret (e.g. a user password) or a short-lived authentication token (like a Kerberos token). WAMP does not care or interpret the ticket presented by the client.

> Caution: This scheme is extremely simple and flexible, but the resulting security may be limited. E.g., the ticket value will be sent over the wire. If the transport WAMP is running over is not encrypted, a man-in-the-middle can sniff and possibly hijack the ticket. If the ticket value is reused, that might enable replay attacks.

A typical authentication begins with the client sending a `HELLO` message specifying the `ticket` method as (one of) the authentication methods:

```
[1, "realm1",
  {
    "roles": ...,
    "authmethods": ["ticket"],
    "authid": "joe"
  }
]
```

The `HELLO.Details.authmethods|list` is used by the client to announce the authentication methods it is prepared to perform. For Ticket-based, this MUST include `"ticket"`.

The `HELLO.Details.authid|string` is the authentication ID (e.g. username) the client wishes to authenticate as. For Ticket-based authentication, this MUST be provided.

If the server is unwilling or unable to perform Ticket-based authentication, it'll either skip forward trying other authentication methods (if the client announced any) or send an `ABORT` message.

If the server is willing to let the client authenticate using a ticket and the server recognizes the provided `authid`, it'll send a `CHALLENGE` message:

```
[4, "ticket", {}]
```

The client will send an `AUTHENTICATE` message containing a ticket:

```
[5, "secret!!!", {}]
```

The server will then check if the ticket provided is permissible (for the `authid` given).

If the authentication succeeds, the server will finally respond with a `WELCOME` message:

```
[2, 3251278072152162,
  {
    "authid": "joe",
    "authrole": "user",
    "authmethod": "ticket",
    "authprovider": "static",
    "roles": ...
  }
]
```

where

1. `authid|string`: The authentication ID the client was (actually) authenticated as.
2. `authrole|string`: The authentication role the client was authenticated for.
3. `authmethod|string`: The authentication method, here `"ticket"`

4. `authprovider|string`: The actual provider of authentication. For Ticket-based authentication, this can be freely chosen by the app, e.g. `static` or `dynamic`.

The `WELCOME.Details` again contain the actual authentication information active. If the authentication fails, the server will response with an `ABORT` message.

13.2. Challenge Response Authentication

WAMP Challenge-Response ("WAMP-CRA") authentication is a simple, secure authentication mechanism using a shared secret. The client and the server share a secret. The secret never travels the wire, hence WAMP-CRA can be used via non-TLS connections. The actual pre-sharing of the secret is outside the scope of the authentication mechanism.

A typical authentication begins with the client sending a `HELLO` message specifying the `wampcra` method as (one of) the authentication methods:

```
    [1, "realm1",
        {
            "roles": ...,
            "authmethods": ["wampcra"],
            "authid": "peter"
        }
    ]
```

The `HELLO.Details.authmethods|list` is used by the client to announce the authentication methods it is prepared to perform. For WAMP-CRA, this MUST include `"wampcra"`.

The `HELLO.Details.authid|string` is the authentication ID (e.g. username) the client wishes to authenticate as. For WAMP-CRA, this MUST be provided.

If the server is unwilling or unable to perform WAMP-CRA authentication, it MAY either skip forward trying other authentication methods (if the client announced any) or send an `ABORT` message.

If the server is willing to let the client authenticate using WAMP-CRA, and the server recognizes the provided `authid`, it MUST send a `CHALLENGE` message:

```
    [4, "wampcra",
        {
            "challenge": "{ \"nonce\": \"LHRTC9zeOIrt_9U3\",
                \"authprovider\": \"userdb\", \"authid\": \"peter\",
                \"timestamp\": \"2014-06-22T16:36:25.448Z\",
                \"authrole\": \"user\", \"authmethod\": \"wampcra\",
                \"session\": 3251278072152162}"
        }
    ]
```

The `CHALLENGE.Details.challenge|string` is a string the client needs to create a signature for. The string MUST BE a JSON serialized object which MUST contain:

1. `authid|string`: The authentication ID the client will be authenticated as when the authentication succeeds.
2. `authrole|string`: The authentication role the client will be authenticated as when the authentication succeeds.
3. `authmethod|string`: The authentication methods, here `"wampcra"`
4. `authprovider|string`: The actual provider of authentication. For WAMP-CRA, this can be freely chosen by the app, e.g. `userdb`.
5. `nonce|string`: A random value.
6. `timestamp|string`: The UTC timestamp (ISO8601 format) the authentication was started, e.g. `2014-06-22T16:51:41.643Z`.
7. `session|int`: The WAMP session ID that will be assigned to the session once it is authenticated successfully.

The client needs to compute the signature as follows:

```
    signature := HMAC[SHA256]_{secret} (challenge)
```

That is, compute the HMAC-SHA256 using the shared `secret` over the `challenge`.

After computing the signature, the client will send an `AUTHENTICATE` message containing the signature, as a base64-encoded string:

```
    [5, "gir1mSx+deCDUV7wRM5SGIn/+R/ClqLZuH4m7FJeBVI=", {}]
```

The server will then check if

- the signature matches the one expected
- the `AUTHENTICATE` message was sent in due time

If the authentication succeeds, the server will finally respond with a `WELCOME` message:

```
    [2, 3251278072152162,
        {
            "authid": "peter",
            "authrole": "user",
            "authmethod": "wampcra",
            "authprovider": "userdb",
            "roles": ...
        }
    ]
```

The `WELCOME.Details` again contain the actual authentication information active.

If the authentication fails, the server will response with an `ABORT` message.

Server-side Verification

The challenge sent during WAMP-CRA contains

1. random information (the `nonce`) to make WAMP-CRA robust against replay attacks
2. timestamp information (the `timestamp`) to allow WAMP-CRA timeout on authentication requests that took too long
3. session information (the `session`) to bind the authentication to a WAMP session ID
4. all the authentication information that relates to authorization like `authid` and `authrole`

Three-legged Authentication

The signing of the challenge sent by the server usually is done directly on the client. However, this is no strict requirement.

E.g. a client might forward the challenge to another party (hence the "three-legged") for creating the signature. This can be used when the client was previously already authenticated to that third party, and WAMP-CRA should run piggy packed on that authentication.

The third party would, upon receiving a signing request, simply check if the client is already authenticated, and if so, create a signature for WAMP-CRA.

In this case, the secret is actually shared between the WAMP server who wants to authenticate clients using WAMP-CRA and the third party server, who shares a secret with the WAMP server.

This scenario is also the reason the challenge sent with WAMP-CRA is not simply a random value, but a JSON serialized object containing sufficient authentication information for the third party to check.

Password Salting

WAMP-CRA operates using a shared secret. While the secret is never sent over the wire, a shared secret often requires storage of that secret on the client and the server - and storing a password verbatim (unencrypted) is not recommended in general.

WAMP-CRA allows the use of salted passwords following the PBKDF2 key derivation scheme. With salted passwords, the password itself is never stored, but only a key derived from the password and a password salt. This derived key is then practically working as the new shared secret.

When the password is salted, the server will during WAMP-CRA send a `CHALLENGE` message containing additional information:

```
[4, "wampcra",
    {
        "challenge": "{ \"nonce\": \"LHRTC9zeOIrt_9U3\",
            \"authprovider\": \"userdb\", \"authid\": \"peter\",
            \"timestamp\": \"2014-06-22T16:36:25.448Z\",
            \"authrole\": \"user\", \"authmethod\": \"wampcra\",
            \"session\": 3251278072152162}",
        "salt": "salt123",
        "keylen": 32,
        "iterations": 1000
    }
]
```

The `CHALLENGE.Details.salt|string` is the password salt in use. The `CHALLENGE.Details.keylen|int` and `CHALLENGE.Details.iterations|int` are parameters for the PBKDF2 algorithm.

13.3. Salted Challenge Response Authentication

The WAMP Salted Challenge Response Authentication Mechanism ("WAMP-SCRAM"), is a password-based authentication method where the shared secret is neither transmitted nor stored as cleartext. WAMP-SCRAM is based on RFC5802 (*Salted Challenge Response Authentication Mechanism*) and RFC7677 (*SCRAM-SHA-256 and SCRAM-SHA-256-PLUS*).

WAMP-SCRAM supports the Argon2 (draft-irtf-cfrg-argon2) password-based key derivation function, a memory-hard algorithm intended to resist cracking on GPU hardware. PBKDF2 (RFC2898) is also supported for applications that are required to use primitives currently approved by cryptographic standards.

Security Considerations

With WAMP-SCRAM, if the authentication database is stolen, an attacker cannot impersonate a user unless they guess the password offline by brute force.

In the event that the server's authentication database is stolen, and the attacker either eavesdrops on one authentication exchange or impersonates a server, the attacker gains the ability to impersonate that particular user on that server. If the same salt is used on other servers, the attacker would gain the ability to impersonate that user on all servers using the same salt. That's why it's important to use a per-user random salt.

An eavesdropper that captures a user authentication exchange has enough information to mount an offline, brute-force dictionary attack for that particular user. If passwords are sufficiently strong, the cost/time needed to crack a password becomes prohibitive.

Note that when HTML/JavaScript assets are served to a web browser, WAMP-SCRAM does not safeguard against a man-in-the-middle tampering with those assets. Those assets could be tampered with in a way that captures the user's password and sends it to the attacker.

In light of the above security concerns, a secure TLS transport is therefore advised to prevent such attacks. The channel binding feature of SCRAM can be used to ensure that the TLS endpoints are the same between client and router.

Deviations from RFC5802

1. To simplify parsing, SCRAM attributes in the authentication exchange messages are encoded as members of the `Options`/`Details` objects without escaping the `,` and `=` characters. However, the `AuthMessage` used to compute the client and server signatures DOES use the exact syntax specified in RFC5802, section 7. This makes it possible to use existing test vectors to verify WAMP-SCRAM implementations.
2. Hashing based on the weaker SHA-1 specified in RFC5802 is intentionally not supported by WAMP-SCRAM, in favor of the stronger SHA-256 specified in RFC7677.
3. The Argon2 key derivation function MAY be used instead of PBKDF2.
4. Nonces are required to be base64-encoded, which is stricter than the `printable` syntax specification of RFC5802.
5. The `"y"` channel binding flag is not used as there is currently no standard way for WAMP routers to announce channel binding capabilities.
6. The use of `authzid` for user impersonation is not supported.

authmethod Type String

`"wamp-scram"` SHALL be used as the `authmethod` type string for WAMP-SCRAM authentication. Announcement by routers of WAMP-SCRAM support is outside the scope of this document.

Base64 encoding

Base64 encoding of octet strings is restricted to canonical form with no whitespace, as defined in RFC4648 (*The Base16, Base32, and Base64 Data Encodings*).

Nonces

In WAMP-SCRAM, a *nonce* (number used once) is a base64-encoded sequence of random octets. It SHOULD be of sufficient length to make a replay attack unfeasible. A length of 16 octets (128 bits) is recommended for each of the client and server-generated nonces.

See RFC4086 (*Randomness Requirements for Security*) for best practices concerning randomness.

Salts

A *salt* is a base64-encoded sequence of random octets.

To prevent rainbow table attacks in the event of database theft, the salt MUST be generated randomly by the server **for each user**. The random salt is stored with each user record in the authentication database.

Username/Password String Normalization

Username and password strings SHALL be normalized according to the *SASLprep* profile described in RFC4013, using the *stringprep* algorithm described in RFC3454.

While SASLprep preserves the case of usernames, the server MAY choose to perform case insensitive comparisons when searching for a username in the authentication database.

Channel Binding

Channel binding is a feature that allows a higher layer to establish that the other end of an underlying secure channel is bound to its higher layer counterpart. See RFC5056 (*On the Use of Channel Bindings*) for an in-depth discussion.

RFC5929 defines binding types for use with TLS transports, of which `tls-unique` and `tls-server-end-point` are applicable for WAMP-SCRAM. For each channel binding type, there is a corresponding definition of the *channel binding data* that must be sent in response to the authentication challenge.

Negotiation and announcement of channel binding is outside the scope of this document. RFC5929 section 6 recommends that application protocols use `tls-unique` exclusively, except perhaps where server-side proxies are commonly deployed.

Note that WAMP-SCRAM channel binding is not generally possible with web browser clients due to the lack of a suitable API for this purpose.

The tls-unique Channel Binding Type

The `tls-unique` channel binding type allows the WAMP layer to establish that the other peer is authenticating over the same, unique TLS connection. The channel binding data for this type corresponds to the bytes of the first TLS Finished message, as described in RFC5929, section 3. RFC5929 section 10.2 addresses the concern of disclosing this data over the TLS channel (in short, the TLS Finished message would already be visible to eavesdroppers).

To safeguard against the *triple handshake attack* described in RFC7627, this channel binding type MUST be used over a TLS channel that uses the *extended master secret* extension, or over a TLS channel where session resumption is not permitted.

The tls-server-end-point Channel Binding Type

The `tls-server-end-point` channel binding type allows the WAMP layer to establish that the other peer is authenticating over a TLS connection to a server having been issued a Public Key Infrastructure Certificate. The channel binding data for this type is a hash of the TLS server's certificate, computed as described in RFC5929, section 4.1. The certificate is hashed to accommodate memory-constrained implementations.

Authentication Exchange

WAMP-SCRAM uses a single round of challenge/response pairs after the client authentication request and before the authentication outcome.

The mapping of RFC5802 messages to WAMP messages is as follows:

SCRAM Message	WAMP Message
client-first-message	HELLO
server-first-message	CHALLENGE
client-final-message	AUTHENTICATE
server-final-message with verifier	WELCOME
server-final-message with server-error	ABORT

Table 9

Initial Client Authentication Message

WAMP-SCRAM authentication begins with the client sending a `HELLO` message specifying the `wamp-scram` method as (one of) the authentication methods:

```
    [1, "realm1",
        {
            "roles": ...,
            "authmethods": ["wamp-scram"],
            "authid": "user",
            "authextra":
                {
                    "nonce": "egVDf3DMJh0=",
                    "channel_binding": null
                }

        }
    ]
```

where:

1. `authid|string`: The identity of the user performing authentication. This corresponds to the `username` parameter in RFC5802.
2. `authextra.nonce|string`: A base64-encoded sequence of random octets, generated by the client. See Nonces.
3. `authextra.channel_binding|string`: Optional string containing the desired channel binding type. See Channel Bindings.

Upon receiving the `HELLO` message, the server MUST terminate the authentication process by sending an `ABORT` message under any of the following circumstances:

- The server does not support the WAMP-SCRAM `authmethods`, and there are no other methods left that the server supports for this `authid`.
- The the server does not support the requested `channel_binding` type.
- (Optional) The server does not recognize the given `authid`. In this case, the server MAY proceed with a mock `CHALLENGE` message to avoid leaking information on the existence of usernames. This mock `CHALLENGE` SHOULD contain a generated `salt` value that is

always the same for a given `authid`, otherwise an attacker may discover that the user doesn't actually exist.

Initial Server Authentication Message

If none of the above failure conditions apply, and the server is ready to let the client authenticate using WAMP-SCRAM, then it SHALL send a `CHALLENGE` message:

```
[4, "wamp-scram",
    {
        "nonce": "egVDf3DMJh0=SBmkFIh7sSo=",
        "salt": "aBc+fx0NAVA=",
        "kdf": "pbkdf2",
        "iterations": 4096,
        "memory": null
    }
]
```

where:

1. `nonce|string`: A server-generated nonce that is appended to the client-generated nonce sent in the previous `HELLO` message. See Nonces.
2. `salt|string`: The base64-encoded salt for this user, to be passed to the key derivation function. This value is stored with each user record in the authentication database. See Salts.
3. `kdf`: The key derivation function (KDF) used to hash the password. This value is stored with each user record in the authentication database. See Key Derivation Functions.
4. `iterations|integer`: The execution time cost factor to use for generating the `SaltedPassword` hash. This value is stored with each user record in the authentication database.
5. `memory|integer`: The memory cost factor to use for generating the `SaltedPassword` hash. This is only used by the Argon2 key derivation function, where it is stored with each user record in the authentication database.

The client MUST respond with an ABORT message if `CHALLENGE.Details.nonce` does not begin with the client nonce sent in `HELLO.Details.nonce`.

The client SHOULD respond with an ABORT message if it detects that the cost parameters are unusually low. Such low-cost parameters could be the result of a rogue server attempting to obtain a weak password hash that can be easily cracked. What constitutes unusually low parameters is implementation-specific and is not covered by this document.

Final Client Authentication Message

Upon receiving a valid `CHALLENGE` message, the client SHALL respond with an `AUTHENTICATE` message:

```
[5, "dHzbZapWIk4jUhN+Ute9ytag9zjfMHgsqmmiz7AndVQ=",
    {
        "nonce": "egVDf3DMJh0=SBmkFIh7sSo=",
        "channel_binding": null,
        "cbind_data": null
    }
]
```

where:

1. `Signature|string` argument: The base64-encoded `ClientProof`, computed as described in the SCRAM-Algorithms section.
2. `nonce|string`: The concatenated client-server nonce from the previous `CHALLENGE` message.
3. `channel_binding|string`: Optional string containing the channel binding type that was sent in the original `HELLO` message.
4. `cbind_data|string`: Optional base64-encoded channel binding data. MUST be present if and only if `channel_binding` is not `null`. The format of the binding data is dependent on the binding type. See Channel Binding.

Upon receiving the `AUTHENTICATE` message, the server SHALL then check that:

- The `AUTHENTICATE` message was received in due time.
- The `ClientProof` passed via the `Signature|string` argument is validated against the `StoredKey` and `ServerKey` stored in the authentication database. See SCRAM Algorithms.
- `nonce` matches the one previously sent via `CHALLENGE`.
- The `channel_binding` matches the one sent in the `HELLO` message.
- The `cbind_data` sent by the client matches the channel binding data that the server sees on its side of the channel.

Final Server Authentication Message - Success

If the authentication succeeds, the server SHALL finally respond with a `WELCOME` message:

```
[2, 3251278072152162,
    {
        "authid": "user",
        "authrole": "frontend",
        "authmethod": "wamp-scram",
        "authprovider": "static",
        "roles": ...,
        "authextra":
            {
                "verifier":
                  "v=6rriTRBi23WpRR/wtup+mMhUZUn/dB5nLTJRsjl95G4="
            }
    }
]
```

where:

1. `authid|string`: The authentication ID the client was actually authenticated as.
2. `authrole|string`: The authentication role the client was authenticated for.
3. `authmethod|string`: The authentication method, here `"wamp-scram"`.
4. `authprovider|string`: The actual provider of authentication. For WAMP-SCRAM authentication, this can be freely chosen by the app, e.g. static or dynamic.
5. `authextra.verifier|string`: The base64-encoded `ServerSignature`, computed as described in the SCRAM Algorithms section.

The client SHOULD check the `verifier` for mutual authentication, terminating the session if invalid.

Final Server Authentication Message - Failure

If the authentication fails, the server SHALL respond with an `ABORT` message.

The server MAY include a SCRAM-specific error string in the `ABORT` message as a `Details.scram` attribute. SCRAM error strings are listed in RFC5802, section 7, under `server-error-value`.

SCRAM Algorithms

This section is non-normative.

RFC5802 specifies the algorithms used to compute the `ClientProof`, `ServerSignature`, `ServerKey`, and `StoredKey` values referenced by this document. Those algorithms are summarized here in pseudocode for reference.

Notation

- `"="`: The variable on the left-hand side is the result of the expression on the right-hand side.
- `"+"`: String concatenation.
- `IfNull(attribute, value, else)`: If the given `attribute` is absent or null, evaluates to the given `value`, otherwise evaluates to the given `else` value.
- `Decimal(integer)`: The decimal string representation of the given `integer`.
- `Base64(octets)`: Base64 encoding of the given octet sequence, restricted to canonical form with no whitespace, as defined in RFC4648.
- `UnBase64(str)`: Decode the given Base64 string into an octet sequence.
- `Normalize(str)`: Normalize the given string using the SASLprep profile RFC4013 of the "stringprep" algorithm RFC3454.
- `XOR`: The exclusive-or operation applied to each octet of the left and right-hand-side octet sequences.
- `SHA256(str)`: The SHA-256 cryptographic hash function.
- `HMAC(key, str)`: Keyed-hash message authentication code, as defined in RFC2104, with SHA-256 as the underlying hash function.

- `KDF(str, salt, params...)`: One of the supported key derivations function, with the output key length the same as the SHA-256 output length (32 octets). `params...` are the additional parameters that are applicable for the function: `iterations` and `memory`.
- `Escape(str)`: Replace every occurrence of "`,`" and "`=`" in the given string with "`=2C`" and "`=3D`" respectively.

Data Stored on the Server

For each user, the server needs to store:

1. A random, per-user salt.
2. The type string corresponding to the key derivation function (KDF) used to hash the password (e.g. `"argon2id13"`). This is needed to handle future revisions of the KDF, as well as allowing migration to stronger KDFs that may be added to WAMP-SCRAM in the future. This may also be needed if the KDF used during user registration is configurable or selectable on a per-user basis.
3. Parameters that are applicable to the key derivation function : `iterations` and possibly `memory`.
4. The `StoredKey`.
5. The `ServerKey`.

where `StoredKey` and `ServerKey` are computed as follows:

```
SaltedPassword  = KDF(Normalize(password), salt, params...)
ClientKey       = HMAC(SaltedPassword, "Client Key")
StoredKey       = SHA256(ClientKey)
ServerKey       = HMAC(SaltedPassword, "Server Key")
```

Note that `"Client Key"` and `"Server Key"` are string literals.

The manner in which the `StoredKey` and `ServerKey` are shared with the server during user registration is outside the scope of SCRAM and this document.

AuthMessage

In SCRAM, `AuthMessage` is used for computing `ClientProof` and `ServerSignature`. `AuthMessage` is computed using attributes that were sent in the first three messages of the authentication exchange.

```
ClientFirstBare = "n=" + Escape(HELLO.Details.authid) + "," +
                  "r=" + HELLO.Details.authextra.nonce

ServerFirst = "r=" + CHALLENGE.Details.nonce + "," +
              "s=" + CHALLENGE.Details.salt + "," +
              "i=" + Decimal(CHALLENGE.Details.iterations)

CBindName = AUTHENTICATE.Extra.channel_binding

CBindData = AUTHENTICATE.Extra.cbind_data

CBindFlag = IfNull(CBindName, "n", "p=" + CBindName)

CBindInput = CBindFlag + ",," +
             IfNull(CBindData, "", UnBase64(CBindData))

ClientFinalNoProof = "c=" + Base64(CBindInput) + "," +
                     "r=" + AUTHENTICATE.Extra.nonce

AuthMessage = ClientFirstBare + "," + ServerFirst + "," +
              ClientFinalNoProof
```

ClientProof

`ClientProof` is computed by the client during the authentication exchange as follows:

```
SaltedPassword  = KDF(Normalize(password), salt, params...)
ClientKey       = HMAC(SaltedPassword, "Client Key")
StoredKey       = SHA256(ClientKey)
ClientSignature = HMAC(StoredKey, AuthMessage)
ClientProof     = ClientKey XOR ClientSignature
```

The `ClientProof` is then sent to the server, base64-encoded, via the `AUTHENTICATE.Signature` argument.

The server verifies the `ClientProof` by computing the `RecoveredStoredKey` and comparing it to the actual `StoredKey`:

```
ClientSignature    = HMAC(StoredKey, AuthMessage)
RecoveredClientKey = ClientSignature XOR ReceivedClientProof
RecoveredStoredKey = SHA256(RecoveredClientKey)
```

Note that the client MAY cache the `ClientKey` and `StoredKey` (or just `SaltedPassword`) to avoid having to perform the expensive KDF computation for every authentication exchange. Storing these values securely on the client is outside the scope of this document.

ServerSignature

`ServerSignature` is computed by the server during the authentication exchange as follows:

```
ServerSignature = HMAC(ServerKey, AuthMessage)
```

The `ServerSignature` is then sent to the client, base64-encoded, via the `WELCOME.Details.authextra.verifier` attribute.

The client verifies the `ServerSignature` by computing it and comparing it with the `ServerSignature` sent by the server:

```
ServerKey       = HMAC(SaltedPassword, "Server Key")
ServerSignature = HMAC(ServerKey, AuthMessage)
```

Key Derivation Functions

SCRAM uses a password-based key derivation function (KDF) to hash user passwords. WAMP-SCRAM supports both Argon2 and PBKDF2 as the KDF. Argon2 is recommended because of its memory hardness and resistance against GPU hardware. PBKDF2, which does not feature memory hardness, is also supported for applications that are required to use primitives currently approved by cryptographic standards.

The following table maps the `CHALLENGE.Details.kdf` type string to the corresponding KDF.

KDF type string	Function
`"argon2id13"`	Argon2id variant of Argon2, version 1.3
`"pbkdf2"`	PBKDF2

Table 10

To promote interoperability, WAMP-SCRAM client/server implementations SHOULD support both of the above KDFs. During authentication, there is no "negotiation" of the KDF, and the client MUST use the same KDF than the one used to create the keys stored in the authentication database.

Which KDF is used to hash the password during user registration is up to the application and/or server implementation, and is not covered by this document. Possibilities include:

- making the KDF selectable at runtime during registration,
- making the KDF statically configurable on the server, or,
- hard-coding the KDF selection on the server.

Argon2

The Argon2 key derivation function, proposed in draft-irtf-cfrg-argon2, is computed using the following parameters:

- `CHALLENGE.Details.salt` as the cryptographic salt,
- `CHALLENGE.Details.iterations` as the number of iterations,
- `CHALLENGE.Details.memory` as the memory size (in kibibytes),
- 1 as the parallelism parameter,

- Argon2id as the algorithm variant, and,
- 32 octets as the output key length.

For WAMP-SCRAM, the parallelism parameter is fixed to 1 due to the password being hashed on the client side, where it is not generally known how many cores/threads are available on the client's device.

Section 4 of the Argon2 internet draft recommends the general procedure for selecting parameters, of which the following guidelines are applicable to WAMP-SCRAM:

- A 128-bit salt is recommended, which can be reduced to 64-bit if space is limited.
- The `memory` parameter is to be configured to the maximum amount of memory usage that can be tolerated on client devices for computing the hash.
- The `iterations` parameter is to be determined experimentally so that execution time on the client reaches the maximum that can be tolerated by users during authentication. If the execution time is intolerable with `iterations` = 1, then reduce the `memory` parameter as needed.

PBKDF2

The PBKDF2 key derivation function, defined in RFC2898, is used with SHA-256 as the pseudorandom function (PRF).

The PDBKDF2 hash is computed using the following parameters:

- `CHALLENGE.Details.salt` as the cryptographic salt,
- `CHALLENGE.Details.iterations` as the iteration count, and,
- 32 octets as the output key length (`dkLen`), which matches the SHA-256 output length.

RFC2898 section 4.1 recommends at least 64 bits for the salt.

The `iterations` parameter SHOULD be determined experimentally so that execution time on the client reaches the maximum that can be tolerated by users during authentication. RFC7677 section 4 recommends an iteration count of at least 4096, with a significantly higher value on non-mobile clients.

13.4. Cryptosign-based Authentication

WAMP-Cryptosign is a WAMP authentication method based on *public-private key cryptography*. Specifically, it is based on Ed25519 digital signatures as described in [RFC8032].

Ed25519 is an elliptic curve signature scheme that instantiates the Edwards-curve Digital Signature Algorithm (EdDSA) with elliptic curve parameters which are equivalent to Curve25519. **Curve25519** is a SafeCurve, which means it is easy to implement and avoid security issues resulting from common implementation challenges and bugs. Ed25519 is intended to operate at around the 128-bit security level, and there are robust native implementations available as open-source, e.g. libsodium, which can be used from script languages, e.g. PyNaCl.

An implementation of *WAMP-Cryptosign* MUST provide

- Client Authentication

and MAY implement one or more of

- TLS Channel Binding
- Router Authentication
- Trustroots and Certificates
- Remote Attestation

The following sections describe each of above features of *WAMP-Cryptosign*.

Examples of complete authentication message exchanges can be found at the end of this chapter in Example Message Exchanges.

In WAMP, the following **cryptographic primitives** are used with *WAMP-Cryptosign* authentication:

Elliptic Curves:

SECG	Usage in WAMP
secp256r1	Transport Encryption (WAMP transport encryption)
curve25519	Session Authentication (WAMP-Cryptosign authentication)
secp256k1	Data Signatures (WAMP-Cryptosign certificates, WAMP E2E encryption)

Table 11

- RFC4492: Elliptic Curve Cryptography (ECC) Cipher Suites for Transport Layer Security (TLS)
- RFC7748: Elliptic Curves for Security

Hash Functions:

SECG	Usage in WAMP
sha256	Session Authentication (WAMP-Cryptosign authentication)
keccak256	Data Signatures (WAMP-Cryptosign certificates, WAMP E2E encryption)

Table 12

Note that `sha256` refers to the SHA-2 algorithm, while `sha3-256` is a different algorithm refering to SHA-3.

Signature Schemes:

SECG	Usage in WAMP
ed25519	Session Authentication (WAMP-Cryptosign Authentication)
ecdsa	Data Signatures (Ethereum, WAMP-Cryptosign Certificates, WAMP-E2E)

Table 13

13.4.1. Client Authentication

A *Client* is authenticated to a *Router* by:

1. sending a `HELLO`, announcing its public key
2. signing a (random) challenge received in `CHALLENGE` with its private key
3. let the router verify the signature, proofing the client actually controls the private key, and thus the authenticity of the client as identified by the public key
4. let the router admit the client to a realm under a role, based on the public key

Thus, the client is identified using its public key, and the *Router* needs to know said public key and its desired realm and role in advance.

A *Client* for which the *Router* does not previously know the client's public key MAY use the Trustroots and Certificates feature to trust a *Client* based on an additional certificate presented by the client.

A *Router* is optionally (see Router Authentication) authenticated to a *Client* by:

1. client includes a (random) `HELLO.Details.challenge|string`
2. the router sends the signature as part of its challenge to the client, in `CHALLENGE.extra.signature|string`

Again, in this case, the *Router* includes a trustroot and certificate for the client to verify.

13.4.1.1. Computing the Signature

The challenge sent by the router is a 32 bytes random value, encoded as a Hex string in `CHALLENGE.extra.challenge|string`.

When no channel binding is active, the Ed25519 signature over the 32 bytes message MUST be computed using the WAMP-Cryptosign *private key* of the authenticating client.

When channel binding is active, the challenge MUST first be XOR'ed bytewise with the channel ID, e.g. the 32 bytes from TLS with channel binding `"tls-unique"`, and the resulting message (which again has length 32 bytes) MUST be signed using the WAMP-Cryptosign *private key* of the authenticating client.

The client MUST return the concatenation of the signature and the message signed (96 bytes) in the `AUTHENTICATE` message.

13.4.1.2. Example Message Flow

A typical authentication begins with the client sending a `HELLO` message specifying the `cryptosign` method as (one of) the authentication methods:

```
[1, "realm1", {
    "roles": {/* see below */},
    "authmethods": ["cryptosign"],
    "authid": "client01@example.com",
    "authextra": {
        "pubkey":
"545efb0a2192db8d43f118e9bf9aee081466e1ef36c708b96ee6f62dddad9122"
    }
}]
```

The `HELLO.Details.authmethods|list` is used by the client to announce the authentication methods it is prepared to perform. For WAMP-Cryptosign, this MUST include `"cryptosign"`.

The `HELLO.Details.authid|string` is the authentication ID (e.g. username) the client wishes to authenticate as. For WAMP-Cryptosign authentication, this MAY be provided. If no `authid` is provided, the router SHOULD automatically chose and assign an `authid` (e.g. the Hex encode public key).

The `HELLO.Details.authextra|dict` contains the following members for WAMP-Cryptosign:

Field	Type	Required	Description
pubkey	string	yes	The client public key (32 bytes) as a Hex encoded string, e.g. `545efb0a2192db8d43f118e9bf9aee081466e1ef36c708b96ee6f62`
channel_binding	string	no	If TLS channel binding is in use, the TLS channel binding type, `unique"`.
challenge	string	no	A client chosen, random challenge (32 bytes) as a Hex encode to be signed by the router.
trustroot	string	no	When the client includes a client certificate (see below), the Et address of the trustroot of the certificate chain to be used, e.g. `0x72b3486d38E9f49215b487CeAaDF27D6acf22115`, which can be *Standalone Trustroot* or an *On-chain Trustroot* (see Trustroots)

Table 14

The client needs to announce the WAMP `roles` and features it supports, for example:

```
{"callee": {"features": {"call_canceling": True,
                         "caller_identification": True,
                         "pattern_based_registration": True,
                         "progressive_call_results": True,
                         "registration_revocation": True,
                         "shared_registration": True}},
"caller": {"features": {"call_canceling": True,
                        "caller_identification": True,
                        "progressive_call_results": True}},
"publisher": {"features": {"publisher_exclusion": True,
                           "publisher_identification": True,
                           "subscriber_blackwhite_listing": True}},
"subscriber": {"features": {"pattern_based_subscription": True,
                            "publisher_identification": True,
                            "subscription_revocation": True}}}}
```

If the router is unwilling or unable to perform WAMP-Cryptosign authentication, it'll either skip forward trying other authentication methods (if the client announced any) or send an `ABORT` message.

If the router is willing to let the client authenticate using WAMP-Cryptosign and the router recognizes the provided `HELLO.Details.authextra.pubkey|string`, it'll send a `CHALLENGE` message:

```
[4, "cryptosign", {
    "challenge":
"fa034062ad76352b53a25358854577730db82f367aa439709c91296d04a5716c",
    "channel_binding": null
}]
```

The client will send an `AUTHENTICATE` message containing a signature:

```
[5
'e2f0297a193b63b7a4a92028e9e2e6107f82730560d54a657bd982cb4b3151490399debbbde9
98e494d3c3b2a5e2e91271291e10dee85a6cfaa127885ddd8b0afa034062ad76352b53a253588
54577730db82f367aa439709c91296d04a5716c', {}]
```

If the authentication succeeds, the server will router respond with a `WELCOME` message:

```
[2, 7562122397119786, {
    "authextra": {
        "x_cb_node": "intel-nuci7-27532",
        "x_cb_peer": "tcp4:127.0.0.1:49032",
        "x_cb_pid": 27637,
        "x_cb_worker": "worker001"
    },
    "authid": "client01@example.com",
    "authmethod": "cryptosign",
    "authprovider": "static",
    "authrole": "user",
    "realm": "realm1",
    "roles": {/* see below */}
}]
```

where

1. `authid|string`: The authentication ID the client was (actually) authenticated as.
2. `authrole|string`: The authentication role the client was authenticated for.
3. `authmethod|string`: The authentication method, here `"cryptosign"`
4. `authprovider|string`: The actual provider of authentication. For Ticket-based authentication, this can be freely chosen by the app, e.g. `static` or `dynamic`.

The `WELCOME.Details` again contain the actual authentication information active. If the authentication fails, the router will response with an `ABORT` message.

When the authentication is successful, `WELCOME.Details.roles|dict` will announce the roles and features the router supports:

```
{"broker": {"features": {"event_retention": True,
                                     "pattern_based_subscription": True,
                                     "publisher_exclusion": True,
                                     "publisher_identification": True,
                                     "session_meta_api": True,
                                     "subscriber_blackwhite_listing": True,
                                     "subscription_meta_api": True,
                                     "subscription_revocation": True}},
            "dealer": {"features": {"call_canceling": True,
                                     "caller_identification": True,
                                     "pattern_based_registration": True,
                                     "progressive_call_results": True,
                                     "registration_meta_api": True,
                                     "registration_revocation": True,
                                     "session_meta_api": True,
                                     "shared_registration": True,
                                     "testament_meta_api": True}}
    }
```

13.4.1.3. Test Vectors

The following test vectors allow to verify an implementation of WAMP-Cryptosign signatures. You can use `channel_id`, `private_key` and `challenge` as input, and check the computed signature matches `signature`.

The test vectors contain instances for both with and without a `channel_id`, which represents the TLS channel ID when using TLS with `tls-unique` channel binding.

```
test_vectors_1 = [
    {
        'channel_id': None,
        'private_key':
'4d57d97a68f555696620a6d849c0ce582568518d729eb753dc7c732de2804510',
        'challenge':
'ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff',
        'signature':
'b32675b221f08593213737bef8240e7c15228b07028e19595294678c90d11c0cae80a357331b
fc5cc9fb71081464e6e75013517c2cf067ad566a6b7b728e5d03ffffffffffffffffffffffff
ffffffffffffffffffffffffffffffffffffffff'
    },
    {
        'channel_id': None,
        'private_key':
'd511fe78e23934b3dadb52fcd022974b80bd92bccc7c5cf404e46cc0a8a2f5cd',
        'challenge':
'b26c1f87c13fc1da14997f1b5a71995dff8fbe0a62fae8473c7bdbd05bfb607d',
        'signature':
'd4209ad10d5aff6bfbc009d7e924795de138a63515efc7afc6b01b7fe5201372190374886a70
207b042294af5bd64ce725cd8dceb344e6d11c09d1aaaf4d660fb26c1f87c13fc1da14997f1b5
a71995dff8fbe0a62fae8473c7bdbd05bfb607d'
    },
    {
        'channel_id': None,
        'private_key':
'6e1fde9cf9e2359a87420b65a87dc0c66136e66945196ba2475990d8a0c3a25b',
        'challenge':
'b05e6b8ad4d69abf74aa3be3c0ee40ae07d66e1895b9ab09285a2f1192d562d2',
        'signature':
'7beb282184baadd08f166f16dd683b39cab53816ed81e6955def951cb2ddad1ec184e206746f
d82bda075af03711d3d5658fc84a76196b0fa8d1ebc92ef9f30bb05e6b8ad4d69abf74aa3be3c
0ee40ae07d66e1895b9ab09285a2f1192d562d2'
    },
    {
        'channel_id':
'62e935ae755f3d48f80d4d59f6121358c435722a67e859cc0caa8b539027f2ff',
        'private_key':
'4d57d97a68f555696620a6d849c0ce582568518d729eb753dc7c732de2804510',
        'challenge':
'ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff',
        'signature':
'9b6f41540c9b95b4b7b281c3042fa9c54cef43c842d62ea3fd6030fcb66e70b3e80d49d44c29
d1635da9348d02ec93f3ed1ef227dfb59a07b580095c2b82f80f9d16ca518aa0c2b707f2b2a60
9edeca73bca8dd59817a633f35574ac6fd80d00'
    },
    {
        'channel_id':
'62e935ae755f3d48f80d4d59f6121358c435722a67e859cc0caa8b539027f2ff',
        'private_key':
'd511fe78e23934b3dadb52fcd022974b80bd92bccc7c5cf404e46cc0a8a2f5cd',
        'challenge':
'b26c1f87c13fc1da14997f1b5a71995dff8fbe0a62fae8473c7bdbd05bfb607d',
        'signature':
'305aaa3ac25e98f651427688b3fc43fe7d8a68a7ec1d7d61c61517c519bd4a427c3015599d83
ca28b4c652333920223844ef0725eb5dc2febfd6af7677b73f01d0852a29b460fc92ec943242a
c638a053bbacc200512b18b30d15083cbdc9282'
```

```
    },
    {
        'channel_id':
'62e935ae755f3d48f80d4d59f6121358c435722a67e859cc0caa8b539027f2ff',
        'private_key':
'6e1fde9cf9e2359a87420b65a87dc0c66136e66945196ba2475990d8a0c3a25b',
        'challenge':
'b05e6b8ad4d69abf74aa3be3c0ee40ae07d66e1895b9ab09285a2f1192d562d2',
        'signature':
'ee3c7644fd8070532bc1fde3d70d742267da545d8c8f03e63bda63f1ad4214f4d2c4bfdb4eb9
526def42deeb7e31602a6ff99eba893e0a4ad4d45892ca75e608d2b75e24a189a7f78ca776ba3
6fc53f6c3e31c32f251f2c524f0a44202f2902d'
    },
]
```

13.4.2. TLS Channel Binding

TLS Channel Binding is an optional feature for WAMP-Cryptosign when running on top of TLS for link encryption. The use of "channel binding" to bind authentication at application layers to secure sessions at lower layers in the network stack protects against certain attack scenarios. For more background information, please see

- RFC5056: On the Use of Channel Bindings to Secure Channels
- Binding Security Tokens to TLS Channels

A client that wishes to use TLS Channel Binding with WAMP-Cryptosign must include an attribute `channel_binding` in the `authextra` sent in `HELLO.Details`:

```
[1, 'realm1', {
  'authextra': {
    'channel_binding': 'tls-unique',
    'pubkey':
'545efb0a2192db8d43f118e9bf9aee081466e1ef36c708b96ee6f62dddad9122'
  },
  'authmethods': ['cryptosign']
  }]
```

The `channel_binding`, if present, MUST be a string with a value of `tls-unique` or `tls-exporter`, to specify the channel binding type that is to be used:

- `tls-unique`: RFC5929: Channel Bindings for TLS
- `tls-exporter`: RFC9266: Channel Bindings for TLS 1.3

When a router receives a HELLO message from a client with a TLS channel binding attribute present, the router MUST:

1. get the TLS channel ID (32 bytes) of the TLS session with the respective channel type requested
2. generate new challenge (32 random bytes)
3. expect the client to send back a signature in AUTHENTICATE computed over `challenge XOR channel_id`

and send back the `channel_binding` in use, and the `challenge` in a CHALLENGE message:

```
[4, 'cryptosign', {
  'challenge':
'0e9192bc08512c8198da159c1ae600ba91729215f35d56102ee318558e773537',
  'channel_binding': 'tls-unique'}]
```

The authenticating client MUST verify the actual channel binding in use matches the one it requested. If a router does not support the `channel_binding` the client requested, it may chose to continue the authentication without channel binding, and hence `CHALLENGE.Extra` would not contain a `channel_binding`.

The client MUST then locally fetch the `channel_id` from the underlying TLS connection and sign `CHALLENGE.Extra.challenge XOR channel_id` using its private key.

13.4.3. Router Authentication

With the basic *Client Authentication* mechanism in WAMP-Cryptosign, the router is able to authenticate the client, since to successfully sign `CHALLENGE.Extra.challenge` the client will need the private key corresponding to the public key which the client announced in `HELLO.Details.pubkey` to be authenticated under.

However, from this alone, the client can not be sure the router against which it is authenticating is actually valid, as in authentic. *Router Authentication* adds this capability.

To request a router to authenticate, a client will start the authentication handshake by sending `HELLO.Details.challenge|string`:

```
[1, 'realm1', {
  'authextra': {
    'challenge':
'bbae60ea44cdd7b20dc7010a618b0f0803fab25a817520b4b7f057299b524deb',
    'pubkey':
'545efb0a2192db8d43f118e9bf9aee081466e1ef36c708b96ee6f62dddad9122'
    }}]
```

Similar to *Client Authentication*, the `challenge` must encode a 32 bytes random value as a string in HEX format, and the router MUST respond by signing this challenge value with its (the router's) private key, and send back the signature in `CHALLENGE.Extra.signature`

```
[4, 'cryptosign', {
  'challenge':
'0e9192bc08512c8198da159c1ae600ba91729215f35d56102ee318558e773537',
  'pubkey':
'4a3838f6fe75251e613329d53fc69b262d5eac97fb1d73bebbaed4015b53c862',
  'signature':
'fd5128d2d207ba58a9d1d6f41b72c747964ad9d1294077b3b1eee6130b05843ab12c53c7f251
9f73d4feb82db19d8ca0fc26b62bde6518e79a882f5795bc9f00bbae60ea44cdd7b20dc7010a6
18b0f0803fab25a817520b4b7f057299b524deb'}]
```

When *Router Authentication* is used, the router MUST also send its public key in `CHALLENGE.Extra.pubkey`.

Further, *Router Authentication* can be combined with *TLS Channel Binding*, in which case the value signed by the router will be `HELLO.Details.challenge XOR channel_id`.

13.4.4. Trustroots and Certificates

13.4.4.1. Certificate Chains

A public-key certificate is a signed statement that is used to establish an association between an identity and a public key. This is called a machine identity. The entity that vouches for this association and signs the certificate is the issuer of the certificate and the identity whose public key is being vouched for is the subject of the certificate. In order to associate the identity and the public key, a chain of certificates is used. The certificate chain is also called the certification path or chain of trust.

What is a Certificate Chain?

A certificate chain is a list of certificates (usually starting with an end-entity certificate) followed by one or more CA certificates (usually the last one being a self-signed certificate), with the following properties:

- The **issuer** of each certificate (except the last one) matches the **subject of** the next certificate in the list.
- Each certificate (except the last one) is supposed to be signed by the secret key corresponding to the next certificate in the chain (i.e. the signature of one certificate can be verified using the public key contained in the following certificate).
- The last certificate in the list is a **trust anchor**: a certificate that you trust because it was delivered to you by some trustworthy procedure. A trust anchor is a CA certificate (or more precisely, the public verification key of a CA) used by a relying party as the starting point for path validation.

More information about certificate chains, while in the context of x509, can be found in this white paper published by the *PKI forum*: Understanding Certification Path Construction.

What are On-chain Trust Anchors?

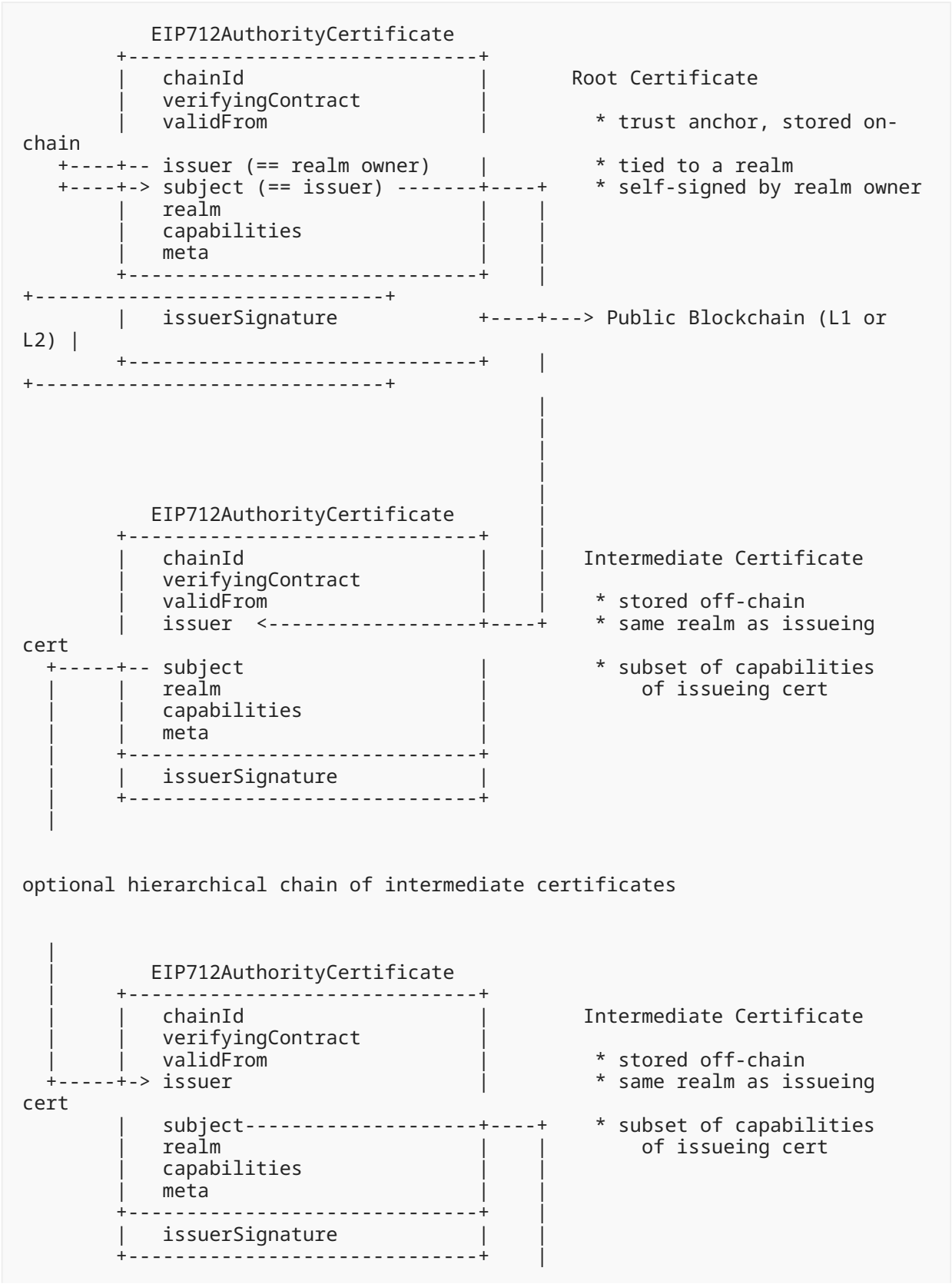
In x509, the set of trusted root CA certificates are stored in a machine/device local certificate store. This set of trusted root CA certificates are:

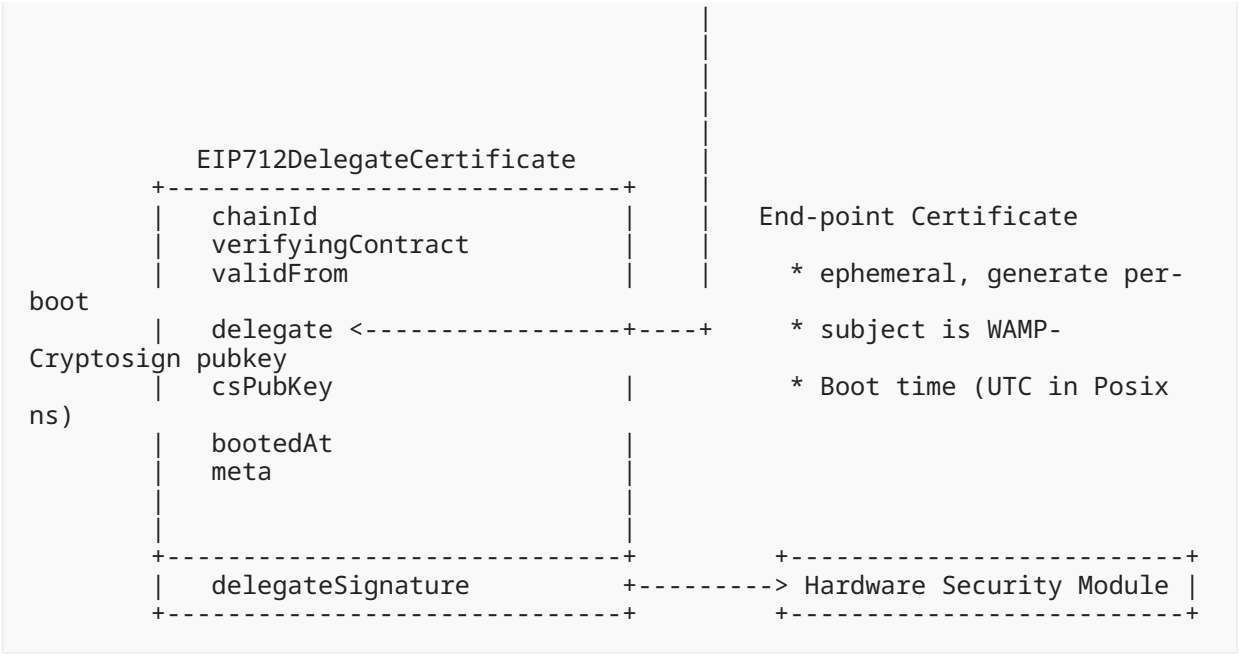
1. filled and fixed by the software or device vendor with a default root CA certificates set
2. may be extendable or replaceable by a user provided custom root CA certificates set

With 1., the user implicitly trusts the vendor, and all root CAs in the set installed by the vendor. With 2., the user must manage a public-private key infrastructure, and when information is to be shared with other parties, the use PKI must be made available to those parties, and the parties will operationally and administratively depend on the PKI hosting party. In summary, x509 follows a centralized and hierarchical trust model.

With WAMP-Cryptosign, we use a public blockchain for certificate chain trust anchors. Using a public blockchain, specifically Ethereum, provides a decentralized, shared and cryptographically secure storage for root CA certificates, that is trust anchors. These anchors can be associated with other entities stored on-chain, such as *federated Realms*.

The following diagram shows the structure of certificate chains in WAMP-Cryptosign:


EIP712AuthorityCertificate
chainId
verifyingContract
validFrom
issuer (== realm owner)
subject (== issuer)
realm
capabilities
meta
issuerSignature
Root Certificate
* trust anchor, stored on-chain
* tied to a realm
* self-signed by realm owner
Public Blockchain (L1 or L2)
EIP712AuthorityCertificate
chainId
verifyingContract
validFrom
issuer
subject
realm
capabilities
meta
issuerSignature
Intermediate Certificate
* stored off-chain
* same realm as issueing cert
* subset of capabilities of issueing cert
optional hierarchical chain of intermediate certificates
EIP712AuthorityCertificate
chainId
verifyingContract
validFrom
issuer
subject
realm
capabilities
meta
issuerSignature
Intermediate Certificate
* stored off-chain
* same realm as issueing cert
* subset of capabilities of issueing cert



13.4.4.2. Certificate Types

The certificate types `EIP712AuthorityCertificate` and `EIP712DelegateCertificate` follow EIP712 and use Ethereum signatures.

`EIP712AuthorityCertificate`:

```
[
    {
        "name": "chainId",
        "type": "uint256"
    },
    {
        "name": "verifyingContract",
        "type": "address"
    },
    {
        "name": "validFrom",
        "type": "uint256"
    },
    {
        "name": "issuer",
        "type": "address"
    },
    {
        "name": "subject",
        "type": "address"
    },
    {
        "name": "realm",
        "type": "address"
    },
    {
        "name": "capabilities",
        "type": "uint64"
    },
    {
        "name": "meta",
        "type": "string"
    }
]
```

`EIP712DelegateCertificate`:

```
[
    {
        "name": "chainId",
        "type": "uint256"
    },
    {
        "name": "verifyingContract",
        "type": "address"
    },
    {
        "name": "validFrom",
        "type": "uint256"
    },
    {
        "name": "delegate",
        "type": "address"
    },
    {
        "name": "csPubKey",
        "type": "bytes32"
    },
    {
        "name": "bootedAt",
        "type": "uint64"
    },
    {
        "name": "meta",
        "type": "string"
    }
]
```

The EIP712 types for certificates contain:

- `chainId`: the chain ID of the blockchain this signed typed data is bound to
- `verifyingContract`: the address of the (main) smart contract this signed typed data is bound to

This prevents cross-chain and cross-contract attacks. The `chainId` is an integer according to EIP155:

- Ethereum Mainnet (ChainID 1)
- Goerli Testnet (ChainID 5)
- zkSync 2.0 Alpha Testnet (ChainID 280)

Besides EIP712, other comparable approaches to specify cryptographically hashable, typed structured data ("messages") include:

- Veriform: cryptographically verifiable and canonicalized message format similar to Protocol Buffers, with an "embedded-first" (heapless) implementation suitable for certificates or other signed objects
- objecthash: A way to cryptographically hash objects (in the JSON-ish sense) that works cross-language. And, therefore, cross-encoding.

13.4.4.3. Capabilities

- **Bit 0**: `CAPABILITY_ROOT_CA`
- **Bit 1**: `CAPABILITY_INTERMEDIATE_CA`
- **Bit 2**: `CAPABILITY_PUBLIC_RELAY`
- **Bit 3**: `CAPABILITY_PRIVATE_RELAY`
- **Bit 4**: `CAPABILITY_GATEWAY`
- **Bit 5**: `CAPABILITY_EXCHANGE`
- **Bit 6**: `CAPABILITY_PROVIDER`
- **Bit 7**: `CAPABILITY_CONSUMER`
- **Bits 8 - 63**: future use, all set to `0`

Permission to create a `CAPABILITY_PUBLIC_RELAY` certificate on a realm can be configured by the realm owner for:

- `PRIVATE`: signed by realm owner
- `PERMISSIONED`: signed by requestor and realm owner
- `OPEN`: signed by requestor

Permission for `CAPABILITY_ROOT_CA` is always `PRIVATE`.

13.4.4.4. Certificate Chain Verification

use of a specific method/mechanism, when it comes to establishing trust (i.e. certifying public keys).

To verify a certificate chain and respective certificate signatures

```
[
    (EIP712DelegateCertificate, Signature),       // delegate certificate
    (EIP712AuthorityCertificate, Signature),      // intermediate CA
certificate
    ...
    (EIP712AuthorityCertificate, Signature),      // intermediate CA
certificate
    (EIP712AuthorityCertificate, Signature)       // root CA certificate
]
```

the following Certificate Chain Rules (CCR) must be checked:

1. **CCR-1**: The `chainId` and `verifyingContract` must match for all certificates to what we expect, and `validFrom` before current block number on the respective chain.
2. **CCR-2**: The `realm` must match for all certificates to the respective realm.
3. **CCR-3**: The type of the first certificate in the chain must be a `EIP712DelegateCertificate`, and all subsequent certificates must be of type `EIP712AuthorityCertificate`.
4. **CCR-4**: The last certificate must be self-signed (`issuer` equals `subject`), it is a root CA certificate.

5. **CCR-5**: The intermediate certificate's `issuer` must be equal to the `subject` of the previous certificate.
6. **CCR-6**: The root certificate must be `validFrom` before the intermediate certificate
7. **CCR-7**: The `capabilities` of intermediate certificate must be a subset of the root cert
8. **CCR-8**: The intermediate certificate's `subject` must be the delegate certificate `delegate`
9. **CCR-9**: The intermediate certificate must be `validFrom` before the delegate certificate
10. **CCR-10**: The root certificate's signature must be valid and signed by the root certificate's `issuer`.
11. **CCR-11**: The intermediate certificate's signature must be valid and signed by the intermediate certificate's `issuer`.
12. **CCR-12**: The delegate certificate's signature must be valid and signed by the `delegate`.

13.4.4.5. Trustroots

Certificate chains allow to verify a delegate certificate following the Issuers-Subjects up to a *Root CA*, which is a self-signed certificate (issuer and subject are identical). The *Root CA* represents the *Trustroot* of all involved delegates.

When both a connecting WAMP client and the WAMP router are using the same *Root CA* and thus use a common *Trustroot*, they are said to be authorized in the same trust domain (identified by the trustroot).

Trustroots are identified by their Ethereum address, which is computed from the issuer public key according to EIP-55.

There are two types of *Root CAs* and *Trustroots*:

1. *Standalone Trustroot*
2. *On-chain Trustroot*

A *Standalone Trustroot* is managed by a single operator/owner, does not allow infrastructure elements (nodes, client, realms) to be integrated between different operators/owners and is privately stored on the respective operators systems only, usually as files or in databases.

Note that the Ethereum *address*, can be *computed* deterministically from the public key of the issuer of a certificate, even when the certificate (or the issuer) is *not* stored on-chain.

An *On-chain Trustroot* in contrast is stored in Ethereum and publically shared between different operators/owners which allows infrastructure elements (nodes, clients, realms) to be integrated. For example, clients/nodes operated by different operators can authenticate to each other and nodes operated by different operators can authenticate to each other sharing the hosting of one realm.

The management of *On-Chain Trustroots* depends on the policy of the trustroot which is chosen and fixed when the trustroot is created:

1. *Open*
2. *Permitted*
3. *Private*

With an *Open On-chain Trustroot*, new certificates can be added to a certificate chain freely and only requires a signature by the respective intermediate CA issuer.

13.4.4.5.1. Standalone Trustroots

For a *Standalone Trustroot* the `trustroot` MUST be specified in `HELLO.Details.authextra.trustroot|string`

```
{'authextra': {'certificates': [/* certificate, see below */],
               'challenge':
'2763e7fdb1c34a74e8497daf6c913744d11161a94cec3b16aeec60a788612e17',
               'channel_binding': 'tls-unique',
               'pubkey':
'12ae0184b180e9a9c5e45be4a1afbce3c6491320063701cd9c4011a777d04089',
               'trustroot': '0xf766Dc789CF04CD18aE75af2c5fAf2DA6650Ff57'},
```

and `certificates` MUST contain

- a single `EIP712DelegateCertificate`, and have the complete certificate chain of `EIP712AuthorityCertificates` up to `trustroot` pre-agreed (locally stored or built-in) OR
- the complete chain of certificates starting with a `EIP712DelegateCertificate` followed by one or more `EIP712AuthorityCertificates` up to `trustroot`.

Example 3 contains an example gor the latter, with a bundled complete certificate chain, that is the last certificate in the list is self-signed (is a root CA certificate) and matches `trustroot`

```
trustroot == 0xf766Dc789CF04CD18aE75af2c5fAf2DA6650Ff57
          == certificates[-1].issuer
          == certificates[-1].subject
```

13.4.4.5.2. On-chain Trustroots

For an *On-chain Trustroot* the `trustroot` MUST be specified in `HELLO.Details.authextra.trustroot|string`

```
{'authextra': {'certificates': [/* certificate, see below */],
               'challenge':
'2763e7fdb1c34a74e8497daf6c913744d11161a94cec3b16aeec60a788612e17',
               'channel_binding': 'tls-unique',
               'pubkey':
'12ae0184b180e9a9c5e45be4a1afbce3c6491320063701cd9c4011a777d04089',
               'trustroot': '0xf766Dc789CF04CD18aE75af2c5fAf2DA6650Ff57'},
```

and `certificates` MUST contain a single `EIP712DelegateCertificate`, and have the complete certificate chain of `EIP712AuthorityCertificates` up to `trustroot` stored on-chain (Ethereum).

This is called a free-standing, on-chain CA.

When the `trustroot` is associated with an on-chain *Realm* that has `trustroot` configured as the *Realm CA*, this is called *On-chain CA with CA associated with On-chain Realm*.

13.4.5. Remote Attestation

Remote attestation is a method by which a host (WAMP client) authenticates its hardware and software configuration to a remote host (WAMP router). The goal of remote attestation is to enable a remote system (challenger) to determine the level of trust in the integrity of the platform of another system (attestator).

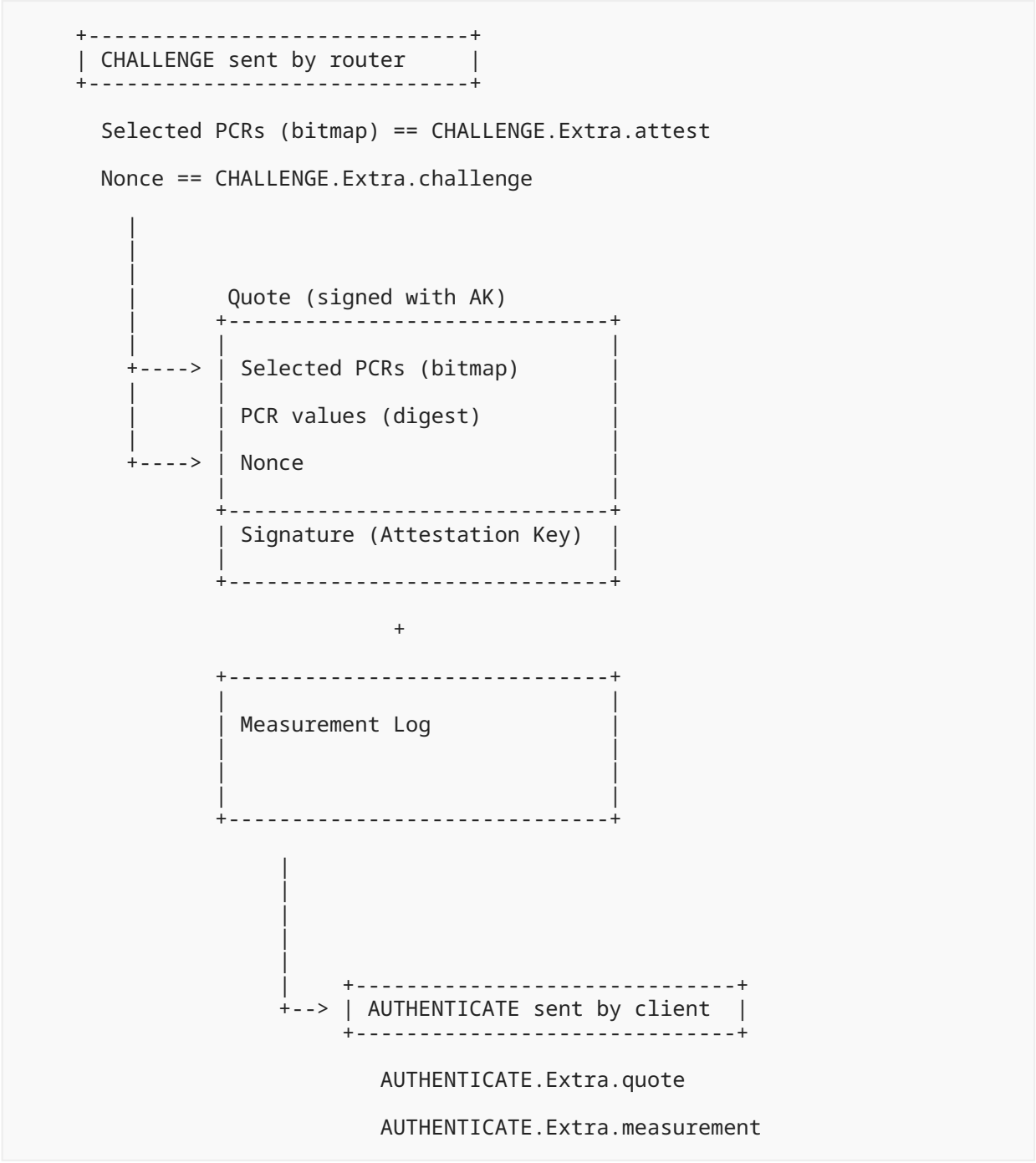
Remote attestation allows to

- perform security decisions based on security policy and measurement log
- tie device identity into authentication infrastructure
- verify device state in access control decisions
- avoid exfiltration of credentials

Remote attestation is requested by the router sending `CHALLENGE.extra.attest|list[int]` with a list of device PCRs to be quoted. A list of all PCRs available (usually 24) in a PCR bank of a device can be obtained running tpm2_pcrread without arguments.

A client receiving such a `CHALLENGE` MUST include an *Event Log* with PCRs collected from *measured boot* signed by the device's security module's *Attestation Key (AK)* and using the challenge sent by the router `CHALLENGE.extra.challenge|string` as a nonce. TPM 2.0 of the TCG specifies a suitable function in tss2_quote (also see here).

The client MUST include the signed attestation in `AUTHENTICATE.Extra.quote` and the corresponding measurement log in `AUTHENTICATE.Extra.measurement`. The following diagram illustrates *Remote Attestation* with WAMP-Cryptosign:



13.4.6. Example Message Exchanges

- Example 1
- Example 2
- Example 3

13.4.6.1. Example 1

- *with* router challenge
- *without* TLS channel binding

```
WAMP-Transmit(-, -) >>
  HELLO::
    [1,
     'devices',
     {'authextra': {'challenge':
'bbae60ea44cdd7b20dc7010a618b0f0803fab25a817520b4b7f057299b524deb',
                    'channel_binding': None,
                    'pubkey':
'545efb0a2192db8d43f118e9bf9aee081466e1ef36c708b96ee6f62dddad9122'},
      'authmethods': ['cryptosign'],
      'roles': {'callee': {'features': {'call_canceling': True,
                                        'caller_identification': True,
                                        'pattern_based_registration': True,
                                        'progressive_call_results': True,
                                        'registration_revocation': True,
                                        'shared_registration': True}},
                'caller': {'features': {'call_canceling': True,
                                        'caller_identification': True,
                                        'progressive_call_results': True}},
                'publisher': {'features': {'publisher_exclusion': True,
                                           'publisher_identification': True,
                                           'subscriber_blackwhite_listing':
True}},
                'subscriber': {'features': {'pattern_based_subscription':
True,
                                            'publisher_identification': True,
                                            'subscription_revocation':
True}}}}]
>>

WAMP-Receive(-, -) <<
  CHALLENGE::
    [4,
     'cryptosign',
     {'challenge':
'0e9192bc08512c8198da159c1ae600ba91729215f35d56102ee318558e773537',
      'channel_binding': None,
      'pubkey':
'4a3838f6fe75251e613329d53fc69b262d5eac97fb1d73bebbaed4015b53c862',
      'signature':
'fd5128d2d207ba58a9d1d6f41b72c747964ad9d1294077b3b1eee6130b05843ab12c53c7f251
9f73d4feb82db19d8ca0fc26b62bde6518e79a882f5795bc9f00bbae60ea44cdd7b20dc7010a6
18b0f0803fab25a817520b4b7f057299b524deb'}]
<<

WAMP-Transmit(-, -) >>
  AUTHENTICATE::
    [5,

'a3a178fe792ed772a8fc092f8341e455de96670c8901264a7c312dbf940d5743626fe9fbc29b
23dcd2169b308eca309de85a89ccd296b24835de3d95b16b77030e9192bc08512c8198da159c1
ae600ba91729215f35d56102ee318558e773537',
     {}]
>>

WAMP-Receive(-, -) <<
  WELCOME::
```

```
    [2,
     3735119691078036,
     {'authextra': {'x_cb_node': 'intel-nuci7-49879',
                    'x_cb_peer': 'tcp4:127.0.0.1:53976',
                    'x_cb_pid': 49987,
                    'x_cb_worker': 'worker001'},
      'authid': 'client01@example.com',
      'authmethod': 'cryptosign',
      'authprovider': 'static',
      'authrole': 'device',
      'realm': 'devices',
      'roles': {'broker': {'features': {'event_retention': True,
                                        'pattern_based_subscription': True,
                                        'publisher_exclusion': True,
                                        'publisher_identification': True,
                                        'session_meta_api': True,
                                        'subscriber_blackwhite_listing':
True,
                                        'subscription_meta_api': True,
                                        'subscription_revocation': True}},
                'dealer': {'features': {'call_canceling': True,
                                        'caller_identification': True,
                                        'pattern_based_registration': True,
                                        'progressive_call_results': True,
                                        'registration_meta_api': True,
                                        'registration_revocation': True,
                                        'session_meta_api': True,
                                        'shared_registration': True,
                                        'testament_meta_api': True}}},
      'x_cb_node': 'intel-nuci7-49879',
      'x_cb_peer': 'tcp4:127.0.0.1:53976',
      'x_cb_pid': 49987,
      'x_cb_worker': 'worker001'}]
<<

WAMP-Transmit(3735119691078036, client01@example.com) >>
  GOODBYE::
    [6, {}, 'wamp.close.normal']
>>

WAMP-Receive(3735119691078036, client01@example.com) <<
  GOODBYE::
    [6, {}, 'wamp.close.normal']
<<
```

13.4.6.2. Example 2

- *with* router challenge
- *with* TLS channel binding

```
WAMP-Transmit(-, -) >>
  HELLO::
    [1,
     'devices',
     {'authextra': {'challenge':
'4f861f12796c2972b7b0026522a687aa851d90355122a61d4f1fdce4d06b564f',
                    'channel_binding': 'tls-unique',
                    'pubkey':
'545efb0a2192db8d43f118e9bf9aee081466e1ef36c708b96ee6f62dddad9122'},
      'authmethods': ['cryptosign'],
      'roles': {'callee': {'features': {'call_canceling': True,
                                        'caller_identification': True,
                                        'pattern_based_registration': True,
                                        'progressive_call_results': True,
                                        'registration_revocation': True,
                                        'shared_registration': True}},
                'caller': {'features': {'call_canceling': True,
                                        'caller_identification': True,
                                        'progressive_call_results': True}},
                'publisher': {'features': {'publisher_exclusion': True,
                                           'publisher_identification': True,
                                           'subscriber_blackwhite_listing':
True}},
                'subscriber': {'features': {'pattern_based_subscription':
True,
                                            'publisher_identification': True,
                                            'subscription_revocation':
True}}}}]
>>

WAMP-Receive(-, -) <<
  CHALLENGE::
    [4,
     'cryptosign',
     {'challenge':
'358625312c6c3bf64ed51d17d210ce21af1639c774cabf5735a9651d7d91fc6a',
      'channel_binding': 'tls-unique',
      'pubkey':
'4a3838f6fe75251e613329d53fc69b262d5eac97fb1d73bebbaed4015b53c862',
      'signature':
'aa05f4cd7747d36b79443f1d4703a681e107edc085d876b508714e2a3a8135bacaae1c018452
c4acb3ad2818aa97a6d23e5ac7e3734c7b1f40e6232a70938205a6f5a1f034a28090b195fb2ce
2454a82532f5c8baf6ba1dfb5ddae63c09ce72f'}]
<<

WAMP-Transmit(-, -) >>
  AUTHENTICATE::
    [5,

'25114474580d6e99a6126b091b4565c23db567d686c5b8c3a94e3f2f09dc80300c5b40a12423
6733fa56396df721eb12ac092362379bd5b27b4db9e2beaa1408dcf59bd361a2921448f0e45e1
2f303097924f5798a83b895cf6b179a6d664d0a',
     {}]
>>

WAMP-Receive(-, -) <<
  WELCOME::
```

```
    [2,
     7325966140445461,
     {'authextra': {'x_cb_node': 'intel-nuci7-49879',
                    'x_cb_peer': 'tcp4:127.0.0.1:54046',
                    'x_cb_pid': 49987,
                    'x_cb_worker': 'worker001'},
      'authid': 'client01@example.com',
      'authmethod': 'cryptosign',
      'authprovider': 'static',
      'authrole': 'device',
      'realm': 'devices',
      'roles': {'broker': {'features': {'event_retention': True,
                                        'pattern_based_subscription': True,
                                        'publisher_exclusion': True,
                                        'publisher_identification': True,
                                        'session_meta_api': True,
                                        'subscriber_blackwhite_listing':
True,
                                        'subscription_meta_api': True,
                                        'subscription_revocation': True}},
                'dealer': {'features': {'call_canceling': True,
                                        'caller_identification': True,
                                        'pattern_based_registration': True,
                                        'progressive_call_results': True,
                                        'registration_meta_api': True,
                                        'registration_revocation': True,
                                        'session_meta_api': True,
                                        'shared_registration': True,
                                        'testament_meta_api': True}}},
      'x_cb_node': 'intel-nuci7-49879',
      'x_cb_peer': 'tcp4:127.0.0.1:54046',
      'x_cb_pid': 49987,
      'x_cb_worker': 'worker001'}]
<<
2022-07-13T17:38:29+0200 session joined: {'authextra': {'x_cb_node': 'intel-
nuci7-49879',
               'x_cb_peer': 'tcp4:127.0.0.1:54046',
               'x_cb_pid': 49987,
               'x_cb_worker': 'worker001'},
 'authid': 'client01@example.com',
 'authmethod': 'cryptosign',
 'authprovider': 'static',
 'authrole': 'device',
 'realm': 'devices',
 'resumable': False,
 'resume_token': None,
 'resumed': False,
 'serializer': 'cbor.batched',
 'session': 7325966140445461,
 'transport': {'channel_framing': 'websocket',
               'channel_id': {'tls-unique': b'\xe9s\xbe\xe2M\xce\xa9\xe2'
                                            b'\x06%\xf9I\xc0\xe3\xcd('
                                            b'\xd62\xcc\xbe\xfeI\x07\xc2'
                                            b'\xfa\xc2r\x87\x10\xf7\xb1`'},
               'channel_serializer': None,
               'channel_type': 'tls',
               'http_cbtid': None,
               'http_headers_received': None,
```

```
                'http_headers_sent': None,
                'is_secure': True,
                'is_server': False,
                'own': None,
                'own_fd': -1,
                'own_pid': 50690,
                'own_tid': 50690,
                'peer': 'tcp4:127.0.0.1:8080',
                'peer_cert': None,
                'websocket_extensions_in_use': None,
                'websocket_protocol': None}}

WAMP-Transmit(7325966140445461, client01@example.com) >>
  GOODBYE::
    [6, {}, 'wamp.close.normal']
>>

WAMP-Receive(7325966140445461, client01@example.com) <<
  GOODBYE::
    [6, {}, 'wamp.close.normal']
<<
```

13.4.6.3. Example 3

- *with* router challenge
- *with* TLS channel binding
- *with* client trustroot and certificates

```
WAMP-Transmit(-, -) >>
  HELLO::
    [1,
     'devices',
     {'authextra': {'certificates': [({'domain': {'name': 'WMP', 'version':
'1'},
                                       'message': {'bootedAt':
1658765756680628959,
                                                   'chainId': 1,
                                                   'csPubKey':
'12ae0184b180e9a9c5e45be4a1afbce3c6491320063701cd9c4011a777d04089',
                                                   'delegate':
'0xf5173a6111B2A6B3C20fceD53B2A8405EC142bF6',
                                                   'meta': '',
                                                   'validFrom': 15212703,
                                                   'verifyingContract':
'0xf766Dc789CF04CD18aE75af2c5fAf2DA6650Ff57'},
                                       'primaryType':
'EIP712DelegateCertificate',
                                       'types': {'EIP712DelegateCertificate':
[{'name': 'chainId',

  'type': 'uint256'},

 {'name': 'verifyingContract',

  'type': 'address'},

 {'name': 'validFrom',

  'type': 'uint256'},

 {'name': 'delegate',

  'type': 'address'},

 {'name': 'csPubKey',

  'type': 'bytes32'},

 {'name': 'bootedAt',

  'type': 'uint64'},

 {'name': 'meta',

  'type': 'string'}],
                                                  'EIP712Domain': [{'name':
'name',
                                                                    'type':
'string'},
                                                                   {'name':
'version',
                                                                    'type':
'string'}]}},

'8fe06bb269110c6bc0e011ea2b7da07091c674f7fe67458c1805157157da702b70b56cdf6626
```

```
66dc386820ded0116b6b84151df1ed65210eeecd7e477cdb765b1b'),
                                    ({'domain': {'name': 'WMP', 'version':
'1'},
                                      'message': {'capabilities': 12,
                                                  'chainId': 1,
                                                  'issuer':
'0xf766Dc789CF04CD18aE75af2c5fAf2DA6650Ff57',
                                                  'meta': '',
                                                  'realm':
'0xA6e693CC4A2b4F1400391a728D26369D9b82ef96',
                                                  'subject':
'0xf5173a6111B2A6B3C20fceD53B2A8405EC142bF6',
                                                  'validFrom': 15212703,
                                                  'verifyingContract':
'0xf766Dc789CF04CD18aE75af2c5fAf2DA6650Ff57'},
                                      'primaryType':
'EIP712AuthorityCertificate',
                                      'types':
{'EIP712AuthorityCertificate': [{'name': 'chainId',

   'type': 'uint256'},

  {'name': 'verifyingContract',

   'type': 'address'},

  {'name': 'validFrom',

   'type': 'uint256'},

  {'name': 'issuer',

   'type': 'address'},

  {'name': 'subject',

   'type': 'address'},

  {'name': 'realm',

   'type': 'address'},

  {'name': 'capabilities',

   'type': 'uint64'},

  {'name': 'meta',

   'type': 'string'}],
                                                 'EIP712Domain': [{'name':
'name',
                                                                   'type':
'string'},
                                                                  {'name':
'version',
                                                                   'type':
'string'}]}},
```

```
'0c0eb60a108dbd72a204b41c1d18505358e4e7886b0c9787192a33ac9e0f94c92ce158f8de57
6fa9cccf28a8c9404ed66c2d355ea4ae7ee65cff0b73215b91bb1c'),
                                    ({'domain': {'name': 'WMP', 'version':
'1'},
                                      'message': {'capabilities': 63,
                                                  'chainId': 1,
                                                  'issuer':
'0xf766Dc789CF04CD18aE75af2c5fAf2DA6650Ff57',
                                                  'meta': '',
                                                  'realm':
'0xA6e693CC4A2b4F1400391a728D26369D9b82ef96',
                                                  'subject':
'0xf766Dc789CF04CD18aE75af2c5fAf2DA6650Ff57',
                                                  'validFrom': 15212703,
                                                  'verifyingContract':
'0xf766Dc789CF04CD18aE75af2c5fAf2DA6650Ff57'},
                                      'primaryType':
'EIP712AuthorityCertificate',
                                      'types':
{'EIP712AuthorityCertificate': [{'name': 'chainId',

   'type': 'uint256'},

  {'name': 'verifyingContract',

   'type': 'address'},

  {'name': 'validFrom',

   'type': 'uint256'},

  {'name': 'issuer',

   'type': 'address'},

  {'name': 'subject',

   'type': 'address'},

  {'name': 'realm',

   'type': 'address'},

  {'name': 'capabilities',

   'type': 'uint64'},

  {'name': 'meta',

   'type': 'string'}],
                                                 'EIP712Domain': [{'name':
'name',
                                                                   'type':
'string'},
                                                                  {'name':
'version',
                                                                   'type':
'string'}]}},
```

```
'be35c8d6ae735d3bd8b5e27b1e1a067eba53e6a1cb4ef0f607c4717435e8ffa676246e7d08df
b4e83c78ad26f423b727b5d2c90627bdf6c94c1dbdf01979c34b1c')],
                     'challenge':
'2763e7fdb1c34a74e8497daf6c913744d11161a94cec3b16aeec60a788612e17',
                     'channel_binding': 'tls-unique',
                     'pubkey':
'12ae0184b180e9a9c5e45be4a1afbce3c6491320063701cd9c4011a777d04089',
                     'trustroot':
'0xf766Dc789CF04CD18aE75af2c5fAf2DA6650Ff57'},
      'authmethods': ['cryptosign'],
      'roles': {'callee': {'features': {'call_canceling': True,
                                        'caller_identification': True,
                                        'pattern_based_registration': True,
                                        'progressive_call_results': True,
                                        'registration_revocation': True,
                                        'shared_registration': True}},
                'caller': {'features': {'call_canceling': True,
                                        'caller_identification': True,
                                        'progressive_call_results': True}},
                'publisher': {'features': {'publisher_exclusion': True,
                                           'publisher_identification': True,
                                           'subscriber_blackwhite_listing':
True}},
                'subscriber': {'features': {'pattern_based_subscription':
True,
                                            'publisher_identification': True,
                                            'subscription_revocation':
True}}}}]
>>

WAMP-Receive(-, -) <<
  CHALLENGE::
    [4,
     'cryptosign',
     {'challenge':
'e4b40f72f9604754789d472225483bace926b9668d72c9122545e540d8d98f23',
      'channel_binding': 'tls-unique',
      'pubkey':
'4a3838f6fe75251e613329d53fc69b262d5eac97fb1d73bebbaed4015b53c862',
      'signature':
'ce456092998d796533d7ef2bab543300409d161066c9520c9284df6bbfb82947b37fb78d69fd
56e5118afec62e35e015c60569af2e18ed92fedc738552242d039a38790e9c94064d89335393d
39973c14074cd1008d7266de74c641103e30609'}]
<<

WAMP-Transmit(-, -) >>
  AUTHENTICATE::
    [5,

'16c89629e72aff3f44661e701341b2221a2fa9d93205826fad85e70d3a8dab70a8f54314c14d
470ebeb77a0dd16c833928c01134a52b2e73862b7d3f258b600059ef9181d4370b6d19e7691e9
a407f29784315dfc949d4696ce5e1f6535ba73d',
     {}]
>>

WAMP-Receive(-, -) <<
  WELCOME::
```

```
    [2,
     869996509191260,
     {'authextra': {'x_cb_node': 'intel-nuci7-30969',
                    'x_cb_peer': 'tcp4:127.0.0.1:59172',
                    'x_cb_pid': 31090,
                    'x_cb_worker': 'worker001'},
      'authid': '0xf5173a6111B2A6B3C20fceD53B2A8405EC142bF6',
      'authmethod': 'cryptosign',
      'authprovider': 'static',
      'authrole': 'user',
      'realm': 'realm1',
      'roles': {'broker': {'features': {'event_retention': True,
                                        'pattern_based_subscription': True,
                                        'publisher_exclusion': True,
                                        'publisher_identification': True,
                                        'session_meta_api': True,
                                        'subscriber_blackwhite_listing':
True,
                                        'subscription_meta_api': True,
                                        'subscription_revocation': True}},
                'dealer': {'features': {'call_canceling': True,
                                        'caller_identification': True,
                                        'pattern_based_registration': True,
                                        'progressive_call_results': True,
                                        'registration_meta_api': True,
                                        'registration_revocation': True,
                                        'session_meta_api': True,
                                        'shared_registration': True,
                                        'testament_meta_api': True}}},
      'x_cb_node': 'intel-nuci7-30969',
      'x_cb_peer': 'tcp4:127.0.0.1:59172',
      'x_cb_pid': 31090,
      'x_cb_worker': 'worker001'}]
<<

WAMP-Transmit(869996509191260, 0xf5173a6111B2A6B3C20fceD53B2A8405EC142bF6) >>
  GOODBYE::
    [6, {}, 'wamp.close.normal']
>>

WAMP-Receive(869996509191260, 0xf5173a6111B2A6B3C20fceD53B2A8405EC142bF6) <<
  GOODBYE::
    [6, {}, 'wamp.close.normal']
<<
```

13.5. Dynamic Authentication API

Write me.

13.6. Authorization

WAMP allows user services to integrate seamlessly while enabling *Clients* to perform *Actions*, namely, to

- **register** procedures using fully qualified URIs or URI patterns in order to receive invocations

- **call** procedures at (fully qualified) URIs
- **subscribe** to topics using fully qualified URIs or URI patterns in order to receive events
- **publish** events to (fully qualified) URIs

Performing these actions requires *Clients* to have an open *Session* to the same shared *Realm*.

A *Session* is established between a *Client* to a *Router*, and is initiated by a *Client*.

Sessions MAY be required to *Authenticate* access to a *Realm* hosted by a *Router*.

Authentication is the sequence of operations that allow a *Router* to verify the identity of a *Session*, often as a prerequisite to allowing access to resources within a *Realm*.

When the *Session* authenticates to a *Router* successfully, the *Router* will have established the

- `realm|string`,
- `authrole|string`, and
- `authid|string`

for that *Session* running in the *Client*.

The triple `(realm, authrole, authid)` is called *Principal*, and a *Session* is authenticated under that *Principal*.

At any moment, there can be zero, one, or many *Sessions* with different *session ids* authenticated under the *same Principal*.

Sessions MAY be required to *Authorize* in order to perform a specific *Action* on an URI - or an URI pattern - within a *Realm*.

This distinction between *Authentication* and *Authorization* follows the established practice called "AAA":

- **A**uthentication: Establishes who it is (**"subject"**)
- **A**uthorization: Decides within a *Realm* whether an *Action* (**"operation"**) on an URI or URI pattern (**"object"**) is allowed for the requesting *Principal* (**"subject"**)
- **A**ccounting: Records, for a *Realm*, what *Action* (**"operation"**) on what URI or URI pattern (**"object"**) was requested by which *Principal* (**"subject"**), and whether it was allowed or denied

14. Advanced Security Features

This section covers some advanced features and techniques provided by WAMP mainly but not limited to security and cryptography.

14.1. Payload Passthru Mode

In some situations, you may want to reduce the access the router has to the information users transmit, or payload data is presented in some specific format that can not be simply recognized by WAMP router serializer.

Here are some use cases:

- Using WAMP via gateways to other technologies like MQTT Brokers or AMQP Queues. So the actual payload is, for example, MQTT message that should be delivered to a WAMP topic as is.
- Sensitive user data that should be delivered to a target *Callee* without any possibility of unveiling it in transit.

The above use cases can be fulfilled with the `Payload Passthru Mode` feature. This feature allows:

- Specifying additional attributes within `CALL`, `PUBLISH`, `EVENT`, `YIELD`, `RESULT` messages to signal the *Router* to skip payload inspection/conversion.
- The forwarding of these additional attributes via `INVOCATION` and `ERROR` messages
- Encrypting and decrypting payload using cryptographic algorithms.
- Providing additional information about payload format and type.

Feature Announcement

Support for this advanced feature MUST be announced by *Callers* (`role := "caller"`), *Callees* (`role := "callee"`), *Dealers* (`role := "dealer"`), *Publishers* (`role := "publisher"`), *Subscribers* (`role := "subscriber"`) and *Brokers* (`role := "broker"`) via

HELLO.Details.roles.<role>.features.payload_passthru_mode|bool := true

Payload Passthru Mode can work only if all three nodes (*Caller*, *Dealer*, *Callee* or *Publisher*, *Broker*, *Subscriber*) support and announced this feature.

Cases where a *Caller* sends a `CALL` message with `payload passthru` without announcing it during the `HELLO` handshake MUST be treated as *PROTOCOL ERRORS* and underlying WAMP connections must be aborted with the `wamp.error.protocol_violation` error reason.

Cases where a *Caller* sends a `CALL` message with `payload passthru` to a *Dealer*, the latter not announcing `payload passthru` support during `WELCOME` handshake MUST be treated as *PROTOCOL ERRORS* and the underlying WAMP connections must be aborted with the `wamp.error.protocol_violation` error reason.

Cases where a *Caller* sends a `CALL` message with `payload passthru` to a *Dealer* that supports this feature, which then must be routed to a *Callee* which doesn't support `payload passthru`, MUST be treated as *APPLICATION ERRORS* and the *Dealer* MUST respond to the *Caller* with a `wamp.error.feature_not_supported` error message.

Cases where a *Publisher* sends a `PUBLISH` message with `payload passthru`, without announcing it during `HELLO` handshake, MUST be treated as *PROTOCOL ERRORS* and the underlying WAMP connections must be aborted with the `wamp.error.protocol_violation` error reason.

Cases where a *Publisher* sends a `PUBLISH` message with `payload passthru` to a *Broker*, with the latter not announcing `payload passthru` support during the `WELCOME` handshake, MUST be treated as *PROTOCOL ERRORS* and the underlying WAMP connections must be aborted with the `wamp.error.protocol_violation` error reason.

Cases where a *Publisher* sends a `PUBLISH` message with `payload passthru` to a *Broker* that supports this feature, which then must be routed to a *Subscriber* which doesn't support `payload passthru`, cannot be recognized at the protocol level due to asynchronous message processing and must be covered at the *Subscriber* side.

Cases where a *Callee* sends a `YIELD` message with `payload passthru` without announcing it during the `HELLO` handshake MUST be treated as *PROTOCOL ERRORS* and the underlying WAMP connections must be aborted with the `wamp.error.protocol_violation` error reason.

Cases where a *Callee* sends a `YIELD` message with `payload passthru` to a *Dealer*, with the latter not announcing `payload passthru` support during the `WELCOME` handshake, MUST be treated as *PROTOCOL ERRORS* and the underlying WAMP connections must be aborted with the `wamp.error.protocol_violation` error reason.

Cases where a *Callee* sends a `YIELD` message with `payload passthru` to a *Dealer* that supports this feature, which then must be routed to the *Caller* which doesn't support `payload passthru`, MUST be treated as *APPLICATION ERRORS* and the *Dealer* MUST respond to the *Callee* with a `wamp.error.feature_not_supported` error message.

Message Attributes

To use payload passthru mode, the options for `CALL`, `PUBLISH` and `YIELD` messages MUST be extended with additional attributes. These additional attributes must be forwarded via `INVOCATION`, `EVENT` and `RESULT` messages, respectively, as well as `ERROR` messages in the case of failures.

```
    CALL.Options.ppt_scheme|string
    CALL.Options.ppt_serializer|string
    CALL.Options.ppt_cipher|string
    CALL.Options.ppt_keyid|string
    ---
    INVOCATION.Details.ppt_scheme|string
    INVOCATION.Details.ppt_serializer|string
    INVOCATION.Details.ppt_cipher|string
    INVOCATION.Details.ppt_keyid|string
    ---
    YIELD.Options.ppt_scheme|string
    YIELD.Options.ppt_serializer|string
    YIELD.Options.ppt_cipher|string
    YIELD.Options.ppt_keyid|string
    ---
    RESULT.Details.ppt_scheme|string
    RESULT.Details.ppt_serializer|string
    RESULT.Details.ppt_cipher|string
    RESULT.Details.ppt_keyid|string
    ---
    ERROR.Details.ppt_scheme|string
    ERROR.Details.ppt_serializer|string
    ERROR.Details.ppt_cipher|string
    ERROR.Details.ppt_keyid|string
```

```
    PUBLISH.Options.ppt_scheme|string
    PUBLISH.Options.ppt_serializer|string
    PUBLISH.Options.ppt_cipher|string
    PUBLISH.Options.ppt_keyid|string
    ---
    EVENT.Details.ppt_scheme|string
    EVENT.Details.ppt_serializer|string
    EVENT.Details.ppt_cipher|string
    EVENT.Details.ppt_keyid|string
    ---
    ERROR.Options.ppt_scheme|string
    ERROR.Options.ppt_serializer|string
    ERROR.Options.ppt_cipher|string
    ERROR.Options.ppt_keyid|string
```

ppt_scheme Attribute

The `ppt_scheme` identifies the Payload Schema. It is a required string attribute. For End-2-End Encryption flow this attribute can contain the name or identifier of a key management provider that is known to the target peer, so it can be used with help of additional `ppt_*` attributes to obtain information about encryption keys. For gateways and external schemas this can contain the name of related technology. The one predefined is `mqtt`. Others may be introduced later. A *Router* can recognize that `Payload Passthru Mode` is in use by checking the existence and non-empty value of this attribute within the options of `CALL`, `PUBLISH` and `YIELD` messages.

ppt_serializer Attribute

The `ppt_serializer` attribute is optional. It specifies what serializer was used to encode the payload. It can be a `native` value to indicate that the incoming data is tunneling through other technologies specified by the `ppt_scheme`, or it can be ordinary `json`, `msgpack`, `cbor`, `flatbuffers` data serializers. For some predefined `ppt_scheme` schemas this option may be omitted as schema defines the concrete serializer. See predefined schemas below.

ppt_cipher Attribute

The `ppt_cipher` attribute is optional. It is required if the payload is encrypted. This attribute specifies the cryptographic algorithm that was used to encrypt the payload. It can be `xsalsa20poly1305`, `aes256gcm` for now.

ppt_keyid Attribute

The `ppt_keyid` attribute is optional. This attribute can contain the encryption key id that was used to encrypt the payload. The `ppt_keyid` attribute is a string type. The value can be a hex-encoded string, URI, DNS name, Ethereum address, UUID identifier - any meaningful value which allows the target peer to choose a private key without guessing. The format of the value may depend on the `ppt_scheme` attribute.

ppt_ Predefined Schemes

MQTT Predefined Scheme

Attribute	Required?	Value
ppt_scheme	Y	mqtt
ppt_serializer	N*	native, json, msgpack, cbor
ppt_cipher	N	-
ppt_keyid	N	-

Table 15

*: If `ppt_serializer` is not provided then it is assuming as `native`. So no additional serialization will be applied to payload and payload will be serialized within WAMP message with session serializer.

End-to-End Encryption Predefined Scheme

For `End-to-End Encryption` flow both peers must support chosen `ppt_serializer` regardless of their own session serializer.

Attribute	Required?	Value
ppt_scheme	Y	wamp
ppt_serializer	Y	cbor, flatbuffers
ppt_cipher	N	xsalsa20poly1305, aes256gcm
ppt_keyid	N	*

Table 16

*: The least significant 20 bytes (160 bits) of the SHA256 of the public key (32 bytes) of the data encryption key, as a hex-encoded string with prefix `0x` and either uppercase/lowercase alphabetic characters, encoding a checksum according to EIP55.

Custom Scheme Example

Attribute	Required?	Value
ppt_scheme	Y	x_my_ppt
ppt_serializer	N	custom
ppt_cipher	N	custom
ppt_keyid	N	custom

Table 17

When `Payload Passthru Mode` is used for gateways to other technologies, such as MQTT Brokers, then the `ppt_serializer` attribute may be set to the `native` value. This means that the payload is not to be modified by WAMP peers, nor serialized in any manner, and is delivered as-is from the originating peer. Another possible case is when the `ppt_serializer`

attribute is set to any valid serializer, for example `msgpack`. In this case the originating WAMP client peer first applies `ppt_serializer` to serialize the payload (without encryption), then the resulting binary payload is embedded in the WAMP message, the latter having possibly a different serializer depending on the one chosen during WAMP Session establishment.

Important Note Regarding JSON Serialization

With `Payload Passthru Mode`, payloads are treated as binary. To send these binary payloads, the WAMP session serializer MUST support byte arrays. Most serialization formats known to WAMP support byte arrays, but JSON does not support them natively. To use `Payload Passthru Mode` with a JSON serializer, WAMP peers MUST perform the special Binary serialization in JSON. This conversion may have unacceptable overhead, so it is generally advised to use WAMP session serializers with native byte array support, for example, `MessagePack`, `CBOR`, or `FlatBuffers`.

Message Structure

When `Payload Passthru Mode` is in use, the message payload MUST be sent as one binary item within `Arguments|list`, while `ArgumentsKw|dict` MUST be absent or empty.

Since many WAMP messages assume the possibility of simultaneous use of `Arguments|list` and `ArgumentsKw|dict`, WAMP client implementations must package arguments into the following hash table and then serialize it and transmit as a single element within `Arguments|list`.

```
{
    "args": Arguments|list,
    "kwargs": ArgumentsKw|dict
}
```

This will allow maintaining a single interface for client applications, regardless of whether the `Payload Passthru Mode` mode, or especially `Payload End-to-End Encryption` which is built on top of `Payload End-to-End Encryption` is used or not.

Example. Caller-to-Dealer `CALL` with encryption and key ID

```
    [
        48,
        25471,
        {
            "ppt_scheme": "wamp",
            "ppt_serializer": "cbor",
            "ppt_cipher": "xsalsa20poly1305",
            "ppt_keyid": "GTtQ37XGJO2O4R8Dvx4AUo8pe61D9evIWpKGQAPdOh0="
        },
        "com.myapp.secret_rpc_for_sensitive_data",
        [Payload|binary]
    ]
```

Example. Caller-to-Dealer progressive `CALL` with encryption and key ID.

Note that nothing prevents the use of `Payload Passthru Mode` with other features such as, for example, `Progressive Call Results` or `Progressive Call Invocations`.

```
[
    48,
    25471,
    {
        "ppt_scheme": "wamp",
        "ppt_serializer": "flatbuffers",
        "ppt_cipher": "xsalsa20poly1305",
        "ppt_keyid": "GTtQ37XGJO2O4R8Dvx4AUo8pe61D9evIWpKGQAPdOh0=",
        "progress": true
    },
    "com.myapp.progressive_rpc_for_sensitive_data",
    [Payload|binary]
]
```

Example. Caller-to-Dealer `CALL` with MQTT payload. Specifying `"ppt_serializer": "native"` means that the original MQTT message payload is passed as WAMP payload message as is, without any transcoding.

```
[
    48,
    25471,
    {
        "ppt_scheme": "mqtt",
        "ppt_serializer": "native"
    },
    "com.myapp.mqtt_processing",
    [Payload|binary]
]
```

Example. Caller-to-Dealer `CALL` with MQTT payload. Specifying `"ppt_scheme": "mqtt"` simply indicates that the original source of payload data is received from a related system. Specifying `"ppt_serializer": "json"` means that the original MQTT message payload was parsed and encoded with the `json` serializer before embedding it into WAMP message.

```
[
    48,
    25471,
    {
        "ppt_scheme": "mqtt",
        "ppt_serializer": "json"
    },
    "com.myapp.mqtt_processing",
    [Payload|binary]
]
```

Example. Dealer-to-Callee `INVOCATION` with encryption and key ID

```
[
    68,
    35477,
    1147,
    {
        "ppt_scheme": "wamp",
        "ppt_serializer": "cbor",
        "ppt_cipher": "xsalsa20poly1305",
        "ppt_keyid": "GTtQ37XGJO2O4R8Dvx4AUo8pe61D9evIWpKGQAPdOh0="
    },
    [Payload|binary]
]
```

Example. Dealer-to-Callee `INVOCATION` with MQTT payload

```
[
    68,
    35479,
    3344,
    {
        "ppt_scheme": "mqtt",
        "ppt_serializer": "native"
    },
    [Payload|binary]
]
```

Example. Callee-to-Dealer `YIELD` with encryption and key ID

```
[
    70,
    87683,
    {
        "ppt_scheme": "wamp",
        "ppt_serializer": "flatbuffers",
        "ppt_cipher": "xsalsa20poly1305",
        "ppt_keyid": "GTtQ37XGJO2O4R8Dvx4AUo8pe61D9evNSpGMDQWdOh1="
    },
    [Payload|binary]
]
```

Example. Callee-to-Dealer progressive `YIELD` with encryption and key ID

Nothing prevents the use of `Payload Passthru Mode` with other features such as, for example, `Progressive Call Results`.

```
[
    70,
    87683,
    {
        "ppt_scheme": "wamp",
        "ppt_serializer": "flatbuffers",
        "ppt_cipher": "xsalsa20poly1305",
        "ppt_keyid": "GTtQ37XGJO2O4R8Dvx4AUo8pe61D9evNSpGMDQWdOh1=",
        "progress": true
    },
    [Payload|binary]
]
```

Example. Dealer-to-Caller `RESULT` with encryption and key ID

```
[
    50,
    77133,
    {
        "ppt_scheme": "wamp",
        "ppt_serializer": "flatbuffers",
        "ppt_cipher": "xsalsa20poly1305",
        "ppt_keyid": "GTtQ37XGJO2O4R8Dvx4AUo8pe61D9evNSpGMDQWdOh1="
    },
    [Payload|binary]
]
```

Example. Dealer-to-Caller progressive `RESULT` with encryption and key ID

Nothing prevents the use of `Payload Passthru Mode` with other features such as, for example, `Progressive Call Results`.

```
[
    50,
    77133,
    {
        "ppt_scheme": "wamp",
        "ppt_serializer": "flatbuffers",
        "ppt_cipher": "xsalsa20poly1305",
        "ppt_keyid": "GTtQ37XGJO2O4R8Dvx4AUo8pe61D9evNSpGMDQWdOh1=",
        "progress": true
    },
    [Payload|binary]
]
```

Example. Callee-to-Dealer `ERROR` with encryption and key ID

```
[
    8,
    68,
    87683,
    {
        "ppt_scheme": "wamp",
        "ppt_serializer": "cbor",
        "ppt_cipher": "xsalsa20poly1305",
        "ppt_keyid": "GTtQ37XGJO2O4R8Dvx4AUo8pe61D9evNSpGMDQWdOh1="
    },
    "com.myapp.invalid_revenue_year",
    [Payload|binary]
]
```

Example. Publishing event to a topic with encryption and key ID

```
[
    16,
    45677,
    {
        "ppt_scheme": "wamp",
        "ppt_serializer": "cbor",
        "ppt_cipher": "xsalsa20poly1305",
        "ppt_keyid": "GTtQ37XGJO2O4R8Dvx4AUo8pe61D9evNSpGMDQWdOh1="
    },
    "com.myapp.mytopic1",
    [Payload|binary]
]
```

Example. Receiving event for a topic with encryption and key ID

```
[
    36,
    5512315355,
    4429313566,
    {
        "ppt_scheme": "wamp",
        "ppt_serializer": "flatbuffers",
        "ppt_cipher": "xsalsa20poly1305",
        "ppt_keyid": "GTtQ37XGJO2O4R8Dvx4AUo8pe61D9evNSpGMDQWdOh1="
    },
    [Payload|binary]
]
```

About Supported Serializers and Cryptographic Ciphers

WAMP serves as infrastructure for delivering messages between peers. Regardless of what encryption algorithm and serializer were chosen for `Payload Passthru Mode`, a *Router* shall not inspect and analyze the `ppt_` options and payload of encrypted messages. The application is responsible for choosing serializers and ciphers known to every peer involved in message processing.

14.2. Payload End-to-End Encryption

TBD

15. Advanced Transports and Serializers

The only requirements that WAMP expects from a transport are: the transport must be message-based, bidirectional, reliable and ordered. This allows WAMP to run over different transports without any impact at the application layer.

Besides the WebSocket transport, the following WAMP transports are currently specified:

- RawSocket Transport
- Batched WebSocket Transport
- LongPoll Transport
- Multiplexed Transport

> Other transports such as HTTP 2.0 ("SPDY") or UDP might be defined in the future.

15.1. RawSocket Transport

WAMP-over-RawSocket is an (alternative) transport for WAMP that uses length-prefixed, binary messages - a message framing different from WebSocket.

Compared to WAMP-over-WebSocket, WAMP-over-RawSocket is simple to implement, since there is no need to implement the WebSocket protocol which has some features that make it non-trivial (like a full HTTP-based opening handshake, message fragmentation, masking and variable length integers).

WAMP-over-RawSocket has even lower overhead than WebSocket, which can be desirable in particular when running on local connections like loopback TCP or Unix domain sockets. It is also expected to allow implementations in microcontrollers in under 2KB RAM.

WAMP-over-RawSocket can run over TCP, TLS, Unix domain sockets or any reliable streaming underlying transport. When run over TLS on the standard port for secure HTTPS (443), it is also able to traverse most locked down networking environments such as enterprise or mobile networks (unless man-in-the-middle TLS intercepting proxies are in use).

However, WAMP-over-RawSocket cannot be used with Web browser clients, since browsers do not allow raw TCP connections. Browser extensions would do, but those need to be installed in a browser. WAMP-over-RawSocket also (currently) does not support transport-level compression as WebSocket does provide (`permessage-deflate` WebSocket extension).

Endianess

WAMP-over-RawSocket uses *network byte order* ("big-endian"). That means, given a unsigned 32 bit integer

```
    0x 11 22 33 44
```

the first octet sent out to (or received from) the wire is `0x11` and the last octet sent out (or received) is `0x44`.

Here is how you would convert octets received from the wire into an integer in Python:

```
import struct

octets_received = b"\x11\x22\x33\x44"
i = struct.unpack(">L", octets_received)[0]
```

The integer received has the value `287454020`.

And here is how you would send out an integer to the wire in Python:

```
octets_to_be_send = struct.pack(">L", i)
```

The octets to be sent are `b"\x11\x22\x33\x44"`.

Handshake: Client-to-Router Request

WAMP-over-RawSocket starts with a handshake where the client connecting to a router sends 4 octets:

```
    MSB                                 LSB
    31                                    0
    0111 1111 LLLL SSSS RRRR RRRR RRRR RRRR
```

The *first octet* is a magic octet with value `0x7F`. This value is chosen to avoid any possible collision with the first octet of a valid HTTP request (see here and here). No valid HTTP request can have `0x7F` as its first octet.

> By using a magic first octet that cannot appear in a regular HTTP request, WAMP-over-RawSocket can be run e.g. on the same TCP listening port as WAMP-over-WebSocket or WAMP-over-LongPoll.

The *second octet* consists of a 4 bit `LENGTH` field and a 4 bit `SERIALIZER` field.

The `LENGTH` value is used by the *Client* to signal the **maximum message length** of messages it is willing to **receive**. When the handshake completes successfully, a *Router* MUST NOT send messages larger than this size.

The possible values for `LENGTH` are:

```
     0: 2**9 octets
     1: 2**10 octets
    ...
    14: 2**23 octets
    15: 2**24 octets
```

This means a *Client* can choose the maximum message length between **512** and **16M** octets.

The `SERIALIZER` value is used by the *Client* to request a specific serializer to be used. When the handshake completes successfully, the *Client* and *Router* will use the serializer requested by the *Client*.

The possible values for `SERIALIZER` are:

```
    0: illegal
    1: JSON
    2: MessagePack
    3: CBOR
    4: UBJSON
    5: FlatBuffers
    6 - 15: reserved for future serializers
```

Here is a Python program that prints all (currently) permissible values for the *second octet*:

```
SERMAP = {
    1: 'json',
    2: 'msgpack',
    3: 'cbor',
    4: 'ubjson',
    5: 'flatbuffers',
}

# map serializer / max. msg length to RawSocket handshake
# request or success reply (2nd octet)
for ser in SERMAP:
    for l in range(16):
        octet_2 = (l << 4) | ser
        print("serializer: {}, maxlen: {} => 0x{:02x}".format(SERMAP[ser], 2
** (l + 9), octet_2))
```

The *third and forth octet* are **reserved** and MUST be all zeros for now.

Handshake: Router-to-Client Reply

After a *Client* has connected to a *Router*, the *Router* will first receive the 4 octets handshake request from the *Client*.

If the *first octet* differs from `0x7F`, it is not a WAMP-over-RawSocket request. Unless the *Router* also supports other transports on the connecting port (such as WebSocket or LongPoll), the *Router* MUST **fail the connection**.

Here is an example of how a *Router* could parse the *second octet* in a *Clients* handshake request:

```
# map RawSocket handshake request (2nd octet) to
# serializer / max. msg length
for i in range(256):
    ser_id = i & 0x0f
    if ser_id != 0:
        ser = SERMAP.get(ser_id, 'currently undefined')
        maxlen = 2 ** ((i >> 4) + 9)
        print("{:02x} => serializer: {}, maxlen: {}".format(i, ser, maxlen))
    else:
        print("fail the connection: illegal serializer value")
```

When the *Router* is willing to speak the serializer requested by the *Client*, it will answer with a 4 octets response of identical structure as the *Client* request:

```
    MSB                                 LSB
    31                                    0
    0111 1111 LLLL SSSS RRRR RRRR RRRR RRRR
```

Again, the *first octet* MUST be the value `0x7F`. The *third and forth octets* are reserved and MUST be all zeros for now.

In the *second octet*, the *Router* MUST echo the serializer value in `SERIALIZER` as requested by the *Client*.

Similar to the *Client*, the *Router* sets the `LENGTH` field to request a limit on the length of messages sent by the *Client*.

During the connection, *Router* MUST NOT send messages to the *Client* longer than the `LENGTH` requested by the *Client*, and the *Client* MUST NOT send messages larger than the maximum requested by the *Router* in its handshake reply.

If a message received during a connection exceeds the limit requested, a *Peer* MUST **fail the connection**.

When the *Router* is unable to speak the serializer requested by the *Client*, or it is denying the *Client* for other reasons, the *Router* replies with an error:

```
    MSB                                 LSB
    31                                    0
    0111 1111 EEEE 0000 RRRR RRRR RRRR RRRR
```

An error reply has 4 octets: the *first octet* is again the magic `0x7F`, and the *third and forth octet* are reserved and MUST all be zeros for now.

The *second octet* has its lower 4 bits zero'ed (which distinguishes the reply from an success/accepting reply) and the upper 4 bits encode the error:

```
    0: illegal (must not be used)
    1: serializer unsupported
    2: maximum message length unacceptable
    3: use of reserved bits (unsupported feature)
    4: maximum connection count reached
    5 - 15: reserved for future errors
```

> Note that the error code `0` MUST NOT be used. This is to allow storage of error state in a host language variable, while allowing `0` to signal the current state "no error"

Here is an example of how a *Router* might create the *second octet* in an error response:

```
ERRMAP = {
    0: "illegal (must not be used)",
    1: "serializer unsupported",
    2: "maximum message length unacceptable",
    3: "use of reserved bits (unsupported feature)",
    4: "maximum connection count reached"
}

# map error to RawSocket handshake error reply (2nd octet)
for err in ERRMAP:
    octet_2 = err << 4
    print("error: {} => 0x{:02x}").format(ERRMAP[err], err)
```

The *Client* - after having sent its handshake request - will wait for the 4 octets from *Router* handshake reply.

Here is an example of how a *Client* might parse the *second octet* in a *Router* handshake reply:

```
# map RawSocket handshake reply (2nd octet)
for i in range(256):
    ser_id = i & 0x0f
    if ser_id:
        # verify the serializer is the one we requested!
        # if not, fail the connection!
        ser = SERMAP.get(ser_id, 'currently undefined')
        maxlen = 2 ** ((i >> 4) + 9)
        print("{:02x} => serializer: {}, maxlen: {}".format(i, ser, maxlen))
    else:
        err = i >> 4
        print("error: {}".format(ERRMAP.get(err, 'currently undefined')))
```

Serialization

To send a WAMP message, the message is serialized according to the WAMP serializer agreed in the handshake (e.g. JSON, MessagePack, CBOR, UBJSON or FlatBuffers).

The length of the serialized messages in octets MUST NOT exceed the maximum requested by the *Peer*.

If the serialized length exceed the maximum requested, the WAMP message can not be sent to the *Peer*. Handling situations like the latter is left to the implementation.

E.g. a *Router* that is to forward a WAMP `EVENT` to a *Client* which exceeds the maximum length requested by the *Client* when serialized might:

- drop the event (not forwarding to that specific client) and track dropped events
- prohibit publishing to the topic already
- remove the event payload, and send an event with extra information (`payload_limit_exceeded = true`)

Framing

The serialized octets for a message to be sent are prefixed with exactly 4 octets.

```
    MSB                                 LSB
    31                                    0
    RRRR XTTT LLLL LLLL LLLL LLLL LLLL LLLL
```

The *three least significant octets* constitute an unsigned 24 bit integer that provides the length of transport message payload following, excluding the 4 octets that constitute the prefix.

The *most significant octet* has the following structure

```
    MSB   LSB
    7       0
    RRRR XTTT
```

The four bits `RRRR` are reserved for future use and MUST be all zeros for now.

`X` is an extra (25th) bit used to encode the message payload length. This bit is only set when the payload length is exactly 16M octets, with the 24 remaining `L` bits cleared:

```
    MSB                                 LSB
    31                                    0
    RRRR 1TTT 0000 0000 0000 0000 0000 0000
```

When the payload length is 16M-1 octets, for example, the prefix would be:

```
    MSB                                 LSB
    31                                    0
    RRRR 0TTT 1111 1111 1111 1111 1111 1111
```

The three bits `TTT` encode the type of the transport message:

```
0: regular WAMP message
1: PING
2: PONG
3-7: reserved
```

For a regular WAMP message (`TTT == 0`), the length is the length of the serialized WAMP message: the number of octets after serialization (excluding the 4 octets of the prefix).

For a `PING` message (`TTT == 1`), the length is the length of the arbitrary payload that follows. A *Peer* MUST reply to each `PING` by sending exactly one `PONG` immediately, and the `PONG` MUST echo back the payload of the `PING` exactly.

For receiving messages with WAMP-over-RawSocket, a *Peer* will usually read exactly 4 octets from the incoming stream, decode the transport level message type and payload length, and then receive as many octets as the length was giving.

When the transport level message type indicates a regular WAMP message, the transport level message payload is unserialized according to the serializer agreed in the handshake and the processed at the WAMP level.

15.2. Message Batching

WAMP-over-Batched-WebSocket is a variant of WAMP-over-WebSocket where multiple WAMP messages are sent in one WebSocket message.

Using WAMP message batching can increase wire level efficiency further. In particular when using TLS and the WebSocket implementation is forcing every WebSocket message into a new TLS segment.

WAMP-over-Batched-WebSocket is negotiated between Peers in the WebSocket opening handshake by agreeing on one of the following WebSocket subprotocols:

- `wamp.2.json.batched`
- `wamp.2.msgpack.batched`
- `wamp.2.cbor.batched`

Batching with JSON works by serializing each WAMP message to JSON as normally, appending the single ASCII control character `\30` (record separator) octet `0x1e` to *each* serialized messages, and packing a sequence of such serialized messages into a single WebSocket message:

```
Serialized JSON WAMP Msg 1 | 0x1e |
    Serialized JSON WAMP Msg 2 | 0x1e | ...
```

Batching with MessagePack works by serializing each WAMP message to MessagePack as normally, prepending a 32 bit unsigned integer (4 octets in big-endian byte order) with the length of the serialized MessagePack message (excluding the 4 octets for the length prefix), and packing a sequence of such serialized (length-prefixed) messages into a single WebSocket message:

```
    Length of Msg 1 serialization (uint32) |
        serialized MessagePack WAMP Msg 1 | ...
```

With batched transport, even if only a single WAMP message is to be sent in a WebSocket message, the (single) WAMP message needs to be framed as described above. In other words, a single WAMP message is sent as a batch of length **1**. Sending a batch of length **0** (no WAMP message) is illegal and a *Peer* MUST fail the transport upon receiving such a transport message.

15.3. HTTP Longpoll Transport

The *Long-Poll Transport* is able to transmit a WAMP session over plain old HTTP 1.0/1.1. This is realized by the Client issuing HTTP/POSTs requests, one for sending, and one for receiving. Those latter requests are kept open at the server when there are no messages currently pending to be received.

Opening a Session

With the Long-Poll Transport, a Client opens a new WAMP session by sending a HTTP/POST request to a well-known URL, e.g.

```
    http://mypp.com/longpoll/open
```

Here, `http://mypp.com/longpoll` is the base URL for the Long-Poll Transport and `/open` is a path dedicated for opening new sessions.

The HTTP/POST request SHOULD have a `Content-Type` header set to `application/json` and MUST have a request body with a JSON document that is a dictionary:

```
    {
       "protocols": ["wamp.2.json"]
    }
```

The (mandatory) `protocols` attribute specifies the protocols the client is willing to speak. The server will chose one from this list when establishing the session or fail the request when no protocol overlap was found.

The valid protocols are:

- `wamp.2.json.batched`
- `wamp.2.json`
- `wamp.2.msgpack.batched`
- `wamp.2.msgpack`
- `wamp.2.cbor.batched`
- `wamp.2.cbor`

> The request path with this and subsequently described HTTP/POST requests MAY contain a query parameter `x` with some random or sequentially incremented value:
>
> http://mypp.com/longpoll/open?x=382913
>
> The value is ignored, but may help in certain situations to prevent intermediaries from caching the request.

Returned is a JSON document containing a transport ID and the protocol to speak:

```
{
   "protocol": "wamp.2.json",
   "transport": "kjmd3sBLOUnb3Fyr"
}
```

As an implied side-effect, two HTTP endpoints are created

```
http://mypp.com/longpoll/<transport_id>/receive
http://mypp.com/longpoll/<transport_id>/send
```

where `transport_id` is the transport ID returned from `open`, e.g.

```
http://mypp.com/longpoll/kjmd3sBLOUnb3Fyr/receive
http://mypp.com/longpoll/kjmd3sBLOUnb3Fyr/send
```

Receiving WAMP Messages

The Client will then issue HTTP/POST requests (with empty request body) to

```
http://mypp.com/longpoll/kjmd3sBLOUnb3Fyr/receive
```

When there are WAMP messages pending downstream, a request will return with a single WAMP message (unbatched modes) or a batch of serialized WAMP messages (batched mode).

The serialization format used is the one agreed during opening the session.

The batching uses the same scheme as with `wamp.2.json.batched` and `wamp.2.msgpack.batched` transport over WebSocket.

> Note: In unbatched mode, when there is more than one message pending, there will be at most one message returned for each request. The other pending messages must be retrieved by new requests. With batched mode, all messages pending at request time will be returned in one batch of messages.

Sending WAMP Messages

For sending WAMP messages, the *Client* will issue HTTP/POST requests to

```
http://mypp.com/longpoll/kjmd3sBLOUnb3Fyr/send
```

with request body being a single WAMP message (unbatched modes) or a batch of serialized WAMP messages (batched mode).

The serialization format used is the one agreed during opening the session.

The batching uses the same scheme as with `wamp.2.json.batched` and `wamp.2.msgpack.batched` transport over WebSocket.

Upon success, the request will return with HTTP status code 202 ("no content"). Upon error, the request will return with HTTP status code 400 ("bad request").

Closing a Session

To orderly close a session, a Client will issue a HTTP/POST to

```
http://mypp.com/longpoll/kjmd3sBLOUnb3Fyr/close
```

with an empty request body. Upon success, the request will return with HTTP status code 202 ("no content").

15.4. Binary support in JSON

Binary data follows a convention for conversion to JSON strings.

A byte array is converted to a JSON string as follows:

1. convert the byte array to a Base64 encoded (host language) string
2. prepend the string with a `\0` character
3. serialize the string to a JSON string

where Base64 encoding follows Section 4 of [RFC4648].

Example

Consider the byte array (hex representation):

```
10e3ff9053075c526f5fc06d4fe37cdb
```

This will get converted to Base64

```
EOP/kFMHXFJvX8BtT+N82w==
```

prepended with `\0`

```
    \x00EOP/kFMHXFJvX8BtT+N82w==
```

and serialized to a JSON string

```
    "\\u0000EOP/kFMHXFJvX8BtT+N82w=="
```

A JSON string is unserialized to either a string or a byte array using the following procedure:

1. Unserialize a JSON string to a host language (Unicode) string
2. If the string starts with a `\0` character, interpret the rest (after the first character) as Base64 and decode to a byte array
3. Otherwise, return the Unicode string

Below are complete Python and JavaScript code examples for conversion between byte arrays and JSON strings.

Python

Here is a complete example in Python showing how byte arrays are converted to and from JSON:

```
import os, base64, json, sys, binascii

data_in = os.urandom(16)
print("In:   {}".format(binascii.hexlify(data_in)))

# encoding
encoded = json.dumps('\0' + base64.b64encode(data_in).
                                    decode('ascii'))

print("JSON: {}".format(encoded))

# decoding
decoded = json.loads(encoded)
if type(decoded) == unicode:
   if decoded[0] == '\0':
      data_out = base64.b64decode(decoded[1:])
   else:
      data_out = decoded

print("Out:  {}".format(binascii.hexlify(data_out)))

assert(data_out == data_in)
```

JavaScript

Here is a complete example in JavaScript showing how byte arrays are converted to and from JSON:

```
var data_in = new Uint8Array(new ArrayBuffer(16));

// initialize test data
for (var i = 0; i < data_in.length; ++i) {
   data_in[i] = i;
}
console.log(data_in);

// convert byte array to raw string
var raw_out = '';
for (var i = 0; i < data_in.length; ++i) {
   raw_out += String.fromCharCode(data_in[i]);
}

// base64 encode raw string, prepend with \0
// and serialize to JSON
var encoded = JSON.stringify("\0" + window.btoa(raw_out));
console.log(encoded); // "\u0000AAECAwQFBgcICQoLDA0ODw=="

// unserialize from JSON
var decoded = JSON.parse(encoded);

var data_out;
if (decoded.charCodeAt(0) === 0) {
   // strip first character and decode base64 to raw string
   var raw = window.atob(decoded.substring(1));

   // convert raw string to byte array
   var data_out = new Uint8Array(new ArrayBuffer(raw.length));
   for (var i = 0; i < raw.length; ++i) {
      data_out[i] = raw.charCodeAt(i);
   }
} else {
   data_out = decoded;
}

console.log(data_out);
```

15.5. Multiplexed Transport

A Transport may support the multiplexing of multiple logical transports over a single "physical" transport.

By using such a Transport, multiple WAMP sessions can be transported over a single underlying transport at the same time.

As an example, the proposed WebSocket extension "permessage-priority" would allow creating multiple logical Transports for WAMP over a single underlying WebSocket connection.

Sessions running over a multiplexed Transport are completely independent: they get assigned different session IDs, may join different realms and each session needs to authenticate itself.

Because of above, Multiplexed Transports for WAMP are actually not detailed in the WAMP spec, but a feature of the transport being used.

> Note: Currently no WAMP transport supports multiplexing. The work on the MUX extension with WebSocket has stalled, and the `permessage-priority` proposal above is still just a proposal. However, with RawSocket, we should be able to add multiplexing in the the future (with downward compatibility).

16. WAMP Interfaces

WAMP was designed with the goals of being easy to approach and use for application developers. Creating a procedure to expose some custom functionality should be possible in any supported programming language using that language's native elements, with the least amount of additional effort.

Following from that, WAMP uses *dynamic typing* for the application payloads of calls, call results and error, as well as event payloads.

A WAMP router will happily forward *any* application payload on *any* procedure or topic URI as long as the client is *authorized* (has permission) to execute the respective WAMP action (call, register, publish or subscribe) on the given URI.

This approach has served WAMP well, as application developers can get started immediately, and evolve and change payloads as they need without extra steps. These advantages in flexibility of course come at a price, as nothing is free, and knowing that price is important to be aware of the tradeoffs one is accepting when using dynamic typing:

- problematic coordination of *Interfaces* within larger developer teams or between different parties
- no easy way to stabilize, freeze, document or share *Interfaces*
- no way to programmatically describe *Interfaces* ("interface reflection") at run-time

Problems such above could be avoided when WAMP supported an *option* to formally define WAMP-based *Interfaces*. This needs to answer the following questions:

1. How to specify the `args|List` and `kwargs|Dict` application payloads that are used in WAMP calls, errors and events?
2. How to specify the type and URI (patterns) for WAMP RPCs *Procedures* and WAMP PubSub *Topics* that make up an *Interface*, and how to identify an *Interface* itself as a collection of *Procedures* and *Topics*?
3. How to package, publish and share *Catalogs* as a collection of *Interfaces* plus metadata

The following sections will describe the solution to each of above questions using WAMP IDL.

Using WAMP Interfaces finally allows to support the following application developer level features:

1. router-based application payload validation and enforcement
2. WAMP interface documentation generation and autodocs Web service

3. publication and sharing of WAMP Interfaces and Catalogs
4. client binding code generation from WAMP Interfaces
5. run-time WAMP type reflection as part of the WAMP meta API

16.1. WAMP IDL

16.1.1. Application Payload Typing

To define the application payload `Arguments|list` and `ArgumentsKw|dict`, WAMP IDL reuses the FlatBuffers IDL, specifically, we map a pair of `Arguments|list` and `ArgumentsKw|dict` to a FlatBuffers Table with WAMP defined FlatBuffers *Attributes*.

User defined WAMP application payloads are transmitted in `Arguments|list` and `ArgumentsKw|dict` elements of the following WAMP messages:

- `PUBLISH`
- `EVENT`
- `CALL`
- `INVOCATION`
- `YIELD`
- `RESULT`
- `ERROR`

A *Publisher* uses the

- `PUBLISH.Arguments|list` and `PUBLISH.ArgumentsKw|dict`

message elements to send the event payload to be published to the *Broker* in `PUBLISH` messages. When the event is accepted by the *Broker*, it will dispatch an `EVENT` message with

- `EVENT.Arguments|list` and `EVENT.ArgumentsKw|dict`

message elements to all (eligible, and not excluded) *Subscribers*.

A *Caller* uses the

- `CALL.Arguments|list` and `CALL.ArgumentsKw|dict`

message elements to send the call arguments to be used to the *Dealer* in `CALL` messages. When the call is accepted by the *Dealer*, it will forward

- `INVOCATION.Arguments|list` and `INVOCATION.ArgumentsKw|dict`

to the (or one of) *Callee*, and receive `YIELD` messages with

- `YIELD.Arguments|list` and `YIELD.ArgumentsKw|dict`

message elements, which it will return to the original *Caller* in `RESULT` messages with

- `RESULT.Arguments|list` and `RESULT.ArgumentsKw|dict`

In the error case, a *Callee* MAY return an `ERROR` message with

- `ERROR.Arguments|list` and `ERROR.ArgumentsKw|dict`

message elements, which again is returned to the original *Caller*.

> It is important to note that the above messages and message elements are the only ones free for use with application and user defined payloads. In particular, even though the following WAMP messages and message element carry payloads defined by the specific WAMP authentication method used, they do *not* carry arbitrary application payloads: `HELLO.Details["authextra"]|dict`, `WELCOME.Details["authextra"]|dict`, `CHALLENGE.Extra|dict`, `AUTHENTICATE.Extra|dict`.

For example, the Session Meta API includes a procedure to kill all sessions by authid with:

Positional arguments (`args|list`)

1. `authid|string` - The authentication ID identifying sessions to close.

Keyword arguments (`kwargs|dict`)

1. `reason|uri` - reason for closing sessions, sent to clients in `GOODBYE.Reason`
2. `message|string` - additional information sent to clients in `GOODBYE.Details` under the key "message".

as arguments. When successful, this procedure will return a call result with:

Positional results (`results|list`)

1. `sessions|list` - The list of WAMP session IDs of session that were killed.

Keyword results (`kwresults|dict`)

1. `None`

To specify the call arguments in FlatBuffers IDL, we can define a FlatBuffers table for both `args` and `kwargs`:

```
/// Call args/kwargs for "wamp.session.kill_by_authid"
table SessionKillByAuthid
{
    /// The WAMP authid of the sessions to kill.
    authid: string (wampuri);

    /// A reason URI provided to the killed session(s).
    reason: string (kwarg, wampuri);

    /// A message provided to the killed session(s).
    message: string (kwarg);
}
```

The table contains the list `args` as table elements (in order), unless the table element has an *Attribute* `kwarg`, in which case the element one in `kwarg`.

The attributes `wampid` and `wampuri` are special markers that denote values that follow the respective WAMP identifier rules for WAMP IDs and URIs.

When successful, the procedure will return a list of WAMP session IDs of session that were killed. Again, we can map this to FlatBuffers IDL:

```
table WampIds
{
    /// List of WAMP IDs.
    value: [uint64] (wampid);
}
```

16.1.2. WAMP IDL Attributes

WAMP IDL uses *custom FlatBuffer attributes* to

- mark `kwarg` fields which map to WAMP keyword argument vs `arg` (default)
- declare fields of a scalar base type to follow (stricter) WAMP rules (for IDs and URIs)
- specify the WAMP action type, that is *Procedure* vs *Topic*, on service declarations

> "Attributes may be attached to a declaration, behind a field, or after the name of a table/struct/enum/union. These may either have a value or not. Some attributes like deprecated are understood by the compiler; user defined ones need to be declared with the attribute declaration (like priority in the example above), and are available to query if you parse the schema at runtime. This is useful if you write your own code generators/editors etc., and you wish to add additional information specific to your tool (such as a help text)." (from source).

The *Attributes* used in WAMP IDL are defined in `<WAMP API Catalog>/src/wamp.fbs`, and are described in the following sections:

- `arg`, `kwarg`
- `wampid`
- `wampname`, `wampname_s`
- `wampuri`, `wampuri_s`, `wampuri_p`, `wampuri_sp`, `wampuri_pp`, `wampuri_spp`
- `uuid`
- `ethadr`
- `type`

WAMP Positional and Keyword-based Payloads

Positional payloads `args|list` and keyword-based payloads `kwargs|dict` are table elements that have one of the following *Attributes*:

- `arg` (default)

- `kwarg`

One pair of `args` and `kwarg` types is declared by one FlatBuffer table with optional attributes on table fields, and the following rules apply or must be followed:

1. If neither `arg` nor `kwarg` attribute is provided, `arg` is assumed.
2. Only one of either `arg` or `kwarg` MUST be specified.
3. When a field has an attribute `kwarg`, all subsequent fields in the same table MUST also have attribute `kwarg`.

WAMP IDs and URIs

Integers which contain WAMP IDs use *Attribute*

1. `wampid`: WAMP ID, that is an integer `[1, 2^53]`

Strings which contain WAMP names ("URI components"), for e.g. WAMP roles or authids use *Attributes*

2. `wampname`: WAMP URI component (aka "name"), loose rules (minimum required to combine to dotted URIs), must match regular expression `^[^\s\.#]+$`.
3. `wampname_s`: WAMP URI component (aka "name"), strict rules (can be used as identifier in most languages), must match regular expression `^[\da-z_]+$`.

Strings which contain WAMP URIs or URI patterns use *Attribute*

4. `wampuri`: WAMP URI, loose rules, no empty URI components (aka "concrete or fully qualified URI"), must match regular expression `^([^\s\.#]+\.)*([^\s\.#]+)$`.
5. `wampuri_s`: WAMP URI, strict rules, no empty URI components, must match regular expression `^([\da-z_]+\.)*([\da-z_]+)$`.
6. `wampuri_p`: WAMP URI or URI (prefix or wildcard) pattern, loose rules (minimum required to combine to dotted URIs), must match regular expression `^(([^\s\.#]+\.)|\.)*([^\s\.#]+)?$`.
7. `wampuri_sp`: WAMP URI or URI (prefix or wildcard) pattern, strict rules (can be used as identifier in most languages), must match regular expression `^(([\da-z_]+\.)|\.)*([\da-z_]+)?$`.
8. `wampuri_pp`: WAMP URI or URI prefix pattern, loose rules (minimum required to combine to dotted URIs), must match regular expression `^([^\s\.#]+\.)*([^\s\.#]*)$`.
9. `wampuri_spp`: WAMP URI or URI prefix pattern, strict rules (can be used as identifier in most languages), must match regular expression `^([\da-z_]+\.)*([\da-z_]*)$`.

Type/Object UUIDs

Types and generally any objects can be globally identified using UUIDs [RFC4122]. UUIDs can be used in WAMP IDL using the `uuid` *Attribute*.

```
/// UUID (canonical textual representation).
my_field1: string (uuid);

/// UUID (128 bit binary).
my_field2: uint128_t (uuid);
```

The `uint128_t` is a struct type defined as

```
/// An unsigned integer with 128 bits.
struct uint128_t {
    /// Least significand 32 bits.
    w0: uint32;

    /// 2nd significand 32 bits.
    w1: uint32;

    /// 3rd significand 32 bits.
    w2: uint32;

    /// Most significand 32 bits.
    w3: uint32;
}
```

Ethereum Addresses

Ethereum addresses can be used to globally identify types or generally any object where the global ID also needs to be conflict free, consensually shared and owned by a respective Ethereum network user. Ethereum addresses can be used in WAMP IDL using the `ethadr` *Attribute*:

```
/// Ethereum address (checksummed HEX encoded address).
my_field1: string (ethadr);

/// Ethereum address (160 bit binary).
my_field2: uint160_t (ethadr);
```

The `uint160_t` is a struct type defined as

```
/// An unsigned integer with 160 bits.
struct uint160_t {
    /// Least significand 32 bits.
    w0: uint32;

    /// 2nd significand 32 bits.
    w1: uint32;

    /// 3rd significand 32 bits.
    w2: uint32;

    /// 4th significand 32 bits.
    w3: uint32;

    /// Most significand 32 bits.
    w4: uint32;
}
```

WAMP Actions or Service Elements

The type of WAMP service element **procedure**, **topic** or **interface** is designated using the *Attribute*

1. `type`: one of `"procedure"`, `"topic"` or `"interface"`

The `type` *Attribute* can be used to denote WAMP service interfaces, e.g. continuing with above WAMP Meta API procedure example, the `wamp.session.kill_by_authid` procedure can be declared like this:

```
rpc_service IWampMeta(type: "interface",
                      uuid: "88711231-3d95-44bc-9464-58d871dd7fd7",
                      wampuri: "wamp")
{
    session_kill_by_authid (SessionKillByAuthid): WampIds (
        type: "procedure",
        wampuri: "wamp.session.kill_by_authid"
    );
}
```

The value of attribute `type` specifies a WAMP *Procedure*, and the call arguments and result types of the procedure are given by:

- `SessionKillByAuthid`: procedure call arguments `args` (positional argument) and `kwargs` (keyword arguments) call argument follow this type
- `WampIds`: procedure call results `args` (positional results) and `kwargs` (keyword results)

The procedure will be registered under the WAMP URI `wamp.session.kill_by_authid` on the respective realm.

16.1.3. WAMP Service Declaration

WAMP services include

- *Procedures* registered by *Callees*, available for calling from *Callers*
- *Topics* published to by *Publishers*, available for subscribing by *Subscribers*

We map the two WAMP service types to FlatBuffers IDL using the *Attribute* `type == "procedure" | "topic"` as in this example:

```
rpc_service IWampMeta(type: "interface",
                      uuid: "88711231-3d95-44bc-9464-58d871dd7fd7",
                      wampuri: "wamp")
{
    session_kill_by_authid (SessionKillByAuthid): WampIds (
        type: "procedure",
        wampuri: "wamp.session.kill_by_authid"
    );

    session_on_leave (SessionInfo): Void (
        type: "topic",
        wampuri: "wamp.session.on_leave"
    );
}
```

When the procedure `wamp.session.kill_by_authid` is called to kill all sessions with a given `authid`, the procedure will return a list of WAMP session IDs of the killed sessions via `WampIds`. Independently, meta events on topic `wamp.session.on_leave` are published with detailed `SessionInfo` of the sessions left as event payload. This follows a common "do-something-and-notify-observers" pattern for a pair of a procedure and topic working together.

The *Interface* then collects a number of *Procedures* and *Topics* under one named unit of `type == "interface"` which includes a UUID in an `uuid` *Attribute*.

Declaring Services

Declaring services involves three element types:

- *Topics*
- *Procedures*
- *Interfaces*

The general form for declaring *Topics* is:

```
<TOPIC-METHOD> (<TOPIC-PAYLOAD-TABLE>): Void (
    type: "topic",
    wampuri: <TOPIC-URI>
);
```

The application payload transmitted in EVENTs is typed via `<TOPIC-PAYLOAD-TABLE>`. The return type MUST always be `Void`, which is a dummy marker type declared in `wamp.fbs`.

> Note: With *Acknowledge Event Delivery* (future), when a *Subscriber* receives an EVENT, the *Subscriber* will return an *Event-Acknowledgement* including `args`/`kwargs`. Once we do have this feature in WAMP PubSub, the type of the *Event-Acknowledgement* can be specified using a non-`Void` return type.

The general form for declaring *Procedures* is:

```
<PROCEDURE-METHOD> (<CALL-PAYLOAD-TABLE>): <CALLRESULT-PAYLOAD-TABLE> (
    type: "procedure",
    wampuri: <PROCEDURE-URI>
);
```

The application payload transmitted in CALLs is typed via `<CALL-PAYLOAD-TABLE>`. The return type of the CALL is typed via `<CALLRESULT-PAYLOAD-TABLE>`.

The general form for declaring *Interfaces*, which collect *Procedures* and *Topics* is:

```
rpc_service <INTERFACE> (
    type: "interface",
    uuid: <INTERFACE-UUID>,
    wampuri: <INTERFACE-URI-PREFIX>
) {
    /// Method declarations of WAMP Procedures and Topics
}
```

> Note: We are reusing FlatBuffers IDL here, specifically the `rpc_service` service definitions which were designed for gRPC. We reuse this element to declare both WAMP *Topics* and *Procedures* by using the `type` Attribute. Do not get confused with "rpc" in `rpc_service`.

Declaring Progressive Call Results

Write me.

Declaring Call Errors

Write me.

16.2. Interface Catalogs

Collections of types defined in FlatBuffers IDL are bundled in *Interface Catalogs* which are just ZIP files with

- one catalog.yaml file with catalog metadata

- one or more `*.bfbs` compiled FlatBuffer IDL schemas

and optionally

- schema source files
- image and documentation files

16.2.1. Catalog Archive File

The contents of an `example.zip` interface catalog:

```
unzip -l build/example.zip
Archive:  build/example.zip
  Length      Date    Time    Name
---------  ---------- -----   ----
        0  1980-00-00 00:00   schema/
    14992  1980-00-00 00:00   schema/example2.bfbs
    15088  1980-00-00 00:00   schema/example4.bfbs
    13360  1980-00-00 00:00   schema/example3.bfbs
     8932  1980-00-00 00:00   schema/example1.bfbs
     6520  1980-00-00 00:00   schema/wamp.bfbs
     1564  1980-00-00 00:00   README.md
        0  1980-00-00 00:00   img/
    13895  1980-00-00 00:00   img/logo.png
     1070  1980-00-00 00:00   LICENSE.txt
     1288  1980-00-00 00:00   catalog.yaml
---------                     -------
    76709                     11 files
```

The bundled Catalog Interfaces in above are FlatBuffers binary schema files which are compiled using `flatc`

```
flatc -o ./schema --binary --schema --bfbs-comments --bfbs-builtins ./src
```

from FlatBuffers IDL sources, for example:

```
rpc_service IExample1 (
    type: "interface", uuid: "bf469db0-efea-425b-8de4-24b5770e6241"
) {
    my_procedure1 (TestRequest1): TestResponse1 (
        type: "procedure", wampuri: "com.example.my_procedure1"
    );

    on_something1 (TestEvent1): Void (
        type: "topic", wampuri: "com.example.on_something1"
    );
}
```

16.2.2. Catalog Metadata

The `catalog.yaml` file contains catalog metadata in YAML Format:

Field	Description
`name`	Catalog name, which must contain only lower-case letter, numbers, hyphen and underscore so the catalog name can be used in HTTP URLs
`version`	Catalog version (e.g. semver or calendarver version string)
`title`	Catalog title for display purposes
`description`	Catalog description, a short text describing the API catalog
`schemas`	FlatBuffers schemas compiled into binary schema reflection format
`author`	Catalog author
`publisher`	Ethereum Mainnet address of publisher
`license`	SPDX license identifier (see https://spdx.org/licenses/) for the catalog
`keywords`	Catalog keywords to hint at the contents, topic, usage or similar of the catalog
`homepage`	Catalog home page or project page
`git`	Git source repository location
`theme`	Catalog visual theme

Table 18

Here is a complete example:

```
name: example

version: 22.6.1

title: WAMP Example API Catalog

description: An example of a WAMP API catalog.

schemas:
  - schema/example1.bfbs
  - schema/example2.bfbs
  - schema/example3.bfbs
  - schema/example4.bfbs

author: typedef int GmbH

publisher: "0x60CC48BFC44b48A53e793FE4cB50e2d625BABB27"

license: MIT

keywords:
  - wamp
  - sample

homepage: https://wamp-proto.org/

git: https://github.com/wamp-proto/wamp-proto.git

theme:
  background: "#333333"
  text: "#e0e0e0"
  highlight: "#00ccff"
  logo: img/logo.png
```

16.2.3. Catalog Sharing and Publication

Archive File Preparation

The ZIP archive format and tools, by default, include filesystem and other metadata from the host producing the archive. That information usually changes, per-archive run, as e.g. the current datetime is included, which obviously progresses.

When sharing and publishing a WAMP Interface Catalog, it is crucial that the archive only depends on the actual contents of the compressed files.

Removing all unwanted ZIP archive metadata can be achieved using stripzip:

```
stripzip example.zip
```

The user build scripts for compiling and bundling an Interface Catalog ZIP file MUST be repeatable, and only depend on the input source files. A build process that fulfills this requirement is called Reproducible build.

The easiest way to check if your build scripts producing `example.zip` is reproducible is repeat the build and check that the file fingerprint of the resulting archive stays the same:

```
openssl sha256 example.zip
```

Catalog Publication on Ethereum and IPFS

Write me.

16.3. Interface Reflection

Feature status: **sketch**

Reflection denotes the ability of WAMP peers to examine the procedures, topics and errors provided or used by other peers.

I.e. a WAMP *Caller*, *Callee*, *Subscriber* or *Publisher* may be interested in retrieving a machine readable list and description of WAMP procedures and topics it is authorized to access or provide in the context of a WAMP session with a *Dealer* or *Broker*.

Reflection may be useful in the following cases:

- documentation
- discoverability
- generating stubs and proxies

WAMP predefines the following procedures for performing run-time reflection on WAMP peers which act as *Brokers* and/or *Dealers*.

Predefined WAMP reflection procedures to *list* resources by type:

```
    wamp.reflection.topic.list
    wamp.reflection.procedure.list
    wamp.reflection.error.list
```

Predefined WAMP reflection procedures to *describe* resources by type:

```
    wamp.reflection.topic.describe
    wamp.reflection.procedure.describe
    wamp.reflection.error.describe
```

A peer that acts as a *Broker* SHOULD announce support for the reflection API by sending

```
    HELLO.Details.roles.broker.reflection|bool := true
```

A peer that acts as a *Dealer* SHOULD announce support for the reflection API by sending

```
HELLO.Details.roles.dealer.reflection|bool := true
```

> Since *Brokers* might provide (broker) procedures and *Dealers* might provide (dealer) topics, both SHOULD implement the complete API above (even if the peer only implements one of *Broker* or *Dealer* roles).

Reflection Events and Procedures

A topic or procedure is defined for reflection:

```
wamp.reflect.define
```

A topic or procedure is asked to be described (reflected upon):

```
wamp.reflect.describe
```

A topic or procedure has been defined for reflection:

```
wamp.reflect.on_define
```

A topic or procedure has been undefined from reflection:

```
wamp.reflect.on_undefine
```

17. Router-to-Router Links

Write me.

1. Resolve global realm name R_name via ENS to the on-chain address R_adr of the realm.
2. Retrieve list of Domains R_DR routing realm R_adr.
3. Retrieve the node's N1 own domain D_N1 given the node's address N1_adr.
4. Check D_N1 is in R_DR.
5. Select a domain D (!=D_N1) from R_DR and get endpoint E for D.
6. Connect to D and authenticate via WAMP-Cryptosign.
7. Verify connected node N2 by checking against D
8. Subscribe to `wamp.r2r.traffic_payable`
9. When receiving a traffic payable event, buy the respective key by calling `xbr.pool.buy_key`, and calling `wamp.r2r.submit_traffic_payment`, which returns a traffic usage report.

Data Spaces are end-to-end encrypted routing realms connecting data driven microservices.

The message routing between the microservice endpoints in

18. Advanced Profile URIs

WAMP pre-defines the following error URIs for the **Advanced Profile**. WAMP peers SHOULD only use the defined error messages.

18.1. Session Close

The *Client* session has been forcefully terminated by the *Router* - used as a `GOODBYE` (or `ABORT`) reason.

```
wamp.close.killed
```

18.2. Authentication

No authentication method the *Client* offered is accepted.

```
wamp.error.no_matching_auth_method
```

The *Client* attempted to authenticate for a non-existing *Realm* (`realm|string`).

```
wamp.error.no_such_realm
```

The *Client* attempted to authenticate for a non-existing *Role* (`authrole|string`).

```
wamp.error.no_such_role
```

The *Client* authenticated for a non-existing *Principal* (`authid|string`).

```
wamp.error.no_such_principal
```

The authentication as presented by the *Client* is denied (e.g. "wrong password").

```
wamp.error.authentication_denied
```

The *Client* authentication was rejected due to a technical runtime failure ("fail secure" operation).

```
wamp.error.authentication_failed
```

The *Client* did not provide the required, non-anonymous, authentication information.

```
wamp.error.authentication_required
```

18.3. Authorization

The *Principal* is not authorized to perform such *Action*.

```
wamp.error.authorization_denied
```

Authorization of the *Principal* to perform the given *Action* was rejected due to a technical runtime failure ("fail secure" operation).

```
wamp.error.authorization_failed
```

Authorization of the *Principal* is required to perform the given *Action*. This can be used for capability-based access control.

```
wamp.error.authorization_required
```

18.4. Remote Procedure Calls

A *Dealer* or *Callee* terminated a call that timed out

```
wamp.error.timeout
```

A *Peer* requested an interaction with an option that was disallowed by the *Router*

```
wamp.error.option_not_allowed
```

A *Router* rejected client request to disclose its identity

```
wamp.error.option_disallowed.disclose_me
```

A *Router* encountered a network failure

```
wamp.error.network_failure
```

A *Callee* is not able to handle an invocation for a *call* and intends for the *Router* to re-route the *call* to another fitting *Callee*. For details, refer to RPC Call Rerouting

```
wamp.error.unavailable
```

A *Dealer* could not perform a call, since a procedure with the given URI is registered, but all available registrations have responded with `wamp.error.unavailable`

```
wamp.error.no_available_callee
```

A *Dealer* received a `CALL` message with advanced features that cannot be processed by the *Callee*

```
wamp.error.feature_not_supported
```

18.5. Terminology

This chapter contains a list of technical terms used in this specification, along with their respective meanings.

Implementations SHOULD use terms as defined here in their public interfaces and documentation, and SHOULD NOT reinvent or reinterpret terms. Users SHOULD be able to transfer their WAMP knowledge from one implementation to another. This is to support the overarching goal of WAMP to free application developers from restrictions when building distributed applications, both at the network level, and when choosing (or switching) the WAMP implementations used.

Our goal is to maximize **user choice and experience** when developing WAMP-based applications, both formally (open protocol and open source) as well as practically (switching costs).

18.5.1. Fundamental

Term	Definition
User	A person (or organization) running a WAMP *Client* or *Router*
Client	A program run by a *User*, with application code using WAMP for application-level communications
Router	A program run by a *User*, with middleware code using WAMP to provide application routing services
Peer	A WAMP *Client* or *Router*. An implementation might embed, provide or use both roles
Realm	Isolated WAMP URI namespace serving as a routing and administrative domain, optionally protected by **AA**
Transport	A message-based, reliable, ordered, bidirectional (full-duplex) channel over which *Peers* communicate
Connection	An underlying entity (if any) carrying the *Transport*, e.g. a network connection, pipe, queue or such
Session	Transient conversation between a *Client* and a *Router*, within a *Realm* and over a *Transport*
Message	Indivisible unit of information transmitted between peers
Serializer	Encodes WAMP messages, with application payloads, into byte strings for transport

Table 19

18.5.2. Authentication and Authorization (AA)

Term	Definition
Authentication	Establishes the identity of a *Session* within a *Realm*
Principal	A *Principal* (`authid`) is any *User* that can be authenticated under a *Realm* (`realm`) and runs in the security context of a *Role* (`authrole`) within that *Realm*.
Credentials	The authentication information and secrets used during
Authorization	A decision on permitting a *Principal* to perform a given *Action* on an *URI or URI pattern*
Access Control	Policy for selective restriction of *Actions* on *URIs or URI patterns* performed by *Principals*
Role-based Access Control (RBAC)	An *Access Control* policy based on *Realm* (`realm`), *Principal*'s *Role* (`authrole`), *URI or URI pattern*, and *Action*
Discretionary Access Control	An *Access Control* policy controlled by *Users* and enforced by *Routers*
Mandatory Access Control	An *Access Control* policy controlled by *Router Administrators* or *Realm Owners*, and enforced by *Routers*
Capability-based Access Control	An *Access Control* policy where *Callers*, *Callees*, *Publishers*, *Subscribers* directly share capabilities with each other
Subject	The originating *Session* of an *Action* in the context of *Authorization*
Object	A (fully qualified) *URI or URI pattern* representing the target of an *Action* in the context of *Authorization*
Action	One of the four core WAMP operations: **register**, **call**, **subscribe**, and **publish**

Table 20

18.5.3. Remote Procedure Calls

Term	Definition
Caller	A *Caller* is a *Session* that **calls**, with application payloads, a (fully qualified) *Procedure* for call routing
Callee	A *Callee* is a *Session* that responds to *Procedure* call invocations by **yielding** back application result payloads
Procedure	A *Procedure* is an URI or URI pattern that can be registered for call routing by *Callees*
Registration	A *Router* record resulting from a *Callee* successfully **registering** a *Procedure* for call routing
Call	A transient *Router* record resulting from a *Caller* successfully **calling** a *Procedure* for call routing
Invocation	A call request and payload that are routed to a *Callee* having a matching *Registration* for the called *Procedure*

Table 21

18.5.4. Publish and Subscribe

Term	Definition
Publisher	A *Publisher* is a *Session* that **publishes** application payloads to a (fully qualified) *Topic* for event routing
Subscriber	A *Subscriber* is a *Session* that **subscribes** to a *Topic* to receive application payloads on matching events
Topic	A *Topic* is an URI or URI pattern that can be subscribed to for event routing by *Subscribers*
Subscription	A *Router* record resulting from a *Subscriber* successfully **subscribing** to a *Topic* for event routing
Publication	A transient *Router* record resulting from a *Publisher* successfully **publishing** to a *Topic* for event routing
Event	A publication that is routed to *Subscribers* having matching *Subscriptions* to the published *Topic*.

Table 22

19. IANA Considerations

WAMP uses the Subprotocol Identifier `wamp` registered with the WebSocket Subprotocol Name Registry, operated by the Internet Assigned Numbers Authority (IANA).

20. Conformance Requirements

All diagrams, examples, and notes in this specification are non-normative, as are all sections explicitly marked non-normative. Everything else in this specification is normative.

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in RFC 2119 [RFC2119].

Requirements phrased in the imperative as part of algorithms (such as "strip any leading space characters" or "return false and abort these steps") are to be interpreted with the meaning of the key word ("MUST", "SHOULD", "MAY", etc.) used in introducing the algorithm.

Conformance requirements phrased as algorithms or specific steps MAY be implemented in any manner, so long as the end result is equivalent.

20.1. Terminology and Other Conventions

Key terms such as named algorithms or definitions are indicated like *this* when they first occur, and are capitalized throughout the text.

21. Contributors

WAMP was developed in an open process from the beginning, and a lot of people have contributed ideas and other feedback. Here we are listing people who have opted in to being mentioned:

- Alexander Goedde
- Amber Brown
- Andrew Gillis
- David Chappelle
- Elvis Stansvik
- Emile Cormier
- Felipe Gasper
- Johan 't Hart
- Josh Soref
- Konstantin Burkalev
- Pahaz Blinov
- Paolo Angioletti
- Roberto Requena
- Roger Erens
- Christoph Herzog
- Tobias Oberstein
- Zhigang Wang

22. Normative References

[RFC3629] Yergeau, F., "UTF-8, a transformation format of ISO 10646", STD 63, RFC 3629, DOI 10.17487/RFC3629, November 2003, <https://www.rfc-editor.org/info/rfc3629>.

[RFC3986] Berners-Lee, T., Fielding, R., and L. Masinter, "Uniform Resource Identifier (URI): Generic Syntax", STD 66, RFC 3986, DOI 10.17487/RFC3986, January 2005, <https://www.rfc-editor.org/info/rfc3986>.

[RFC4122] Leach, P., Mealling, M., and R. Salz, "A Universally Unique IDentifier (UUID) URN Namespace", RFC 4122, DOI 10.17487/RFC4122, July 2005, <https://www.rfc-editor.org/info/rfc4122>.

[RFC4648] Josefsson, S., "The Base16, Base32, and Base64 Data Encodings", RFC 4648, DOI 10.17487/RFC4648, October 2006, <https://www.rfc-editor.org/info/rfc4648>.

[RFC6455] Fette, I. and A. Melnikov, "The WebSocket Protocol", RFC 6455, DOI 10.17487/RFC6455, December 2011, <https://www.rfc-editor.org/info/rfc6455>.

[RFC7159] Bray, T., Ed., "The JavaScript Object Notation (JSON) Data Interchange Format", RFC 7159, DOI 10.17487/RFC7159, March 2014, <https://www.rfc-editor.org/info/rfc7159>.

[RFC8032] Josefsson, S. and I. Liusvaara, "Edwards-Curve Digital Signature Algorithm (EdDSA)", RFC 8032, DOI 10.17487/RFC8032, January 2017, <https://www.rfc-editor.org/info/rfc8032>.

[RFC8949] Bormann, C. and P. Hoffman, "Concise Binary Object Representation (CBOR)", STD 94, RFC 8949, DOI 10.17487/RFC8949, December 2020, <https://www.rfc-editor.org/info/rfc8949>.

23. Informative References

[RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, DOI 10.17487/RFC2119, March 1997, <https://www.rfc-editor.org/info/rfc2119>.

Author's Address

Tobias Oberstein
typedef int GmbH
Email: tobias.oberstein@typedefint.eu